



KONGUNADU COLLEGE OF ENGINEERING AND TECHNOLOGY

(Autonomous)

Approved by AICET, New Delhi & Affiliated to Anna University, Chennai
Accredited by NBA (CSE, ECE, EEE, Mech), NAAC,
Recognized by UGC with 2(f) & 12(B) An ISO 9001:2005 Certified Institution
Namakkal - Trichy Main Road,
Tholurpatti (post), Thottiam, Trichy – 621215, Tamil Nadu

DEPARTMENT OF ARTIFICIAL INTELLIGENCE AND DATA SCIENCE

LECTURE HANDOUTS

Course Name with Code : 24CS301- DATA STRUCTURES

Course Faculty : Dr.M.SENTHILKUMAR

Batch : 2024-2028

Year / Semester / Section : II / 03 /

COURSE CONTENT

S.NO	DATE	PERIOD	TOPICS COVERED	PAGE NO
UNIT I ABSTRACT DATA TYPES				
1			Abstract Data Types (ADTs)	
2			ADTs and classes	
3			Introduction to OOP, Classes in Python	
4			Inheritance, namespaces	
5			shallow and deep copying	
6			Introduction to analysis of algorithms	
7			Asymptotic notations	
8			Recursion	
9			Analyzing recursive algorithms	
UNIT II LINEAR STRUCTURES				
10			List ADT	
11			Array-based implementations	
12			Linked list implementations, Singly linked lists	
13			Circularly linked lists	
14			Doubly linked lists	
15			Applications of lists	
16			Stack ADT	
17			Queue ADT	
18			Double ended queues	

UNIT III SORTING AND SEARCHING				
19			Bubble sort, Selection sort	
20			Insertion sort, Merge sort	
21			Quick sort	
22			Linear search	
23			Binary search	
24			Hashing, Hash functions	
25			Collision handling	
26			load factors and efficiency	
27			Rehashing	
UNIT IV TREE STRUCTURES				
28			Tree ADT	
29			Binary Tree ADT	
30			Tree traversals	
31			Examples for Inorder, Preorder and Postorder	
32			Binary search trees	
33			AVL trees	
34			Heaps	
35			Types of heaps	
36			Multi-way search trees	
UNIT V GRAPH STRUCTURES				
37			Graph ADT	
38			Representations of graph	
39			Graph traversals	
40			DAG	
41			Topological ordering	
42			Shortest paths	
43			Minimum spanning trees	
44			Examples – Prim's and Kruskal algorithms	
45			Disjoint sets	
46			DFS vs BFS Algorithms	

Course Faculty Signature

OBJECTIVES:**The Student should be made to:**

- Understand the concepts of ADTs and implement them using Python.
- Design and implement linear data structures – lists, stacks, and queues.
- Design, analyze, and implement sorting, searching, and hashing algorithms.
- Formulate problems using tree structures and solve them using Python.
- Devise problems using graph data structure and its traversal using Python.

UNIT I ABSTRACT DATA TYPES 9

Abstract Data Types (ADTs) – ADTs and classes – Introduction to OOP – Classes in Python – Inheritance – namespaces – shallow and deep copying. Introduction to analysis of algorithms – Asymptotic notations – Recursion – Analyzing recursive algorithms.

UNIT II LINEAR STRUCTURES 9

List ADT – Array-based implementations – Linked list implementations – Singly linked lists – Circularly linked lists – Doubly linked lists – Applications of lists – Stack ADT – Queue ADT – Double ended queues.

UNIT III SORTING AND SEARCHING 9

Bubble sort – selection sort – insertion sort – merge sort – quick sort – linear search – binary search – hashing – hash functions – collision handling – load factors and efficiency – rehashing

UNIT IV TREE STRUCTURES 9

Tree ADT – Binary Tree ADT – Tree traversals – Binary search trees – AVL trees – Heaps – Multi-way search trees.

UNIT V GRAPH STRUCTURES 9

Graph ADT – Representations of graph – Graph traversals – DAG – Topological ordering – Shortest paths – Minimum spanning trees – Disjoint sets.

TOTAL: 45 PERIODS

OUTCOMES:

On successful completion of this course, the students will be able to,

- Implement ADTs as Python classes.
- Design, implement, and analyze linear data structures, such as lists, queues, and stacks, according to the needs of different applications.
- Design, implement, and analyze sorting, searching, and indexing techniques.
- Design, implement, and analyze efficient tree structures to meet requirements such as searching, indexing, and sorting.
- Model problems as graph problems and implement efficient graph algorithms to solve them.

TEXT BOOKs:

1. Michael T. Goodrich, Roberto Tamassia, and Michael H. Goldwasser, “Data Structures & Algorithms in Python”, John Wiley & Sons Inc., 2013.

REFERENCES:

1. Lee, Kent D., Hubbard, Steve, “Data Structures and Algorithms with Python” Springer Edition, 2015.
2. Rance D. Necaie, “Data Structures and Algorithms Using Python”, John Wiley & Sons, 2011.
3. Aho, Hopcroft, and Ullman, “Data Structures and Algorithms”, Pearson Education, 1983.
4. Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, and Clifford Stein, “Introduction to Algorithms”, Second Edition, McGraw Hill, 2002.
5. Mark Allen Weiss, “Data Structures and Algorithm Analysis in C++”, Fourth Edition, Pearson Education, 2014.

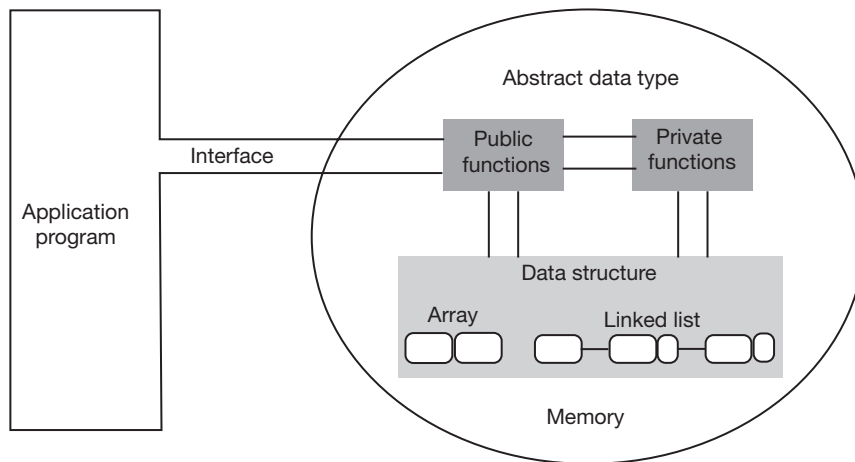
COs – POs Mapping

COURSE OUTCOMES	PO											PSO	
	1	2	3	4	5	6	7	8	9	10	11	1	2
1	2	1	-	-	2	-	-	-	2	-	-	2	3
2	2	2	1	2	2	-	-	-	1	3	-	3	3
3	2	2	1	2	3	-	-	-	-	3	1	3	3
4	2	2	1	2	3	-	-	-	2	3	1	3	3
5	2	2	1	2	3	-	-	-	3		3	3	3
Average	2	1.80	1	2	2.60	-	-	-	2	3	1.67	2.80	3

Abstract Data Types (ADT's)

An Abstract Data Type (ADT) is a theoretical framework that outlines a collection of operations and behaviors for a data structure without detailing how these operations are implemented or how the data is stored in memory.

The concept of revealing only the necessary details while concealing the underlying complexities is known as abstraction.



The ADT model consists of two types of functions: public functions and private functions. It also includes the data structures utilized within a program. Initially, encapsulation is applied, which involves bundling all data into a single unit—the ADT. Following this, abstraction is used to highlight the operations that can be performed on the data structures, as well as to specify which data structures are employed in the program.

1.1 Introduction to OOP

The term “object-oriented” is derived from two words: “object” and “oriented.” An “object” refers to an item or entity that exists in the real world, while “oriented” means being focused on or interested in a specific type of thing or entity.

Object-oriented programming (OOP) is a computer programming methodology that structures software design around data, or objects, rather than emphasizing functions and logic. An object represents a data entity with distinct attributes and behaviors. In OOP, everything is organized as self-contained objects.

This programming model is highly popular among developers and is particularly well-suited for developing large, complex applications that require continuous updates and maintenance.

The main features of OOP includes

1. Objects
2. Classes
3. Data Abstraction
4. Encapsulation
5. Inheritance
6. Polymorphism
7. Dynamic binding
8. Message passing

Object:

Objects are the basic unit of object-oriented programming. Objects are instances of classes that interact by sending messages to each other. There can be more than one instance of a class, each instance of a class can contain its own corresponding data. An object is a collection of data members and associated member functions to perform various tasks.

Class:

A class contains data and related functions contained within it. A class is also known as a collection of similar objects or objects of the same type. Variables declared in a class are called data members and functions are called member function methods (or methods). When a class is created, no memory is allocated.

Data Abstraction:

Data abstraction provides only the necessary information, hiding the background details. (i.e.) C++ classes expose various methods to the outside world without providing internal details about those methods and data. Classes are also called abstract data types because they use the concept of data abstraction.

Encapsulation:

Encapsulation is a way of combining data and functions within a class. The data is not exposed to the outside world; it is only accessible to the member functions encapsulated within the class.

Hiding of data from direct access by the program is called as Information hiding or Data hiding.

Inheritance:

Inheritance is the process of forming a new class by acquiring or inheriting the properties of objects of base class or existing class. A base class is also called a parent class or superclass. A derived class is called a derived class (or) child class.

Polymorphism:

Polymorphism is the ability of an object or reference to take on different forms in different instances.

There are two types of polymorphism.

- **Compile-time polymorphism:** The object is bound to the function call at compile time.
- **Runtime polymorphism:** Objects are bound to function calls only at run time.

Dynamic linking:

Dynamic binding refers to the process of associating a procedure call with procedural code that is executed only at run time. This is also known as late binding.

Messaging:

Message passing means that objects send and receive information by following these steps:

- Create classes that define objects and their behavior.
- Create objects from class definitions.
- Establish connections between objects.

Benefits of OOPS:

- Modularity
- Reusability
- Scalability and Maintainability
- Data Security
- Improved Productivity
- Real-World Modelling

Applications of OOP:

- Real time system
- Simulation and Modelling
- User Interface Design
- Object Oriented Databases
- AI and Expert Systems

1.2 Classes in Python

A class in Python is a custom blueprint designed to create objects. It groups related data and functions together, simplifying their organization and usage. Defining a class introduces a new object type, and multiple instances of this type can be created as needed.

Defining a Class:

Classes are created using **class keyword**. Attributes are variables defined inside the class and represent the properties of the class.

Syntax:

```
class classname:
```

Example: class dog:

Creating an Object:

An Object is an instance of a Class. It represents a specific implementation of the class and holds its own data.

Syntax:

```
Object name= class name()
```

Example:

```
Dog_brand=dog()
```

Accessing class attributes:

Attributes of class can be accessed using the dot **. operator**.

Syntax:

```
Objectname.attribute name
```

Example:

```
Dog_brand.name
```

Using __init__() Function

In Python, class has __init__() function. It automatically initializes object attributes when an object is created.

Example:

class Dog:	---👤 Creating a class
species = "Canine"	---👤 class attributes
def __init__(self, name, age):	---👤 init_method
self.name = name	
self.age = age	

```
dog1 = Dog("Buddy", 3)    ---Ⓜ creating an object
print(dog1.name)          ---Ⓜ Accessing a class object
print(dog1.species)
```

1.3 Inheritance

A key idea in object-oriented programming (OOP) is the process of inheritance, which allows a class (referred to as a child or inherited class) to inherit the properties and methods of another class (referred to as a parent or base class). This encourages modularity, reuse of code, and hierarchical structure of classes.

Creating a Parent Class:

A parent class, often known as a base class in object-oriented programming, specifies shared methods and characteristics that other classes can inherit. Child classes are built on top of these properties and methods. Child classes can access and expand the functionality offered by a parent class through inheritance.

Creating a Child Class

Any class that inherits properties and methods from its original class is referred to as a child class, or subclass. It is also possible for the child class to introduce new attributes and methods, or to override those that were inherited from the parent.

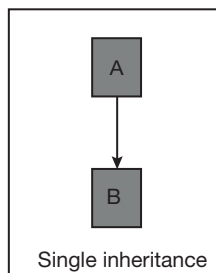
Types of Inheritance

1. Single inheritance: Children's classes inherit their parents' classes.
2. Multiple Inheritance: A child class inherits from more than one parent class.
3. Multilevel inheritance: A class is derived from a class which is also derived from another class.
4. Hierarchical inheritance: multiple classes inherit from a single parent class.
5. Hybrid inheritance: A combination of several types of inheritance.

Single inheritance:

This is the simplest form of inheritance:

It enables a derived class to inherit properties from a parent class, enabling code reuse and introducing additional functionality to existing code



Program:

```

class parent:
    def add(self):
        print("hai");
class child(parent):
    def sub(self):
        print("hello");
obj=child()
obj.sub();
obj.add();

```

Output:

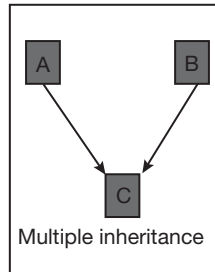
```

hai
hello

```

Multiple Inheritance:

With multiple inheritances of Python, you can build classes based on multiple parent classes. Therefore, the child's class inherits attributes and methods from all parents. A child element can override any method inherited from its parent.

**Program:**

```

class parent:
    def add(self):
        print("hai");
class parent1:
    def sub(self):
        print("hello");
class child(parent,parent1):
    def mul(self):
        print("bye");
obj=child()
obj.sub();
obj.add();
obj.mul();

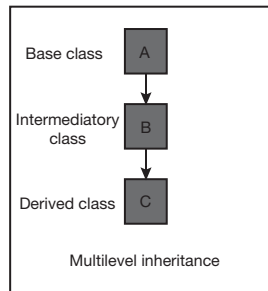
```

Output:

```
hai
hello
bye
```

Multilevel Inheritance:

In multilevel inheritance, the functionality of base and derived classes is passed down to more recently derived classes, similar to the child-grandfather relationship. This is similar to a relationship representing a child and a grandfather.

**Program:**

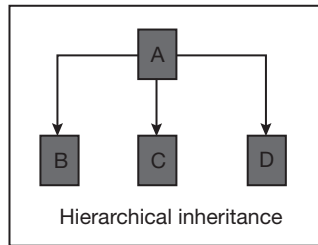
```
class parent:
    def add(self):
        print("hai");
class parent1(parent):
    def sub(self):
        print("hello");
class child(parent1):
    def mul(self):
        print("bye");
obj=child()
obj.sub();
obj.add();
obj.mul();
```

Output:

```
hai
hello
bye
```

Hierarchical Inheritance:

When multiple derived classes are created from a single base class, this type of inheritance is called hierarchical inheritance. In this diagram, there is one parent (base) class and two child (derived) classes.

**Program:**

```

class parent:
    def add(self):
        print("hai");
class parent1(parent):
    def sub(self):
        print("hello");
class child(parent):
    def mul(self):
        print("bye");
obj=child()
obj1.parent()
obj.sub();
obj.add();
obj1.mul();
  
```

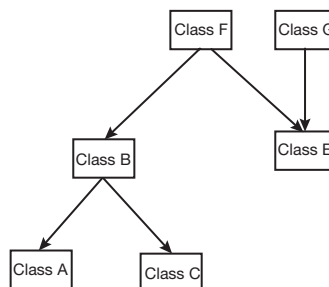
Output:

```

hai
hello
bye
  
```

Hybrid Inheritance:

Inheritance consisting of multiple types of inheritance is called hybrid inheritance.



1.4 Namespaces in Python

Definition: Python Namespaces are collections of different objects that are associated with unique names whose lifespan depends on the scope of a variable. Namespaces are used to organize a program's symbols by putting them into different contexts. The scope is a region from where we can access a particular object.

Scope of a Variable/Object in Python:

“Scope” in Python describes how far a program's variables can be seen and accessed. Where in the code a variable can be used or accessed is defined by its scope.

There are two types of scope in Python:

- Global Scope
- Local Scope.

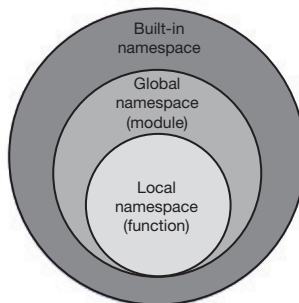
A global variable is one that is defined outside of any function or class and is accessible from anywhere in the program.

A local variable, on the other hand, is one that is defined within a function or class and is only accessible within that function or class.

Types of Namespaces in Python:

Python allows for the organisation of symbols using a variety of namespace types:

- All of the symbols declared at the top level of the module or script are contained in the world-wide namespace.
- The namespace formed when a function or method is defined is known as the local namespace. It includes all of the symbols defined in the method or function.
- Module namespace: All of the symbols declared in the module are contained in this namespace.
- **Built-in namespace:** This is a predefined namespace that contains all the built-in symbols that are available in Python, such as int, str, list, and so on.



Global namespace:

In Python, the global namespace is the namespace that contains all the symbols that are defined at the top level of a module or script. These symbols can be accessed from anywhere in the module or script, and are said to be in the global namespace.

In this example, the *func* and *x* defined at the top level of the script are in the global namespace. They can be accessed from anywhere in the script, including from within the *main* function

```
# define a function in the global namespace
def func():
    print("I am a function in the global namespace")
# define a variable in the global namespace
x = 10
def main():
    # access the function and variable in the global namespace
    func() # prints "I am a function in the global namespace"
    print(x) # prints 10
main()
```

Local namespace:

In Python, a local namespace is a namespace that is created whenever you define a function or method. It contains all the symbols that are defined within the function or method and are said to be local to that function or method.

In this example, the *func* and *x* defined inside the *main* function are in the local namespace. They can only be accessed from within the *main* function, and are not visible outside of it.

```
def main():
    # This is an example of the local namespace
    # define a function in the local namespace
    def func():
        print("I am a function in the local namespace")
        # define a variable in the local namespace
        x = 20
        # use the function and variable in the local namespace
    func() # prints "I am a function in the local namespace"
    print(x) # prints 20
main()
```

Module namespace:

A module namespace in Python is a namespace that includes every symbol specified in a module. All of the symbol in the module's namespace are made available to the caller upon import; however, they are inserted in the module's namespace instead of the caller's.

This helps to prevent the caller from accidentally overwriting symbols that are defined in the module.

```
# define a function in the module namespace
def func():
    print("I am a function in the module namespace")
# define a variable in the module namespace
x = 10
```

Built-in namespace:

In Python, the built-in namespace is a predefined namespace that contains all the built-in symbols that are available in Python. These symbols include functions, classes, and variables that are defined in the Python Standard Library and are always available to use in your Python programs.

Here are a few examples of symbols that are defined in the built-in namespace:

- **int:** This is the class for representing integers.
- **str:** This is the class for representing strings.
- **list:** This is the class for representing lists.
- **print:** This is a function that prints the specified objects to the output stream.
- **len:** This is a function that returns the length of an object.

You can access these symbols simply by using their names, without having to import them from any module. For example, you can create a string by calling *str()*, create a list by calling *list()*, and print to the console by calling *print()*.

Used: Namespaces are employed to organise code to logical categories and to prevent name collisions, which are particularly likely to occur when your source code contains multiple libraries. No qualification is needed for the visibility of all identifiers within the namespace scope.

Types: When creating a namespace, you must choose one of two namespace types: a stand-alone namespace or a domain-based namespace. In addition, if you choose a domain-based namespace, you must choose a namespace mode: Windows 2000 Server mode or Windows Server 2008 mode.

Purpose: Namespace provide the space where we can define or declare identifier i.e. variable, method, classes. Using namespace, you can define the space or context in which identifiers are defined i.e. variable, method, classes. In essence, a namespace defines a scope.

Stored: Active Directory Domain Services (AD DS) stores domain-based namespace configuration data in one or more objects that contain namespace server names, folder targets, and various other configuration data. The namespace servers maintain shares for each namespace hosted.

Creating a namespace using a YAML file

- `apiVersion` specifies the version of the Kubernetes API.
- `kind` specifies the type of Kubernetes resource.
- `metadata` specifies metadata about the resource.
- `name` specifies the name of the resource.

- `labels` specifies the key-value pairs which you want to assign to the resource.

How namespaces work:

- Namespaces are implemented as Python dictionaries.
- Each object in Python has a unique name in a namespace.
- Namespaces exist at different levels, including local, global, and built-in.
- The scope of a variable determines how long it lives.
- The scope of a variable also determines where it can be accessed.
- Namespaces ensure that names are unique within a certain scope.

Examples of namespaces

- **Built-in namespace:** Contains built-in functions like `abs()` and exception names
- **Global namespace:** Contains names defined at the module level
- **Local namespace:** Contains names defined within a function

Using namespaces

- Namespaces help the Python interpreter understand which variable or method is being referenced in the code.
- Namespaces help prevent accidental overwriting of symbols.
- To avoid name shadowing, it's recommended to use unique and descriptive names for variables and functions.

Advantages:

Resolving naming conflicts is the main benefit of namespaces. For instance, there are instances when you may need multiple functions that share the same name. In order to avoid confusing the compiler, namespaces allow you to define these kinds of functions.

Disadvantages:

Data and dependencies are notoriously hard to replicate reliably, which is the main downside. The complexity increases when there are numerous namespaces, as each namespace requires its own datastore instance.

Example: In an operating system, an example of namespace is a directory. A single file or subdirectory is uniquely identified by each name in a directory. In general, names within a namespace aren't permitted to have more than one meaning; that is, distinct meanings cannot share the same name within the same namespace.

When there are no user-defined modules, methods, classes, etc. running on the Python interpreter. Some built-in namespace functions, such as `print()` and `id()`, are always available. A global namespace is generated when a user builds a module; a local namespace is then generated when local functions are created. The **built-in namespace** encompasses the **global namespace** and the global namespace encompasses the **local namespace**.

Ex:

```
# var1 is in the global namespace
var1 = 5
defsome_func():
    # var2 is in the local namespace
    var2 = 6
    defsome_inner_func():
        # var3 is in the nested local
        # namespace
        var3 = 7
```

1.5 Shallow and Deep Copy in Python

Shallow Copy: If you change a changeable object in the object that you shallow copied, the original object will also change because they share the same reference.

Deep Copy: Changes made to changeable objects inside the deep copied object don't affect the source entity because each object is fully copied and stands alone.

There are functions in the copy module that let you create shallow and deep copies of objects:

- Shallow Copy: To make a shallow copy, use `copy.copy()`.
- Deep Copy: To make a deep copy, use `copy.deepcopy()`.

In Python, assignment commands are not copying an object; instead, they make a reference to it. Python has a copy module that lets you make real copies and has tools for **shallow** (`copy.copy()`) and **deep** (`copy.deepcopy()`) copies.

Deep copy in Python

- Before inserting copies of the items discovered in the original into the new compound object, a deep copy creates another object in a recursive way.
- After creating a new collections object, it will recursively add copy of the original's child objects to it. It indicates that modifications made in an object's copy do not affect the original.

Syntax of Python Deepcopy

```
copy.deepcopy()
```

Example:

```
import copy
a = [[1, 2, 3], [4, 5, 6]]
# Creating a deep copy of the nested list 'a'
b = copy.deepcopy(a)
# Modifying an element in the deep-copied list
```

```
b[0][0] = 99
print(b)
```

Output

```
[[99, 2, 3], [4, 5, 6]]
```

Shallow copy in Python

- A shallow copy generates another object while maintaining references to the objects that have been present in the original. It exclusively duplicates the top-level structure, excluding nested elements.
- Modifications to a copy of an object are reflected in the original object.

Syntax of Python Shallowcopy

```
copy.copy()
```

Example:

```
import copy
a = [[1, 2, 3], [4, 5, 6]]
# Creating a shallow copy of the nested list 'original'
b = copy.copy(a)
# Modifying an element in the shallow-copied list
b[0][0] = 99
# Printing the original and shallow-copied lists
print(b)
```

Output

```
[[99, 2, 3], [4, 5, 6]]
```

Only compound objects (those containing other objects, such as lists or class instances) can benefit from the distinction between deep and shallow copying:

- To create another compound object, a shallow copy will utilise the original's objects as references, referencing them as much as possible.
- To create a new compound object, the deep copy first takes the original objects and uses a recursive process to insert copies of each object into it.

Deep copy operations frequently have two issues that shallow copy operations do not:

- Recursive loops can be caused by recursive objects, which are compound objects that either directly or indirectly contain references to themselves.
- Deep copy may copy too much since it copies every aspect, including data that should be shared across copies.

Example1:

```
list1 = [[1, 2, 3], [4, 5, 6], [7, 8, 'a']]
```

```
list2 = list1
list2[1][2] = 4
print('Old List:', list1)
print('ID of Old List:', id(list1))
print('New List:', list2)
print('ID of New List:', id(list2))
```

Output:

```
Old List: [[1, 2, 3], [4, 5, 4], [7, 8, 'a']]
ID of Old List: 1909447368968
New List: [[1, 2, 3], [4, 5, 4], [7, 8, 'a']]
ID of New List: 1909447368968
```

Example 2:

```
import copy
list1 = [[1, 2, 3], [4, 5, 6], [7, 8, 9], [10, 11, 12]]
list2 = copy.copy(list1)
list1.append([13, 14, 15])
print("Old list:", list1)
print("New list:", list2)
```

Output:

```
Old list: [[1, 2, 3], [4, 5, 6], [7, 8, 9], [10, 11, 12], [13, 14, 15]]
New list: [[1, 2, 3], [4, 5, 6], [7, 8, 9], [10, 11, 12]]
```

1.6 Introduction to Analysis of Algorithms

Professor Emeritus at Stanford University, Donald Ervin Knuth is an American computer scientist and mathematician. He is the author of the multi-volume work. He has been referred to as the “father” of algorithmic analysis in *The Art of Computer Programming*.

A problem is resolved through a sequential procedure known as an algorithm. A well-designed algorithm should be optimised in terms of both time and space. Different types of challenges require distinct algorithmic techniques to be resolved in the most efficient fashion.

Using data storage, sorting, processing, and machine learning, algorithms determine the optimal solution to an issue. They increase a program's efficiency in its execution. In every aspect of computers, algorithms are utilised. since it's an excellent method of automating computer choices.

Because even the inadvertent application of an inefficient algorithm can have a major influence on system performance, algorithm analysis is crucial in real-world applications. If an algorithm takes too long to execute, the results it produces may be irrelevant or out of date in time-sensitive applications.

Algorithm analysis is the process of comparing algorithms based on the quantity of computational power they consume. We aim to be able to compare two algorithms and determine that one is superior to the other due to its more efficient utilisation of resources or, alternatively, its usage of fewer resources.

Discovering an algorithm's properties to assess its applicability for different applications or to compare it with other algorithms for the same application is the most basic reason to analyse it.

Analysis of Algorithm in Data Structure: Algorithm analysis in data structures involves a systematic evaluation of how algorithms behave concerning time complexity, space complexity, and practical efficiency when manipulating different types of data. **"Analysis of algorithms"** in data structures refers to the process of evaluating and understanding how efficiently an algorithm performs by examining its time and space complexity, essentially determining how well an algorithm scales with increasing input size, allowing developers to choose the best solution for a given problem based on its performance characteristics across different scenarios (best, worst, average case) when working with various data structures.

Key points about analysis of algorithms:

- **Purpose:**

To measure the performance of an algorithm in terms of computational resources (time and memory) required to execute it, particularly as the input size grows.

- **Asymptotic analysis:**

A theoretical method used to analyze algorithms by focusing on the growth rate of the algorithm's time complexity using notations like Big-O, Big-Omega, and Big-Theta, which describe the upper bound, lower bound, and tight bound respectively.

- **Time complexity:**

Represents the number of operations an algorithm performs as a function of the input size, usually expressed using Big-O notation.

- **Space complexity:**

Represents the amount of memory an algorithm requires to store data as a function of the input size.

Factors considered in analysis:

Best case: The most efficient scenario where the algorithm performs the least number of operations.

Worst case: The least efficient scenario where the algorithm performs the most number of operations.

Average case: The expected behavior of the algorithm on a typical input.

Example analysis concepts:

- **Divide and conquer algorithms:**

Algorithms that break down a problem into smaller subproblems, solve them recursively, and then combine the solutions.

- **Sorting algorithms:**

Algorithms designed to arrange data in a specific order, like bubble sort, insertion sort, merge sort, quicksort.

- **Binary search:**

An efficient search algorithm that works on sorted data, repeatedly dividing the search space in half.

- **Recursive algorithms:**

Algorithms that call themselves to solve a problem by breaking it down into smaller instances of the same problem.

Importance of analysis of algorithms:

- **Choosing the best algorithm:**

By analyzing different algorithms for a problem, developers can select the most efficient one based on the expected input size and performance requirements.

- **Optimizing code:**

Understanding the performance characteristics of an algorithm allows for targeted improvements to the code to enhance its efficiency.

- **Predicting scalability:**

Analysis helps to estimate how an algorithm will behave with large input sizes, which is crucial for designing robust systems.

Role of Algorithms in Computing:

- Algorithms are essential in the field of computation, as they provide a set of instructions that enable a computer to execute a particular task. They are employed to resolve issues and execute tasks in computer systems, including the classification of data, searching for information, image processing, and numerous other functions.
- The algorithm specifies the steps required to achieve the desired result, and the computer executes the instructions to complete the task with precision and efficiency. The development of efficient algorithms is a fundamental area of computer science which has substantial implications in a variety of disciplines, including finance, cryptography, machine learning, and robotics.

- In order to enhance efficiency, accuracy, and decision-making, algorithms are extensively employed in a variety of industrial sectors.

Some of the key applications include:

- **Manufacturing:** Algorithms are employed to enhance the efficacy and reduce waste of production processes and supply chain management.
- **Finance:** Algorithms are employed to analyse financial data and generate predictions, thereby allowing merchants and investors to make well-informed decisions.
- **Healthcare:** Algorithms are employed to optimise treatment plans, diagnose maladies, and process and analyse medical images.
- Algorithms are employed in the retail sector to optimise pricing, provide personalised product recommendations, and manage consumer relationships.
- **Transportation:** Algorithms are employed to optimise delivery and transportation routes, thereby enhancing delivery speed and reducing fuel consumption.
- **Energy:** Algorithms are employed to enhance efficiency and minimise waste by optimising energy generation, distribution, and consumption.
- **Security:** Algorithms are employed to identify and avert security hazards, including fraud, hacking, and cyber-attacks

Example 1: Searching Algorithm with Arrays

Consider a simple linear search algorithm applied to an array. In this case, the algorithm aims to find the position of a specific element in the array.

```
def linear_search(arr, target):
    for i in range(len(arr)):
        if arr[i] == target:
            return i
    return -1
```

Analysis:

- Time Complexity: $O(n)$ in the worst case (linear time), where 'n' is the size of the array.
- Space Complexity: $O(1)$, as the algorithm uses constant extra space.

Example 2: Merge Sort (Performance Analysis)

- **Time Complexity:** In all scenarios (worst-case, average-case, and best-case), the time complexity of Merge Sort is $O(n \log n)$. It is hence quite effective for big datasets.
- **Space Complexity:** Because temporary arrays need more space during the merging phase, Merge Sort has an $O(n)$ space complexity. It is therefore a suitable option for situations where memory is at a premium.

- **Stability:** Merge Sort preserves the relative order of equal elements in the sorted output, making it a stable sorting algorithm. When the initial order of equal items must be maintained, this is desirable.

1.7 Asymptotic Notation

Introduction

Asymptotic notation is a mathematical framework used to describe the efficiency of algorithms, particularly their time and space complexity as the input size approaches infinity. It helps in analyzing the performance of algorithms by providing a way to express their growth rates in relation to input size. Asymptotic notation helps compare algorithms and predict their performance without requiring exact execution times.

Types of Asymptotic Notation

Asymptotic notation consists of three primary types:

- Big O (O)
- Omega (Ω)
- Theta (Θ)

Big O Notation (O) – Upper Bound

Big O notation represents the worst-case scenario of an algorithm. It provides an upper bound on the time complexity, meaning the algorithm will not take longer than this bound.

Mathematical Definition:

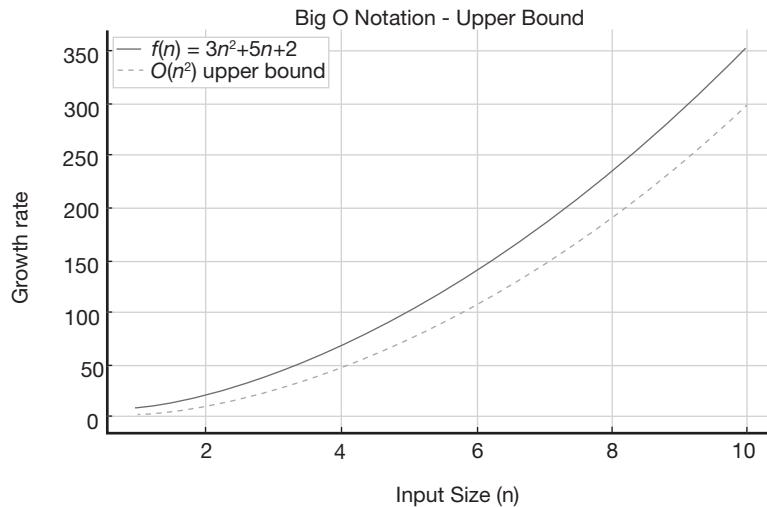
A function $f(n)$ is said to be in $O(g(n))$ if there exist positive constants c and n_0 such that:

$$f(n) \leq c \cdot g(n), \forall n \geq n_0$$

This means that for sufficiently large input sizes, the function $f(n)$ does not exceed a constant multiple of $g(n)$.

Examples:

- $5n^2 + 3n + 2$ is $O(n^2)$ since it grows at most as fast as n^2 for large n .
- $n^3 + 4n^2 + 10n + 50$ is $O(n^3)$.



Here is the graph illustrating the Big O notation (upper bound) for $f(n) = 3n^2 + 5n + 2$. The red dashed line represents the $O(n^2)$ upper bound, showing how the function is bounded for large values of n .

Common Big O Notations:

Notation	Complexity Name	Example Algorithm
$O(1)$	Constant Time	Accessing an array element
$O(\log n)$	Logarithmic Time	Binary Search
$O(n)$	Linear Time	Linear Search
$O(n \log n)$	Log-Linear Time	Merge Sort, Heap Sort
$O(n^2)$	Quadratic Time	Bubble Sort, Selection Sort
$O(n^3)$	Cubic Time	Matrix Multiplication
$O(2^n)$	Exponential Time	Recursive Fibonacci
$O(n!)$	Factorial Time	Traveling Salesman Problem

Omega Notation (Ω) – Lower Bound

Omega notation provides a lower bound on an algorithm's running time, meaning the algorithm will take at least this amount of time in the best case.

Mathematical Definition:

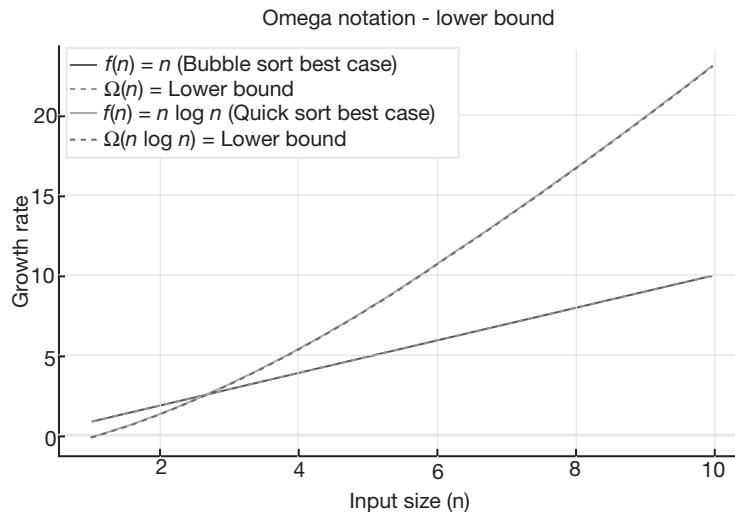
A function $f(n)$ is in $\Omega(g(n))$ if there exist positive constants c and n_0 such that:

$$f(n) \geq c \cdot g(n), \forall n \geq n_0$$

This means that for sufficiently large input sizes, $f(n)$ is at least a constant multiple of $g(n)$.

Examples:

- The best case of Bubble Sort is $\Omega(n)$ when the array is already sorted.
- The best case of QuickSort is $\Omega(n \log \log n)$.



Here is the graph illustrating **Omega (Ω) notation** as a lower bound for algorithm performance. It shows:

- **Bubble Sort Best Case ($\Omega(n)$)** in blue, with its lower bound in red.
- **QuickSort Best Case ($\Omega(n \log \log n)$)** in green, with its lower bound in purple.

Theta Notation (Θ) – Tight Bound

Theta notation represents both the upper and lower bounds of an algorithm's time complexity, providing a precise asymptotic behavior.

Mathematical Definition:

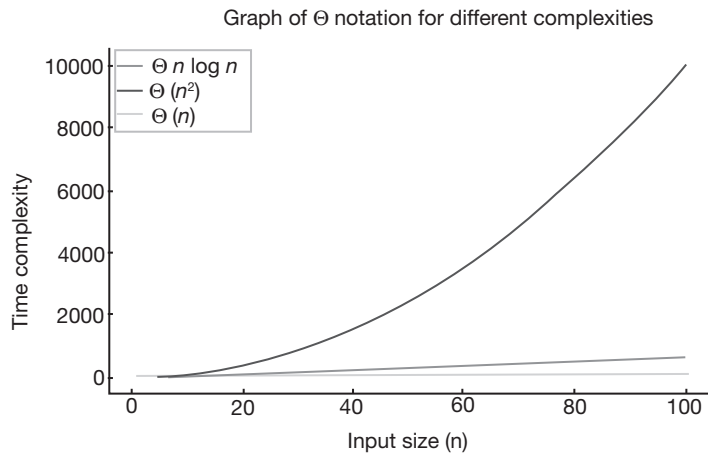
A function $f(n)$ is in $\Theta(g(n))$ if there exist positive constants c_1, c_2 , and n_0 such that:

$$c_1 \cdot g(n) \leq f(n) \leq c_2 \cdot g(n), \forall n \geq n_0$$

This means that $f(n)$ grows at the same rate as $g(n)$ for large values of n .

Examples:

- Merge Sort has a worst-case, best-case, and average-case complexity of $\Theta(n \log n)$.
- Insertion Sort has a worst-case complexity of $\Theta(n^2)$ and a best-case complexity of $\Theta(n)$.



Here's the graph showing the growth rates of the three time complexities:

- $\Theta(n \log n)$ (Merge Sort) in magenta
- $\Theta(n^2)$ (Worst-case Insertion Sort) in blue
- $\Theta(n)$ (Best-case Insertion Sort) in red

Comparison of Asymptotic Notations

Notation	Definition	Usage
O (Big O)	Upper bound on runtime	Worst-case scenario
Ω (Omega)	Lower bound on runtime	Best-case scenario
Θ (Theta)	Tight bound (upper & lower)	Average-case scenario

Importance of Asymptotic Notation

Performance Analysis: Helps determine the efficiency of algorithms before implementation.

Scalability Prediction: Assesses how well an algorithm will perform as input size increases.

Algorithm Comparison: Allows direct comparison of different algorithms based on efficiency.

Optimization: Guides developers in choosing optimal data structures and algorithms.

Real-World Applications of Asymptotic Notation

Search Engines (Google, Bing)

Search algorithms use **Binary Search** ($O(\log \log n)$) and indexing techniques to efficiently find results.

Data Compression

Huffman coding reduces file size using **greedy algorithms** with $O(n \log \log n)$ complexity.

Machine Learning

Training deep learning models can have complexities ranging from $O(n)$ (simple models) to $O(n^2)$ or worse (neural networks).

Cryptography

RSA encryption involves modular exponentiation with **exponential time complexity** ($O(2^n)$) for brute-force attacks.

Robotics & Path Planning

Algorithms like *A Search** and **Dijkstra's Algorithm** operate in $O(n \log \log n)$ or $O(n^2)$ time.

1.8 Recursion

Introduction

Despite being one of the most effective programming approaches, recursion in data structures frequently leaves developers baffled. It's similar to solving a riddle by dissecting it into smaller parts, each of which addresses a portion of the issue before coming together to provide the full solution. Recursion is essential to the efficiency of many fundamental algorithms, particularly those used in sorting and searching.

Recursion in Data Structures

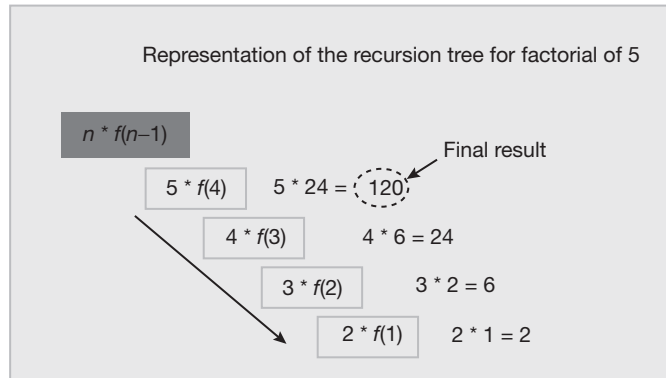
Recursion is a programming method used in data structures when a function calls by itself to resolve an issue. By dividing complicated issues into smaller, identical ones, it makes them easier to tackle directly. Consider it like peeling an onion, where each layer reveals a more basic issue. Recursion generates a series of calls that are identical to one another but do not continue indefinitely. An infinite loop is avoided by stopping the procedure in a basic case.

For example, calculating the factorial of 5 ($5!$) becomes straightforward with recursion.

The equation $5! = 5 \times 4 \times 3 \times 2 \times 1$ translates into "factorial(n) = $n \times$ factorial($n-1$)".

The base case stops at “ $n = 1$ ”.

Each recursive call reduces the problem's size, making recursion a highly efficient approach.



Recursion Work in Programming

By repeating a procedure in a self-referential manner, recursion in data structures addresses issues. Imagine it like peeling an onion, where each layer discloses a smaller portion of the issue until nothing remains to peel. Two essential components of this approach are a base case to halt the process and the recursive case to carry on addressing the problem.

Here's a brief illustration. In programming, calculating a number's factorial is a typical example of recursion.

```
#include <stdio.h>
// Function to calculate factorial
int factorial(int n) {
    if (n == 1) { // Base case
        return 1;
    }
    return n * factorial(n - 1); // Recursive case
}
int main() {
    int result = factorial(5); // Call factorial function
    printf("%d\n", result); // Outputs: 120
    return 0;
}
```

Common Uses of Recursion in Data Structures

Some of the most important programming processes are powered by recursion in data structures. It offers answers to issues that are recurrent or hierarchical by nature.

The following applications demonstrate the potent potential of recursion in structures of data.

- **Tree Traversal Methods:** Pre-order, in-order, and post-order tree structure traversal is made easier by recursion. With the help of these techniques, you may easily examine hierarchical data, including XML documents and folder directories.
- **Graph Algorithms:** Depth-first search (DFS) is a well-known example of recursion in data structures. It effectively investigates the nodes of a graph by delving deeply into it. It is perfect for solving puzzles and mazes because it emulates human curiosity by following paths before turning around.
- **Dynamic Programming:** Recursive solutions are necessary for dynamic programming to effectively handle overlapping subproblems. It all comes down to segmenting issues into smaller, more manageable chunks, much like the Fibonacci series or the best ways to climb stairs.
- **Backtracking:** Recursive retracing works well for tasks like word searches, N-Queens puzzles, and Sudoku solutions. It explores every option, turns back when it reaches a dead end, and keeps going until it finds a solution.
- **Divide and Conquer:** Algorithms like Quick Sort and Merge Sort employ recursion to partition problems into manageable components. These jobs are as simple as dividing a pizza into portions that can be handled thanks to recursion.

Types of Recursion

There are various types of recursion in data structures, and each is appropriate for particular situations.

- Direct Recursion
- Indirect Recursion
- Tail Recursion
- Non-Tail Recursion

Direct Recursion

A function can engage in direct recursion if it calls itself directly. It's as simple as a snake devouring its own tail: basic yet effective in the vast majority of circumstances.

The main characteristics and uses of direct recursion are highlighted in the items that follow.

- Simplifies mathematical computations, such as factorial or Fibonacci sequence calculations.
- Serves as the foundation for numerous dynamic programming solutions.
- Typical tree traversal methods for data that is hierarchical.
- In order to avoid infinite recursion, a well-defined basis scenario is necessary.
- Because it is linear, it is simple to debug and visualize.

Although direct recursion lays the groundwork, recursion need not always take a single, straightforward route. Indirect recursion, the following type, follows a different path.

Indirect Recursion

Indirect recursion is the term used to describe the process by which one function invokes another, which in turn invokes the original function. The distinctive interplay between functions is reminiscent of the game of ping-pong.

Indirect recursion is frequently used in the scenarios listed below.

- Applied to multi-step situations with two or more functions.
- Often seen in mutual recursion, in which tasks that depend on one another cooperate.
- Increases complexity while making it possible to solve difficult logical flows.
- Frequently observed in expression evaluations and compiler design.
- To preserve clarity, function calls must be closely monitored.

While indirect recursion does have some advantages, that is far from being the final chapter. Next, we have tail recursion, which can be a lifesaver in some scenarios.

Tail Recursion

The final operation of a recursive function is the recursive call, which is referred to as tail recursion. It is similar to the act of tying a bow in that it maintains order and efficiency.

The following are key benefits of tail recursion.

1. The recursive call can be optimized by compilers, which lowers memory overhead.
2. Frequently used when intermediate outcomes are not required.
3. Makes iterative activities or mathematical series simpler.
4. It Makes recursion possible to simulate an iterative loop.
5. Perfect for issues that call for deep recursion.

The optimization potential of tail recursion makes it stand out. To truly grasp its benefit, it is necessary to contrast it with non-tail recursion.

Non-Tail Recursion

A function engages in non-tail recursion when it executes more activities following the recursive call. It's more difficult, like piecemeal solving of a puzzle.

The key points about non-tail recursion are mentioned below.

- Preserves interim calculations for operations such as traversing trees.
- Provides flexibility in addressing complex issues.
- More stack frames result in higher memory utilization.
- Uses more memory because there are more stack frames.
- Frequently used in intricate mathematical calculations or algorithms such as DFS.
- To prevent excessive stack overhead, tight control is necessary.

Efficiency is important, yet non-tail recursion works well in many situations. In some situations, tail recursion is the better option due to its optimization advantages.

Common Recursive Algorithms in Data Structures

In data structures, recursion breaks down difficult jobs into smaller, more manageable components. For hierarchical and sequential issues, it offers accurate solutions, ranging from sorting to binary trees.

These examples showcase how recursion in data structure powers key algorithms.

Binary Tree Traversals (Inorder, Preorder, Postorder): Recursive techniques go over nodes in a methodical manner. In-order traversal, for instance, goes to the root, the left subtree, followed by the right subtree.

Graph Algorithms (DFS, BFS): Recursion is used in depth-first search (DFS) to thoroughly examine nodes before turning around. Recursive DFS effectively finds every path, whereas BFS utilizes iteration.

Sorting Algorithms (Quicksort, Mergesort): Quicksort divides elements around a pivot using recursion. Merge sort brings order out of chaos by recursively dividing arrays and merging sorted halves.

Data structures with recursion are more powerful than algorithms and can be used to solve common computing problems.

Real-World Applications of Recursion

Data structures with recursion are more powerful than algorithms and can be used to solve common computing problems. From accessing file systems to creating artificial intelligence solutions, its elegance transfers into problem-solving.

Mentioned below are some fascinating real-world applications where recursion takes center stage.

- **File System Navigation:** Recursion mimics a tree's structure by exploring nested directories with ease. It makes organization manageable by processing files layer by layer.
- **Web Crawling:** Recursion is how crawlers navigate online sites. They create extensive databases by retrieving links on a page and recursively following each one.
- **AI and Puzzles:** AI gaming techniques and puzzles like Sudoku and the N-Queens problem are powered by recursive backtracking. To choose the best course of action, it assesses each potential move.

Analyzing Recursion Complexity

A data structure's complexity in both space and time must be assessed in order to analyze recursion. If the computational demands of recursive functions are not properly taken into account, they can soon become inefficient.

- **Time Complexity:** Examine the frequency with which the function invokes itself. For instance, divide-and-conquer algorithms frequently have recursion with a temporal complexity of $O(n \log n)$.

- **Space Complexity:** Think about the amount of memory that the call stack uses. Deep recursion may result in stack overflow since each recursive call adds a new stack frame.
- **Call Stack Behavior:** Check the recursion depth. While non-tail recursion provides frame for interim computations, tail recursion reduces the amount of stack space used.
- **Base Case Efficiency:** Unnecessary calls are avoided in a well-designed base case. Calculations are spent in inefficient basic cases.
- **Optimizations like Tail Recursion:** By enabling a compiler to enhance recursive calls and reuse stack frames rather than create new ones, tail recursion lowers memory use.

1.9 Mathematical Analysis for Non-Recursive Algorithms

Introduction:

Mathematical Analysis for Non-Recursive Algorithms refers to the process of using mathematical tools and techniques to evaluate the performance, efficiency, and behavior of **non-recursive algorithms**, also known as **iterative algorithms**. This analysis helps to understand how these algorithms scale with input size, how much computational resources they require, and how they compare to other algorithms.

Non-Recursive Algorithms:

Non-recursive algorithms solve problems using iteration (loops) rather than recursion (function calls that refer to themselves). Examples include sorting algorithms like Bubble Sort, searching algorithms like Linear Search, and many others.

Analyzing the Time Efficiency of Non-Recursive Algorithms:

To analyze the time efficiency of non-recursive algorithms, we follow a systematic approach. This involves identifying key parameters, determining the basic operation, and using mathematical tools to express and simplify the algorithm's performance. Below is a step-by-step guide to conducting this analysis:

1. Decide on a Parameter Indicating the Input's Size:

- The first step is to determine the parameter(s) that represent the size of the input.
- For example:
 1. In sorting algorithms, the input size is often the number of elements in the array (n).
 2. In matrix operations, the input size could be the dimensions of the matrix (e.g., $n \times n$).
- The choice of parameter depends on the problem being solved and the structure of the input.

2. Identify the Algorithm's Basic Operation

- The basic operation is the most significant operation that contributes to the algorithm's overall runtime.
- It is typically located in the innermost loop of the algorithm.
- Examples of basic operations include:
 1. Comparisons (e.g., in sorting or searching algorithms).
 2. Arithmetic operations (e.g., addition, multiplication).
 3. Swaps or assignments (e.g., in sorting algorithms).

3. Determine if the Basic Operation Depends on Additional Properties

- Check whether the number of times the basic operation is executed depends solely on the input size or if it also depends on other properties of the input.
- If the count varies based on additional properties, analyze the following cases separately:
 1. **Worst-case efficiency:** The maximum number of operations performed for any input of size n .
 2. **Average-case efficiency:** The expected number of operations performed for a random input of size n .
 3. **Best-case efficiency:** The minimum number of operations performed for any input of size n .

4. Set Up a Summation for the Basic Operation

Express the number of times the basic operation is executed as a summation. For example:

- In a single loop, the summation might be straightforward (e.g., $\sum$ from $i=1$ to n).
- In nested loops, the summation will involve multiple indices (e.g., $\sum$ from $i=1$ to n $\sum$ from $j=1$ to i).

5. Simplify the Summation and Determine the Order of Growth

- Use standard mathematical formulas and rules to simplify the summation and derive a closed-form expression.
- Common formulas include:
 - Arithmetic series: $\sum$ from $i=1$ to n $i = n(n+1)/2$.
 - Geometric series: $\sum$ from $i=0$ to n $r^i = (r^{n+1} - 1)/(r - 1)$.
- Once the summation is simplified, express the result using **Big-O notation** to describe the algorithm's order of growth.

Basic Operation:

- The operation contributing the most to the total running time of an algorithm.
- It is typically the most time consuming operation in the algorithm's innermost loop.
- Examples: Key comparison operation; arithmetic operation (division being the most time-consuming, followed by multiplication)

Examples for Input size and Basic Operations:

Problem	Input size measure	Basic Operation
Searching for key in a list of n items	Number of list's , i.e. n	Key Comparison
Multiplication of two matrices	Matrix dimensions or total number of elements	Multiplication of two numbers
Checking primality of a given integer n	n size = number of digits (in binary representation)	Division
Typical graph problem	#vertices and/or edges	Visiting a vertex or traversing an edge

Example 1:

Consider the problem of finding the value of the largest element in a list of n numbers. Assume that the list is implemented as an array for simplicity.

Algorithm:

```

Max Element(A[0..n - 1])
// Determines the value of the largest element in a given array
// Input: An array A[0..n - 1] of real numbers
// Output: The value of the largest element in A
Max val ← A[0]
for i ← 1 to n - 1 do
    if A[i] > maxval
maxval ← A[i]
return maxval

```

Algorithm analysis:

- The measure of an input's size here is the number of elements in the array, i.e., n .
- There are two operations in the for loop's body:
 1. The comparison $A[i] > \text{maxval}$ and
 2. The assignment $\text{max val} \leftarrow A[i]$.
- The comparison operation is considered as the algorithm's basic operation, because the comparison is executed on each repetition of the loop and not the assignment.
- The number of comparisons will be the same for all arrays of size n ; therefore, there is no need to distinguish among the worst, average, and best cases here.

- Let $C(n)$ denotes the number of times this comparison is executed. The algorithm makes one comparison on each execution of the loop, which is repeated for each value of the loop's variable i within the bounds 1 and $n - 1$, inclusive. Therefore, the sum for $C(n)$ is calculated as follows:

$$C(n) = \sum_{i=1}^{n-1} 1.$$

$$C(n) = \sum_{i=1}^{n-1} 1$$

$$C(n) = \sum_{i=1}^{n-1} 1 = n - 1 \in \Theta(n)$$

Time Complexity:

- The loop runs from 1 to $n-1$, meaning it executes $n-1$ comparisons.
- Best Case:** $O(n)$ (when all elements are distinct or already in sorted order).
- Worst Case:** $O(n)$ (when the largest element is at the last position).
- Average Case:** $O(n)$ (each element is checked once).

Space Complexity:

- The algorithm uses **only one extra variable (maxVal)**, so the space complexity is $O(1)$ (constant space).

Example 2:

Element uniqueness problem. Check whether all the Elements in a given array of n elements are distinct.

Algorithm:

```

Element Uniqueness (A[0..n - 1])
//Determines whether all elements in a given array are distinct
//Input: An array A[0..n - 1] of real numbers
// Output: True if all elements are unique, False otherwise
for i ← 0 to n - 2 do
  for j ← i + 1 to n - 1 do
    if A[i] = A[j] then
      return False
return True

```

Algorithm Analysis:

- The algorithm consists of a nested loop where the outer loop runs $n-1$ times, and the inner loop runs from $i+1$ to $n-1$.

- In total, the number of comparisons is:

$$\sum_{i=0}^{n-2} (n-1-i) = \frac{n(n-1)}{2}$$

- This results in a worst-case time complexity of $O(n^2)$.

$$\begin{aligned} C_{\text{worst}}(n) &= \sum_{i=0}^{n-2} \sum_{j=i+1}^{n-1} 1 = \sum_{i=0}^{n-2} [(n-1) - (i+1) + 1] \\ &= \sum_{i=0}^{n-2} (n-1-i) = \sum_{i=0}^{n-2} (n-1) - \sum_{i=0}^{n-2} i = (n-1) \sum_{i=0}^{n-2} 1 - \frac{(n-2)(n-1)}{2} \\ &= (n-1)^2 - \frac{(n-2)(n-1)}{2} = \frac{(n-1)n}{2} \approx \frac{1}{2}n^2 \in \Theta(n^2). \end{aligned}$$

Space Complexity:

- The algorithm only uses a few integer variables for looping and comparisons, requiring $O(1)$ extra space.

Example 3:

Sort an array in ascending order.

Algorithm:

```
SelectionSort (A[0..n - 1])
for i ← 0 to n - 2 do
  minIndex ← i
  for j ← i + 1 to n - 1 do
    if A[j] < A[minIndex] then
  minIndex ← j
  swap A[i] and A[minIndex]
```

Algorithm Analysis:

- The input size is the number of elements in the array, denoted as n .
- The basic operation is the **comparison** (if $\text{arr}[j] > \text{arr}[j+1]$), as it is executed most frequently and directly affects the algorithm's runtime. Selection Sort is a simple and easy-to-implement algorithm but inefficient for large datasets due to its $O(n^2)$ time complexity. It is better than Bubble Sort in terms of swaps but still not practical for real-world sorting tasks compared to algorithms like Quick Sort or Merge Sort.

Time Complexity:

1. The outer loop runs $(n - 1)$ times.
2. The inner loop runs $(n - 1 - i)$ times.
3. Total comparisons:

$$\sum_{i=0}^{n-2} (n-1-i) = \frac{n(n-1)}{2} = O(n^2)$$

4. Best, Worst, and Average Case: $O(n^2)$.

Space Complexity: $O(1)$.

Example 4:

Multiply two square matrices A and B of size $n \times n$ to produce matrix C.

Algorithm:

MatrixMultiply(A, B, n)

for $i \leftarrow 0$ to $n - 1$ do

for $j \leftarrow 0$ to $n - 1$ do

$C[i][j] \leftarrow 0$

for $k \leftarrow 0$ to $n - 1$ do

$C[i][j] \leftarrow C[i][j] + A[i][k] * B[k][j]$

return C

Algorithm Analysis:

- There are **three nested loops** iterating over n :
 1. The outer loop runs n times (over rows of C).
 2. The middle loop runs n times (over columns of C).
 3. The inner loop runs n times (for summing product values).
- Each element of $C[i][j]$ requires **$O(n)$ multiplications and additions**.
- The total number of operations is:

$$O(n) \times O(n) \times O(n) = O(n^3)$$

Time Complexity:

- **Best, Worst, and Average Case:** $O(n^3)$.

Space Complexity:

- The algorithm requires **$O(n^2)$** space to store the output matrix C.
- **If matrices A and B are not modified**, space complexity remains **$O(n^2)$** .
- **If done in-place (overwriting A or B), space can be optimized.**

1.10 Mathematical Analysis for Recursive Algorithms

Introduction:

Recursive algorithms play a crucial role in computer science, enabling elegant and efficient solutions to complex problems. However, their performance analysis requires a deep understanding of mathematical techniques. **Mathematical Analysis for Recursive Algorithms** is the study of how recursive algorithms perform in terms of time complexity, space complexity, and correctness using mathematical techniques. It involves analyzing the growth rate of recursive functions and deriving recurrence relations to describe their behavior.

Recursive algorithm:

A **recursive algorithm** is an algorithm that **solves a problem by breaking it down into smaller instances of the same problem**, typically by calling itself with a modified input. The recursion continues until it reaches a **base case**, where the problem is simple enough to be solved directly.

Key Components of a Recursive Algorithm:

- **Base Case** – The stopping condition that prevents infinite recursion.
- **Recursive Case** – The step where the function calls itself with a reduced problem size.

General Plan for Analyzing the Time Efficiency of Recursive Algorithms:

Analyzing the time efficiency of recursive algorithms involves systematically determining how the algorithm scales with input size. The following structured approach can be used:

1. Determine the Input Size Parameter

- Identify the variable(s) that define the size of the input.
- For numerical problems, this is often a single integer n .
- For arrays, graphs, or trees, the size may be the number of elements, nodes, or edges.

2. Identify the Basic Operation

- Determine the fundamental operation that significantly impacts the algorithm's runtime.
- This could be a comparison, multiplication, function call, or another key computational step.

3. Account for Variability in Execution

- Analyze whether different inputs of the same size lead to varying execution times.
- If variability exists, evaluate the **best-case**, **worst-case**, and **average-case** complexities separately.

4. Formulate a Recurrence Relation

- Express the algorithm's execution time as a recurrence equation based on smaller input sizes.
- Include an appropriate base case that stops the recursion.

5. Solve the Recurrence Relation or Determine Growth Order

- Use mathematical techniques such as **recursion trees**, **the substitution method**, or **the Master Theorem** to find a closed-form solution.
- If an exact solution is difficult to obtain, determine the asymptotic complexity (Big-O notation) to estimate the order of growth.

Basic Operations in Recursive Algorithms:

Operation	Description	Example
Function Calls	Invoking itself recursively	Factorial, Fibonacci
Arithmetic Operations	Addition, multiplication, etc.	Fibonacci, Merge Sort
Comparisons	Base case checks, branching	Binary Search
Memory Allocation	Stack usage for recursive calls	Nested Recursion
Loop Simulation	Indirect repetition via recursion	Tower of Hanoi
Return Statements	Returning values from function calls	Tail Recursion

Example 1:

Factorial Computation

Algorithm: Recursive Factorial:

```
// Computes n! recursively
// Input: A nonnegative integer n
// Output: The value of n!
if n = 0 return 1
else return F(n - 1) * n
```

Mathematical Analysis:

- **Recurrence Relation:**

$$T(n) = T(n-1) + O(1)$$

- **Expansion:**

$$\begin{aligned}
 T(n) &= T(n-1) + O(1) \\
 &= T(n-2) + O(1) + O(1) \\
 &= T(n-3) + O(1) + O(1) + O(1) = T(1) + O(n)
 \end{aligned}$$

- **Time Complexity:**
 $O(n)$ (Linear Time Complexity)

Example 2:**Merge Sort****Algorithm: Recursive Merge Sort**

```
// Sorts an array recursively using Merge Sort
// Input: An array A of size n
// Output: A sorted array
if n == 1 return A
mid = n / 2
left = MergeSort(A[0...mid])
right = MergeSort(A[mid+1...n])
return Merge(left, right)
```

Mathematical Analysis:

- **Recurrence Relation:**
 $T(n) = 2T(n/2) + O(n)$
- **Expansion (Using Recursion Tree Method):**
 - The number of calls **doubles** at each level.
 - The height of the recursion tree is $O(\log n)$.
 - Each level does $O(n)$ work.
- **Time Complexity:**

$O(n \log n)$ (Log-Linear Time Complexity)

Example 3:**Algorithm: Recursive Tower of Hanoi**

The Tower of Hanoi problem, a recursive algorithm pedagogical mainstay, is the object of our next case study. We (or legendary monks, if you prefer not to handle the discs) have n discs of varying sizes that can be fitted onto one of three pegs in this problem. At first, the discs are arranged on the initial peg according to size, with the largest ones at the bottom and the tiniest ones on top. The objective is to transfer all of the discs to the third peg, with the second one serving as a backup if needed. We are limited to relocating a single disc at a time, and stacking bigger discs on top of smaller ones is not allowed.

Figure 1.2 shows the elegant recursive technique for solving the problem. The following steps are needed to transfer $n > 1$ discs from peg 1 to peg 3 (using peg 2 as an auxiliary): moving $n - 1$ discs recursively from peg 1 to peg 2 (using peg 3 as an auxiliary), then directly moving the largest disc from peg 1 to peg 3, and finally, moving $n - 1$ discs recursively from peg 2 to peg 3 (using

peg 1 as an auxiliary). Obviously, when $n=1$, we just need to transfer the one disc from the source peg to the destination peg.

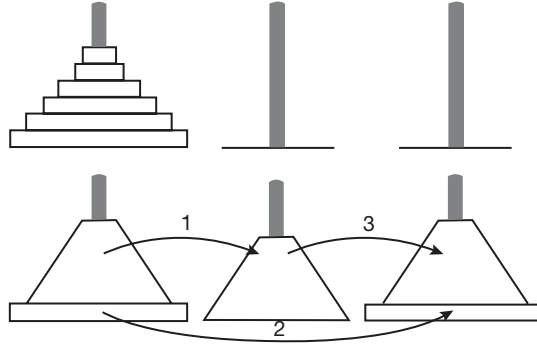


FIG. 1.1 Recursive solution to the Tower of Hanoi puzzle.

Let us employ the general strategy described above to the Tower of Hanoi issue. As the input's size indicator, the number of discs n is the apparent choice, and the algorithm's fundamental operation is the movement of one disc. It is evident that the number of movements $M(n)$ is solely dependent on n , and the recurrence equation for it is as follows:

$$M(n) = M(n-1) + 1 + M(n-1) \text{ for } n > 1$$

The recurrence relation for the number of moves $M(n)$ is as follows, given the evident initial condition $M(1)=1$:

$$M(n) = 2M(n-1) + 1 \text{ for } n > 1, M(1) = 1$$

We use the same backward substitution technique to resolve this recurrence:

$$\begin{aligned} M(n) &= 2M(n-1) + 1 \text{ sub. } M(n-1) = 2M(n-2) + 1 \\ &= 2[2M(n-2) + 1] + 1 = 2^2 M(n-2) + 2 + 1 \text{ sub. } M(n-2) = 2M(n-3) + 1 \\ &= 2^2 [2M(n-3) + 1] + 2 + 1 = 2^3 M(n-3) + 2^2 + 2 + 1 \end{aligned}$$

The pattern of the first three sums on the left suggests that the next one will be $2^4 M(n-4) + 2^3 + 2^2 + 2 + 1$, and generally, after i substitutions, we get

$$M(n) = 2^i M(n-i) + 2^{i-1} + 2^{i-2} + \dots + 2 + 1 = 2^i M(n-i) + 2^i - 1$$

The solution to the recurrence is given by the following formula, as the initial condition is specified for $n=1$, which is achieved for $i=n-1$:

$$\begin{aligned} M(n) &= 2^{n-1} M(n-(n-1)) + 2^{n-1} - 1 \\ &= 2^{n-1} M(1) + 2^{n-1} - 1 = 2^{n-1} + 2^{n-1} - 1 = 2^n - 1. \end{aligned}$$

So, we have an exponential algorithm that, even for reasonable values of n , will run for an unimaginably long time (see to Problem 5 in the exercises in this section for more information). This is not because the method in question is inefficient; in fact, demonstrating that it is the best algorithm for this scenario is not hard at all. The computational difficulty stems from the problem's inherent difficulty. Though concise, recursive algorithms can hide their inefficiency, thus one should use caution when using them. This example still illustrates a significant general principle.

Building a tree of a recursive algorithm's calls can be helpful for examination when the algorithm calls itself more than once. We can mark each node in this tree with the value of the parameter (or, more broadly, parameters) of the recursive calls that they correspond to. Figure 2.5 shows the tree for the Tower of Hanoi example. We can determine how many calls the Tower of Hanoi algorithm has made overall by counting the nodes in the tree:

$$(n) = \sum_{l=0}^{n-1} 2^l = 2^n - 1$$

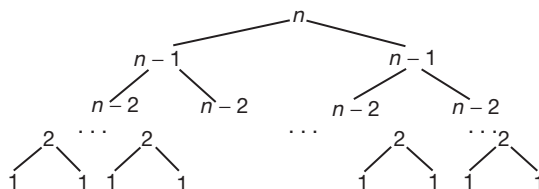


FIG 1.2 Tree of recursive calls made by the recursive algorithm for the Tower of Hanoi puzzle.

The figure is consistent with the move total that was previously obtained.

Example 4:

Binary Search

Algorithm: Recursive Binary Search

```
// Searches for a key in a sorted array recursively
// Input: A sorted array A, left index l, right index r, key x
// Output: The index of x in A, or -1 if not found
if l > r return -1
mid = (l + r) / 2
if A[mid] == x return mid
elseif A[mid] > x returnBinarySearch(A, l, mid-1, x)
elsereturnBinarySearch(A, mid+1, r, x)
```

Mathematical Analysis:

- **Recurrence Relation:**

$$T(n) = T(n/2) + O(1)$$

- **Expansion:**

$$\begin{aligned}T(n) &= T(n/2) + O(1) \\&= T(n/4) + O(1) + O(1) \\&= T(n/8) + O(1) + O(1) + O(1)\end{aligned}$$

- This continues until $n = 1$, leading to $\log n$ levels.
- The number of recursive calls is equal to n
- **Time Complexity:**

$O(\log n)$ (Logarithmic Time Complexity) **.Efficient for large inputs** since it eliminates half the data in each step.

List ADT

A list ADT (Abstract Data Type) is a linear data structure that stores elements in a sequence. It provides a way to organize, access, and modify data.

Operations:

- **Get:** Returns an element at a given position
- **Insert:** Inserts an element at a given position
- **Remove:** Removes the first occurrence of an element
- **RemoveAt:** Removes an element at a specified location
- **Replace:** Replaces an element at a given position with another element
- **Size:** Returns the number of elements in the list
- **IsEmpty:** Returns true if the list is empty, false if it's not
- **IsFull:** Returns true if the list is full, false if it's not

Implementation of List ADT:

1. Array Implementation
2. Linked List Implementation
3. Cursor Implementation

Array Implementation of List

Array is a collection of specific number of data stored in a consecutive memory location.

- Insertion and Deletion operation are expensive as it requires more data movement
- Find and Print list operations takes constant on time.

- Even if the array is dynamically allocated, an estimate of the maximum size of the list is required which considerably wastes the memory space.

20	10	30	40	50	60
A[0]	A[1]	A[2]	A[3]	A[4]	A[5]

Linked List Implementation

A linked list is a dynamic linear data structure whose memory size can be allocated or de-allocated at run time based on the operation.

Linked lists can be used to implement various data structures like a stack, queue, graph, hash maps, etc.

Singly linked lists contain two “buckets” in one node; one bucket holds the data and the other bucket holds the address of the next node of the list. Traversals can be done in one direction only as there is only a single link between two nodes of the same list.

The pointer of the last node points to NULL.

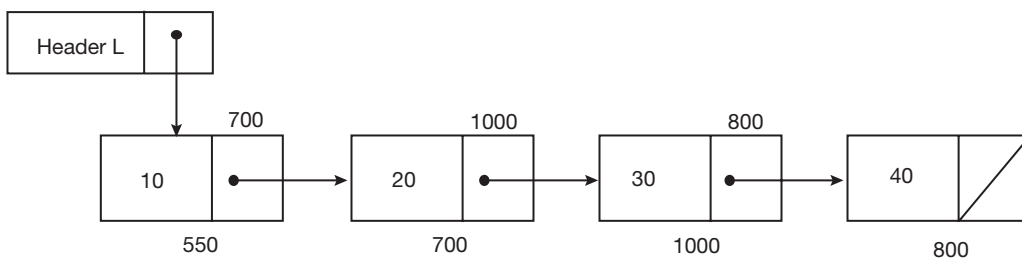


Types of Linked List:

1. Singly Linked List
2. Doubly Linked List
3. Circular Linked List

Singly Linked List:

A singly linked list is a in which each node contains linked list only one link field pointing to the next node in the list.

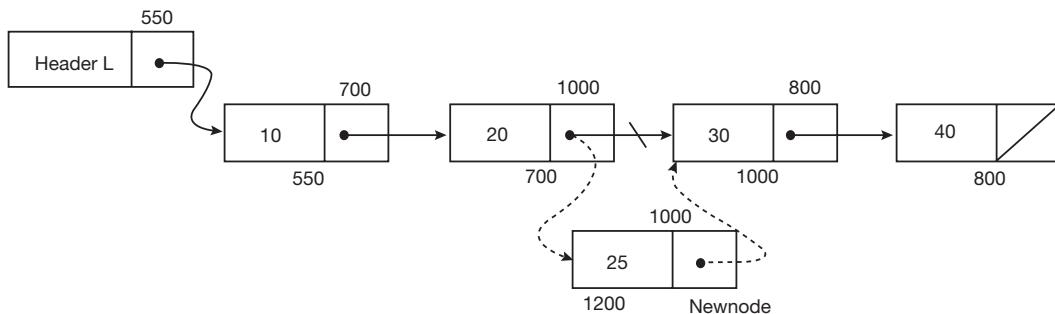


Linked List with a Header Declaration for Linked List Declaration for Linked List

Routine to insert an element in the list:

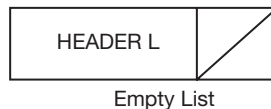
A routine to insert an element into a list is a sequence of instructions that adds a specified element at a particular position or at the end of the list.

```
def insert(self, x, position):
    if position is None:
        print("Invalid position: Cannot insert after None")
        return
    new_node = Node(x)
    new_node.next = position.next
    position.next = new_node
```

**Routine to check whether the list is empty:**

A routine to check whether a list is empty is a sequence of instructions that determines if a given list contains any elements.

```
def is_empty(self):
    """ Check if the linked list is empty """
    return self.head is None
```

**Routine to check whether the current position is Last:**

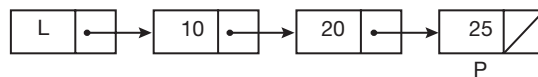
In a singly linked list, a node is considered the last node if its next pointer is None.

```
def is_last(self, position):
    """ Check if a given node is the last node in the list """
    return position is not None and position.next is None
```

Find Routine:

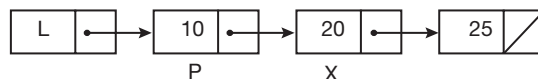
The Find routine in a singly linked list is used to search for a node containing a specific value and return its reference (or None if not found).

```
def find(self, x):
    """ Find a node with value X and return its reference """
    temp = self.head
    while temp is not None and temp.data != x:
        temp = temp.next
    return temp
```

**Find Previous Routine:**

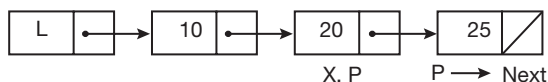
It refers to a function or method that locates the node preceding a given node.

```
def find_previous(self, x):
    """ Find the node before the node containing X """
    if self.head is None or self.head.data == x:
        return None
    temp = self.head
    while temp.next is not None and temp.next.data != x:
        temp = temp.next
    return temp if temp.next is not None else None
```

**FindNext Routine:**

A FindNext Routine in a linked list refers to a function that locates and returns the node that follows a given node. This is straightforward in both singly linked lists and doubly linked lists.

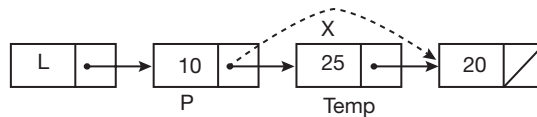
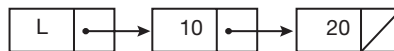
```
def find_next(self, x):
    """ Find the node after the node containing X """
    temp = self.head
    while temp is not None and temp.data != x:
        temp = temp.next
    return temp.next if temp is not None else None
```



Routine to delete an element from the list:

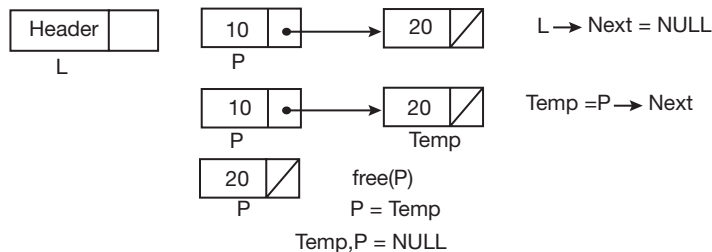
A Delete Routine in a linked list is a function that removes a node containing a specific value from the list. The routine updates the pointers of the adjacent nodes to maintain the integrity of the list after deletion.

```
def delete(self, x):
    if self.head is None:
        print("List is empty.")
        return
    if self.head.data == x:
        temp = self.head
        self.head = self.head.next
        del temp
    return
```

**Before deletion****After deletion****Routine to delete the list:**

A Delete List Routine is a function that removes all nodes from a linked list, effectively freeing the allocated memory and resetting the list structure.

```
def delete_list(self):
    current = self.head
    self.head = None
    while current:
        temp = current.next
        del current
        current = temp
```



Doubly Linked List

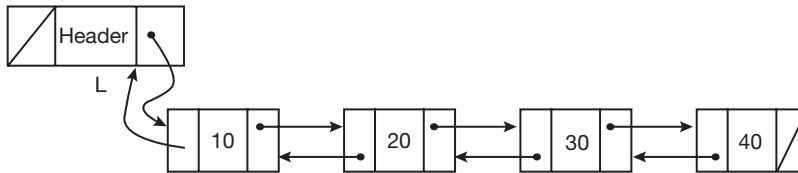
A Doubly Linked List (DLL) is a type of linked list in which each node contains three parts:

1. Data – Stores the actual value.
2. Next Pointer (next) or FLINK – Points to the next node in the list.
3. Previous Pointer (prev) or BLINK – Points to the previous node in the list.

This structure allows traversal in both directions (forward and backward), making it more flexible than a singly linked list.

BLINK	DATA ELEMENT	FLINK
-------	--------------	-------

Single Node in doubly linked list



Doubly Linked List

Types of Insertions in a Doubly Linked List:

1. **Insert at the Beginning** – Adds a node before the head.
2. **Insert at the End** – Adds a node at the tail.
3. **Insert at a Specific Position** – Inserts a node between two existing nodes.

Routine to insert an element at specific position in a doubly linked list:

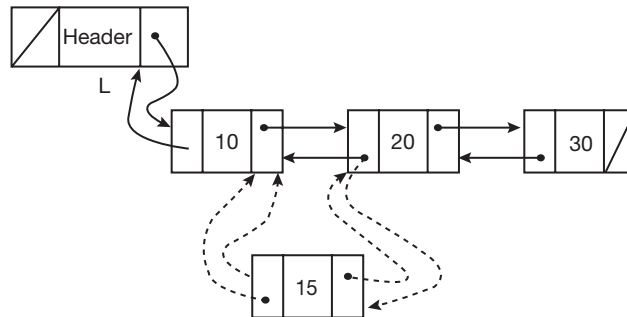
An Insert Routine in a Doubly Linked List (DLL) is a function that adds a new node at a specific position while maintaining the prev and next pointers correctly.

```
def insert_at_position(head, pos, new_data):
    new_node = Node(new_data)
    if pos == 1:
        new_node.next = head
        if head is not None:
            head.prev = new_node
            head = new_node
        return head
    curr = head
    for _ in range(1, pos - 1):
        curr = curr.next
```

```

if curr is None:
    print("Position is out of bounds.")
    return head
new_node.prev = curr
new_node.next = curr.next
curr.next = new_node
if new_node.next is not None:
    new_node.next.prev = new_node

```



Types of Deletions in a Doubly Linked List:

1. **Delete at the Beginning** – Deletes a node before the head.
2. **Delete at the End** – Deletes a node at the tail.
3. **Delete at a Specific Position** – Deletes a node between two existing nodes.

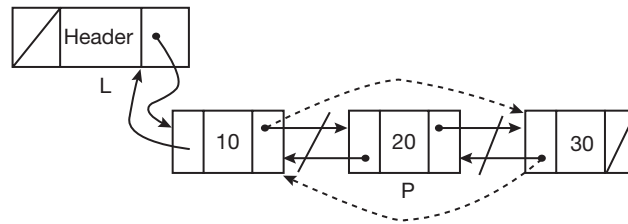
Routine to delete an element at the middle in a doubly linked list:

A Delete Middle Routine in a Doubly Linked List (DLL) removes a node that is neither the head nor the tail.

```

def delete(self, x):
    node = self.find(x)
    if node is None:
        return
    if self.is_last(node):
        if node.prev:
            node.prev.next = None
        else:
            self.head = None
    else:
        if node.prev:
            node.prev.next = node.next
        if node.next:
            node.next.prev = node.prev
    del node

```

**Advantage:**

- Reversing the doubly linked list is very easy.
- It can allocate or reallocate memory easily during its execution.
- As with a singly linked list, it is the easiest data structure to implement.
- Deletion operation is easier

Disadvantage:

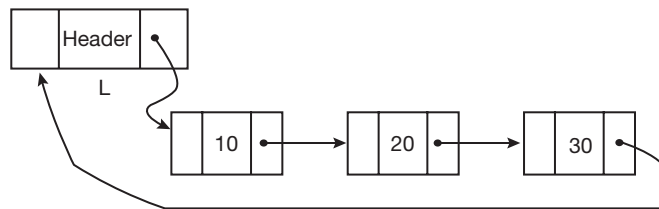
- More Memory Space is required since it has two pointers
- Traversing a doubly linked list can be slower than traversing a singly linked list.
- Implementing and maintaining doubly linked lists can be more complex than singly linked lists.

Circular Linked List

In circular linked list the pointer of the last node points to the first node. Circular linked list can be implemented as Singly linked list and Doubly linked list with or without headers.

1. Singly linked circular list:

A singly circular list is a linked list in which the last node of the list points to the first



Singly Linked circular list Model

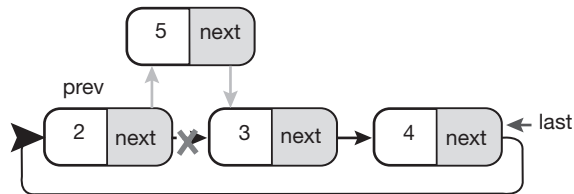
Routine to insert an element at specific position in a Singly Linked Circular list:

An Insert Routine in a Doubly Linked List (DLL) is a function that adds a new node at a specific position while maintaining the prev and next pointers correctly.

```

def insert_at_position(self, position, data):
    new_node = Node(data)
    if position < 0:
        print("Invalid position")
        return
    if position == 0:
        new_node.next = self.head
        self.head = new_node
        return
    current = self.head
    count = 0
    while count < position - 1 and current:
        current = current.next
        count += 1
    if not current:
        print("Position out of range")
        return
    new_node.next = current.next
    current.next = new_node

```



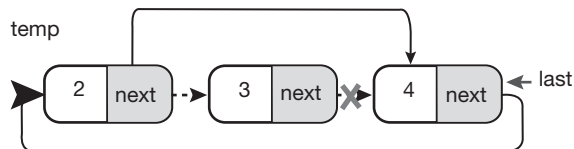
Routine to delete an element in a Singly Linked Circular list:

A Delete Middle Routine in a Singly Circular Linked List removes a node that is neither the head nor the last node while maintaining the circular structure.

```

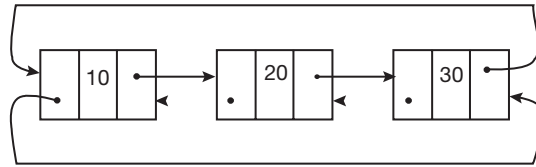
def delete(self, x):
    temp = self.head
    prev = None
    if temp is not None and temp.data == x:
        self.head = temp.next
        temp = None
        return

```



2. Doubly linked circular list:

A doubly linked circular list is a Doubly linked list in which the forward link of the last node points to the first node and backward link of the first node points to the last node of the list.

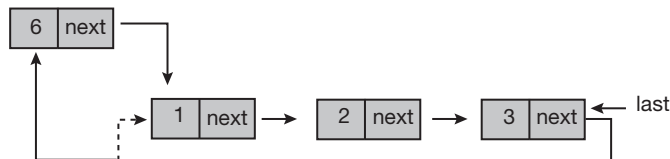


Doubly Linked Circular List with Header

Routine to insert an element at the beginning:

Store the address of the current first node in the new Node (i.e. pointing the new Node to the current first node) and point the last node to new Node

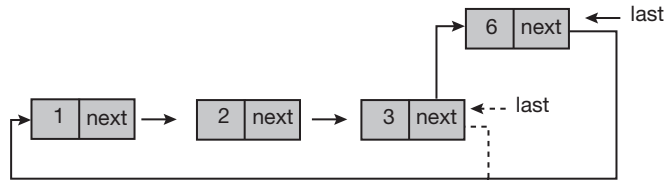
```
def add_begin(self, data):
    if self.last is None:
        return self.add_to_empty(data)
    new_node = Node(data)
    new_node.next = self.last.next
    self.last.next = new_node
    return self.last
```



Routine to insert an element at the end:

Inserting an element at the end of a Doubly Circular Linked List means adding a new node after the current last node

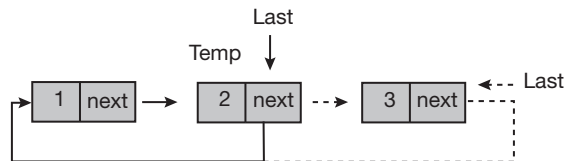
```
def add_end(self, data):
    if self.last is None:
        return None
    new_node = Node(data)
    new_node.next = self.last.next
    self.last.next = new_node
    self.last = new_node
    return self.last
```



Routine to delete last node in the list:

```

def delete_node(self, key):
    if self.last is None:
        return
    if self.last.data == key and self.last.next == self:
        last:
            self.last = None
            return
    temp = self.last
    d = None
    if self.last.data == key:
        while temp.next != self.last:
            temp = temp.next
        temp.next = self.last.next
        self.last = temp.next
        return
    while temp.next != self.last and temp.next.data != key:
        temp = temp.next
    if temp.next.data == key:
        d = temp.next
        temp.next = d.next
    else:
        print("No such key found")
  
```



Advantages of Circular Linked List:

- It allows traversing the list starting at any point.
- It allows quick access to the first and last records.
- Circularly doubly linked list allows to traverse the list in either direction

Applications of List

- Polynomial ADT
- RadixSort
- Robotics
- Task Scheduling
- File Systems
- Undo/Redo Functionality

1. POLYNOMIAL ADT

We can perform the polynomial manipulations such as addition, subtraction and differentiation etc.

Addition of Two Polynomials:

It involves adding two like polynomials by combining their coefficients.

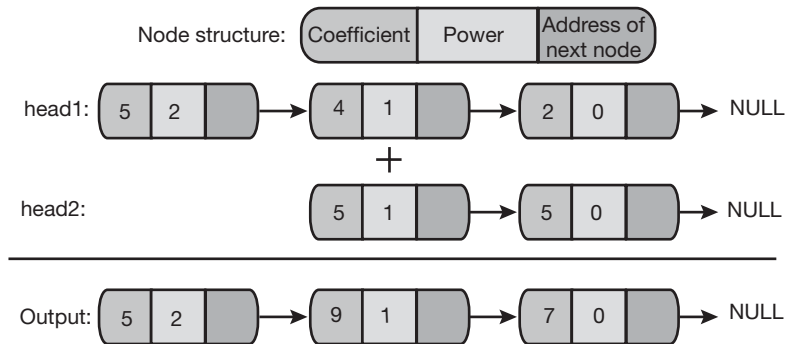
```
def add_poly(list1, list2):
    ptr1, ptr2 = list1, list2
    list3 = None
    while ptr1 is not None and ptr2 is not None:
        new_node = PolyNode(0, 0)
        if ptr1.power == ptr2.power:
            new_node.coeff = ptr1.coeff + ptr2.coeff
            new_node.power = ptr1.power
            ptr1 = ptr1.next
            ptr2 = ptr2.next
        elif ptr1.power > ptr2.power:
            new_node.coeff = ptr1.coeff
            new_node.power = ptr1.power
            ptr1 = ptr1.next
        else:
            new_node.coeff = ptr2.coeff
            new_node.power = ptr2.power
            ptr2 = ptr2.next
            new_node.next = None
        list3 = create(list3, new_node)
```

Example:

List1 $\rightarrow 5x^2 + 4x^1 + 2$

List2 $\rightarrow 5x^1 + 5$

List3 $\rightarrow 5x^2 + 9x^1 + 7$



Subtraction of two polynomials:

To subtract polynomials, you first distribute the negative sign to each term of the polynomial being subtracted, then combine like terms by adding their coefficients.

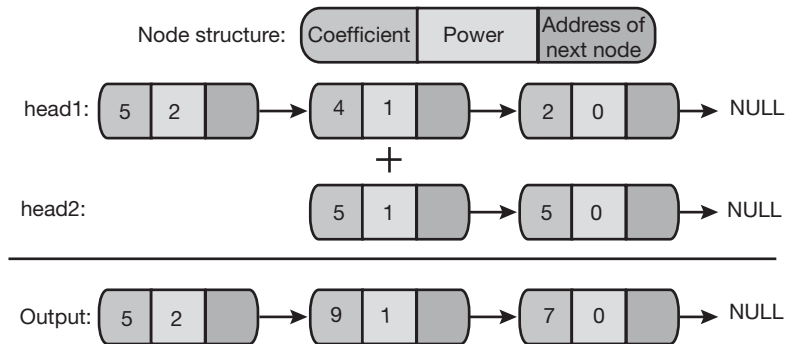
```
def add_poly(list1, list2):
    ptr1, ptr2 = list1, list2
    list3 = None
    while ptr1 is not None and ptr2 is not None:
        new_node = PolyNode(0, 0)
        if ptr1.power == ptr2.power:
            new_node.coeff = ptr1.coeff + ptr2.coeff
            new_node.power = ptr1.power
            ptr1 = ptr1.next
            ptr2 = ptr2.next
        elif ptr1.power > ptr2.power:
            new_node.coeff = ptr1.coeff
            new_node.power = ptr1.power
            ptr1 = ptr1.next
        else:
            new_node.coeff = - ptr2.coeff
            new_node.power = ptr2.power
            ptr2 = ptr2.next
            new_node.next = None
        list3 = create(list3, new_node)
```

Example:

List1 → $5x^2 + 4x^1 + 2$

List2 → $-5x^1 - 5$

List3 → $5x^2 - x^1 - 2$



Poly Differentiation:

“Poly Differentiation” refers to finding the derivative (or rate of change) of a polynomial function, which can be done by applying the power rule and other differentiation rules to each term individually.

```
def differentiate(self):
    ptr1 = self.head
    derivative = Polynomial()
    while ptr1 is not None:
        if ptr1.power != 0:
            new_coeff = ptr1.coeff * ptr1.power
            new_power = ptr1.power - 1
            derivative.insert_end(new_coeff, new_power)
        ptr1 = ptr1.next
    return derivative
```

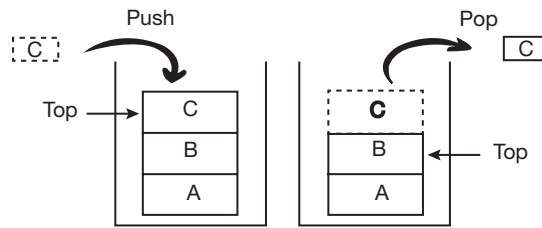
Example:

Input: str = “4X³ + 3X¹ + 2X²”

Output: “12X² + 3X⁰ + 4X¹”

Stack

Stacks in Data Structures is a linear type of data structure that follows the LIFO (Last-In-First-Out) principle and allows insertion and deletion operations from one end of the stack data structure, that is top.

**Stack Model**

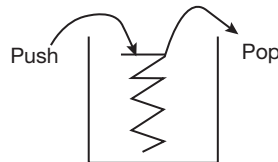
Example:

Pile of coins., a stack of trays in cafeteria.

Operations on Stack

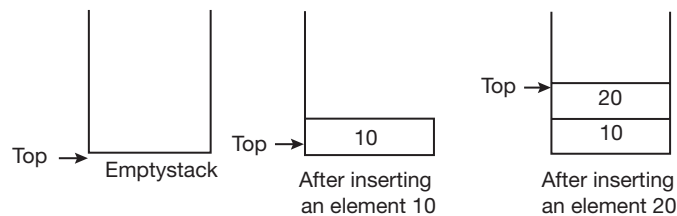
The fundamental operations performed on a stack are

1. Push
2. Pop

**Operations on stack**

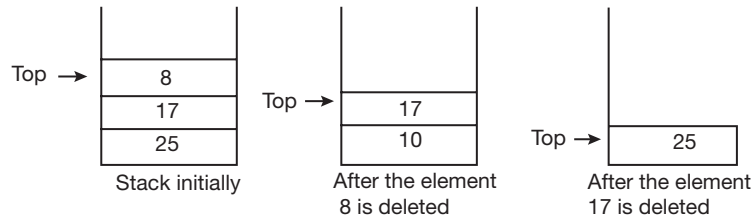
Push:

The process of inserting an element to the top of the stack. For every push operation the top is incremented by 1.

**Push Operation**

Pop:

The process of deleting an element from the top of stack is called pop operation. After every pop operation the top pointer is decremented by 1.



Pop operation

Exceptional Conditions

Over Flow

Attempt to insert an element when the stack is full is said to be overflow.

Under Flow

Attempt to delete an element, when the stack is empty is said to be underflow.

Implementation of Stack

Stack can be implemented using arrays and pointers.

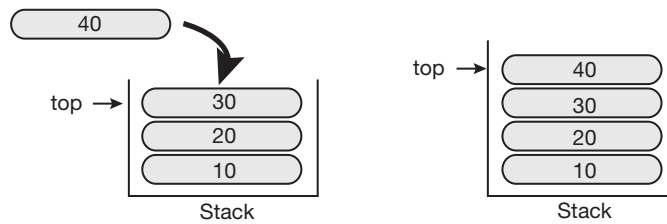
Array Implementation

In this implementation each stack is associated with a pop pointer, which is -1 for an empty stack.

- To push an element X onto the stack, Top Pointer is incremented and then set Stack [Top] = X.
- To pop an element, the stack[Top] value is returned and the top pointer is decremented.
- Pop on an empty stack or push on a full stack will exceed the array bounds.

Routine to Push an Element On to a Stack:

```
class Stack:
    def __init__(self, size):
        self.stack = [None] * size
        self.top = -1
        self.size = size
    def is_full(self):
        return self.top == self.size - 1
    def push(self, x):
        if self.is_full():
            print("Error: Full Stack")
        else:
            self.top += 1
            self.stack[self.top] = x
```



Push Operations in Stack

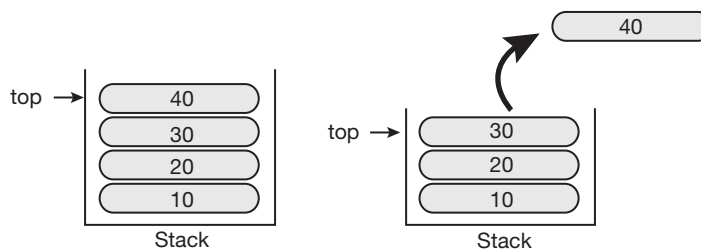
Routine to Pop an Element from the Stack:

Removes an item from the stack. The items are popped in the reversed order in which they are pushed. If the stack is empty, then it is said to be an Underflow condition.

Algorithm for Pop Operation:

- Before popping the element from the stack, we check if the stack is empty .
- If the stack is empty ($\text{top} == -1$), then Stack Underflows and we cannot remove any element from the stack.
- Otherwise, we store the value at top, decrement the value of top by 1 ($\text{top} = \text{top} - 1$) and return the stored top value.

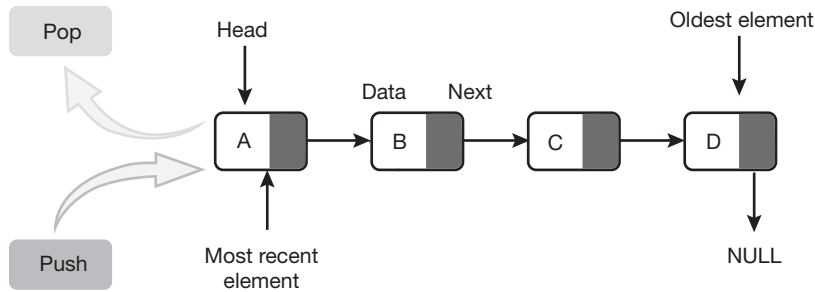
```
class Stack:
    def __init__(self, cap):
        self.cap = cap
        self.top = -1
        self.a = [0] * cap
    def push(self, x):
        if self.top >= self.cap - 1:
            print("Stack Overflow")
            return False
        self.top += 1
        self.a[self.top] = x
        return True
```



Pop operation in Stack

Linked List Implementation of Stack

- Push operation is performed by inserting an element at the front of the list.
- Pop operation is performed by deleting at the front of the list.
- Top operation returns the element at the front of the list.



Routine to create an empty stack:

Creating an empty stack using a linked list involves initializing a stack with no elements. The top pointer is set to None to indicate an empty stack.

lass Stack:

```

def __init__(self):
    self.top = None
def is_empty(self):
    return self.top is None
ef make_empty(self):
    if self is None:
        print("Create Stack First")
        return
    while not self.is_empty():
        self.pop()
  
```

Routine to return top element in a stack:

In a stack, the top element is the most recently added item (Last In, First Out - LIFO)

```

def top_element(self):
    if not self.is_empty():
        return self.top.data
    print("Empty Stack")
    return None
  
```

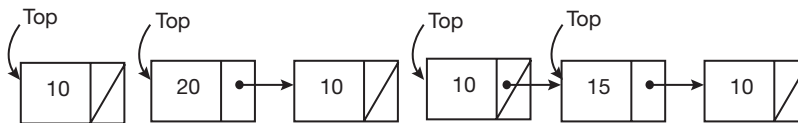
Routine to push an element onto a stack:

The push operation in a stack implemented using a linked list adds a new element to the top of the stack. Since stacks follow LIFO (Last In, First Out), the new node is inserted at the beginning of the linked list.

```

class Stack:
    def __init__(self):
        self.top = None
    def is_empty(self):
        return self.top is None
    def push(self, x):
        try:
            temp_cell = Node(x)
        except MemoryError:
            print("Out of Space")
            return
        temp_cell.next = self.top
        self.top = temp_cell

```



Linked list implementation of stack - Push operation

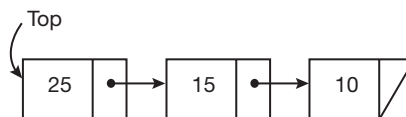
Routine to pop from a stack:

The pop operation in a stack implemented using a linked list removes the top element following the LIFO (Last In, First Out) principle. This means the most recently added element is removed first.

```

def pop(self):
    if self.is_empty():
        print("Empty Stack")
        return
    temp = self.top
    self.top = self.top.next
    del temp

```



Linked list implementation of stack - Pop operation

Applications of Stack

Some of the applications of stack are:

- (i) Evaluating Arithmetic Expression

- (ii) Balancing the symbols
- (iii) Towers of Hanoi
- (iv) Function Calls
- (v) 8 Queen Problem

Different Types of Notations to Represent Arithmetic Expression

There are 3 different ways of representing the algebraic expression. They are

- INFIX NOTATION
- POSTFIX NOTATION
- PREFIX NOTATION

Infix:

In Infix notation, the arithmetic operator appears between the two operands to which it is being applied.

For example:- $A / B + C$

Postfix:

The arithmetic operator appears directly after the two operands to which it applies. Also called reverse polish notation. $((A/B)+C)$

For example:- $AB / C +$

Prefix:

The arithmetic operator is placed before the two operands to which it applies.

Also called as polish notation. $((A/B)+C)$

For example: $+ / ABC$

S.NO	INFIX	PREFIX(or) POLISH	POSTFIX (or) REVERSEPOLISH
1	$(A+B)/(C-D) A +$	$/ + AB - CD$	$AB + CD - /$
2	$B*(C - D)X * A /$	$+ A * B - CD$	$ABCD - * + X$
3	$B - D$	$- / * XABD$	$A * B / D -$
4	$X + Y*(A - B)/(C - D)$	$+ X / * Y - AB - CD$	$XYAB - * CD - / +$
5	$A * B / C + D$	$+ / * ABCD$	$AB * C / D +$

1. Evaluating Arithmetic Expression

To evaluate arithmetic expressions, first convert the given infix expression to postfix expression and then evaluate the postfix expression using stack.

Infix to Postfix Conversion

Read the infix expression one character at a time until it encounters the delimiter. #

Step 1: If the character is an operand, place it on to the output.

Step 2: If the character is an operator, push it onto the stack. If the stack operator has a higher or equal priority than input operator then pop that operator from the stack and place it onto the output.

Step 3: If the character is a left parenthesis is, push it onto the stack.

Step 4: If the character is a right parenthesis is, pop all the operators from the stack till it encounters left parenthesis, discard both the parenthesis in the output.

Infix Expression : $A * B + (C - D / E) \#$

Read Character	Stack	Output
A		A
*	*	A
B	*	AB
+		AB*
	+	AB*
(	(+	AB*
(	(+	AB*C

Read Character	Stack	Output
-	- (+	AB*C
D	- (+	AB*CD
/	/ - (+	AB*CD
E	/ - (+	AB*CDE
)	+	AB*CDE/-
#		AB*CDE/-+

Conversion of Infix Expression to Postfix Expression

Evaluating Postfix Expression:

Read the postfix expression one character at a time until it encounters the delimiter '#'.
 Step 1: If the character is an operand, push it associated value on to the stack.

Step2 : If the character is an operator, POP two values from the stack, apply the operator to them and push the result onto the stack

Example:

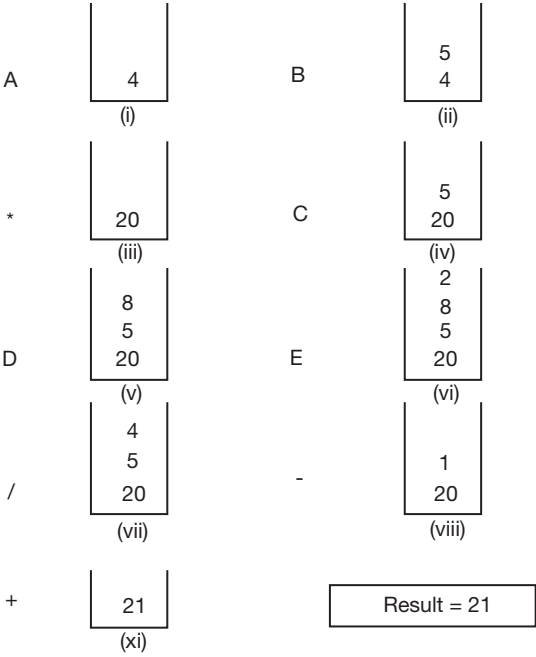
Let us consider the symbols A,B, C, D,E had the associated values:

Read Character	Symbol	Value
	A	4
	B	5
	C	5
	D	8
	E	2

Stack

Read Character Stack

AQ: Please check
this table it seems
it is corrupted



Evaluation of AB*CDE/-+

Balancing the Symbols:

Read one character at a time until it encounters the delimiter '#'.

Step 1: If the character is an opening symbol, push it on to the stack.

Step 2: If the character is a closing symbol, and if the stack is empty report an error as missing opening symbol.

Step 3: If it is a closing symbol and if it has corresponding opening symbol in the stack, POP it from the stack. Otherwise, report an error as mismatched symbols.

Step 4: At the end of file, if the stack is not empty, report an error as Missing closing symbol. Otherwise, report as balanced symbols.

Let us consider the expression as (a+b) #

Read Character	Stack	Read Character	Stack
(	(	a	(
+	(	b	(
)		#	
		Stack is empty	

Illustration for Balanced Symbols

Towers of Hanoi

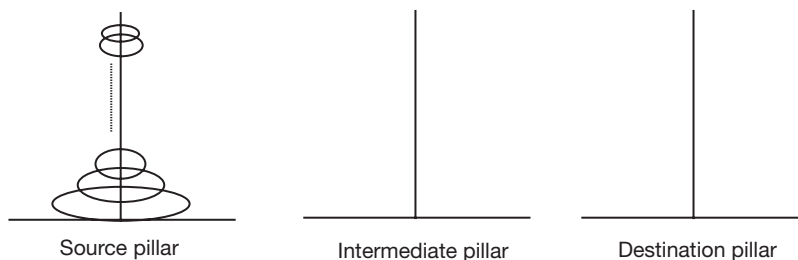
Towers of Hanoi is one of the example illustrating the Recursion technique.

The problem is moving a collection of N disks of decreasing size from one pillar to another pillar. The movement of the disk is restricted by the following rules.

Rule 1 : Only one disk could be moved at a time.

Rule 2: No larger disk could ever reside on a pillar on top of a smaller disk.

Rule 3: A 3rd pillar could be used as an intermediate to store one or more disks, while they were being moved from source to destination



Towers of Hanoi problems

Recursive Solution

N-represents the number of disks.

Step 1: If $N=1$, move the disk from A to C.

Step 2: If $N=2$, move the 1st disk from A to B.

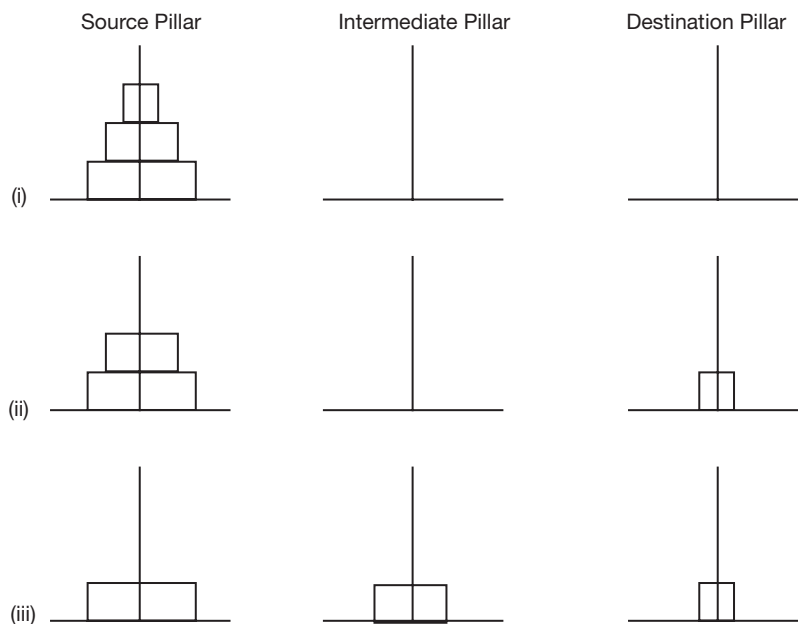
Then move the 2nd disk from A to C,

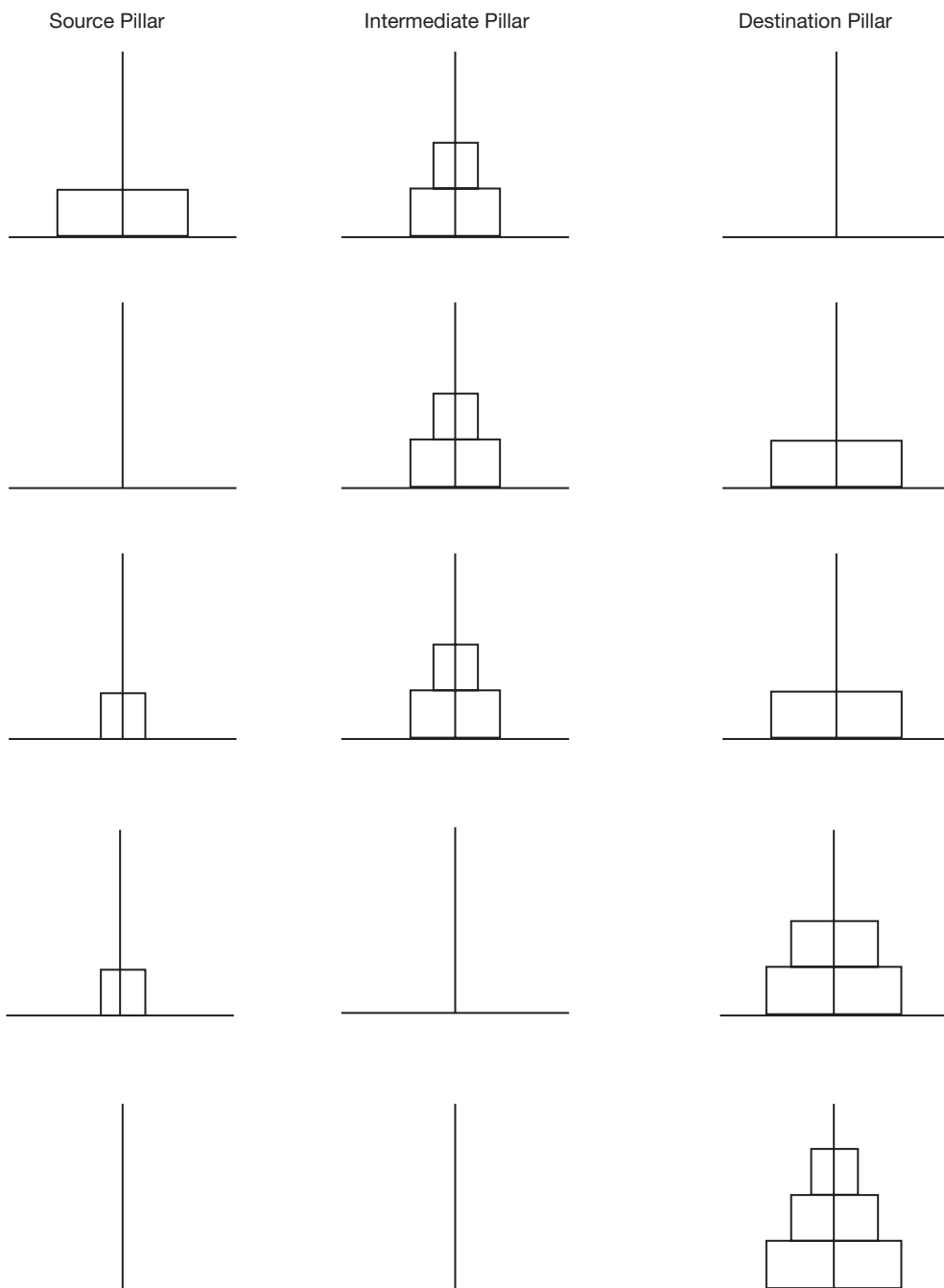
Then move the 1st disk from B to C.

Step 3: If $N=3$, Repeat the step (2) to move the first 2 disks from A to B using C as intermediate.

Then the 3rd disk is moved from A to C. Then repeat the step (2) to move 2 disks from B to C using A as intermediate.

```
def hanoi(n, source, destination, intermediate):
    if n == 1:
        print(f"Move disk from {source} to {destination}")
        return
    else:
        hanoi(n - 1, source, intermediate, destination)
        print(f"Move disk from {source} to {destination}")
        hanoi(n - 1, intermediate, destination, source)
hanoi(3, 'A', 'C', 'B')
```



**Towers of Hanoi Problem for 3 Disks**

THE Queue ADT

Queue Model

A Queue is a linear data structure which follows First In First Out (FIFO) principle, in which insertion is performed at rear end and deletion is performed at front end.



Queue Model

Example: Waiting Line in Reservation Counter

Operations on Queue

The fundamental operations performed on queue are

1. Enqueue
2. Dequeue

Enqueue:

The process of inserting an element in the queue.

Dequeue:

The process of deleting an element from the queue.

Exception Conditions

Overflow: Attempt to insert an element, when the queue is full is said to be overflow condition.

Underflow: Attempt to delete an element from the queue, when the queue is empty is said to be underflow.

Implementation of Queue

Queue can be implemented using arrays and pointers.

Array Implementation

In this implementation queue Q is associated with two pointers namely rear pointer and front pointer.

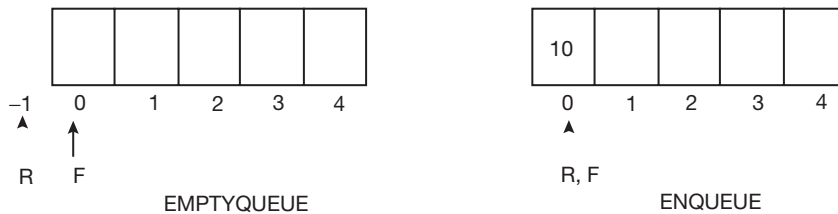
To insert an element X onto the Queue Q, the rear pointer is incremented by 1 and then set Queue [Rear] = X

To delete an element, the Queue [Front] is returned and the Front Pointer is incremented by 1.

Routine to Enqueue:

An enqueue operation is the process of adding an element to a queue or deque (double-ended queue) in a data structure.

```
def is_full(self):
    return self.rear >= self.max_size - 1
def enqueue(self, x):
    if self.is_full():
        print("Queue Overflow")
        return
    self.rear += 1
    self.queue[self.rear] = x
```

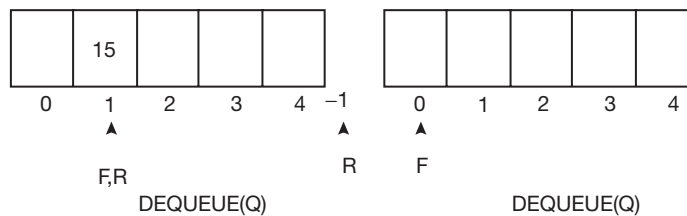


Enqueue on queue

Routine for Dequeue:

A routine for dequeue refers to the algorithm or procedure used to remove an element from a queue or deque (double-ended queue) in a data structure.

```
def delete(self):
    if self.is_empty():
        print("Queue Underflow")
        return
    x = self.queue[self.front]
    print(x)
    if self.front == self.rear:
        self.front = 0
        self.rear = -1
    else:
        self.front += 1
```



Dequeue on queue

Linked List Implementation of Queue

A queue implemented using a linked list follows the FIFO (First In, First Out) principle, where elements are added at the rear (enqueue) and removed from the front (dequeue).

- Enqueue operation is performed at the end of the list
- Dequeue operation is performed at the front of the list.

Routine to check an empty queue:

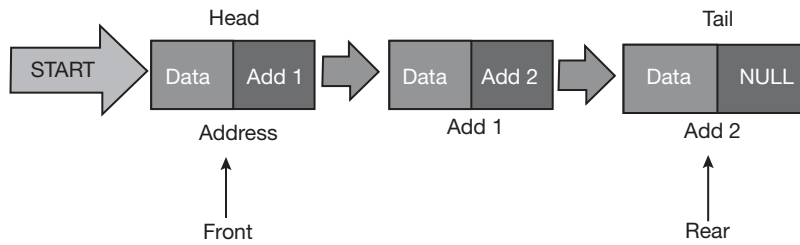
It means, the queue has no elements.

```
def make_empty(self):
    if self is None:
        print("Create Queue First")
        return
    while not self.is_empty():
        self.dequeue()
def create_queue():
    q = Queue()
    if q is None:
        print("Out of Space")
        return None
    q.make_empty()
    return q
```

Routine to enqueue an element in queue:

In a linked list implementation of a queue, the enqueue operation inserts an element at the rear (tail) of the queue. Unlike an array-based queue, a linked list queue does not have a fixed size and dynamically grows as elements are added.

```
class Queue:
    def __init__(self):
        self.front = None
        self.rear = None
    def enqueue(self, x):
        new_node = Node(x)
        if self.rear is None:
            self.front = new_node
            self.rear = new_node
        else:
            self.rear.next = new_node
            self.rear = new_node
```

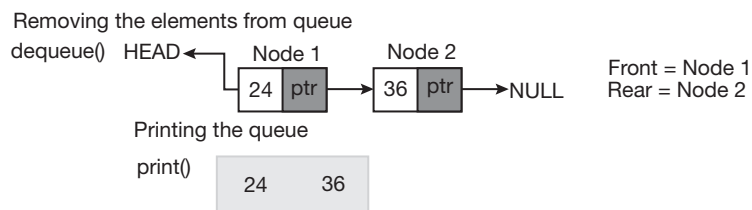


Enqueue operation

Routine to dequeue an element from the queue:

In a linked list implementation of a queue, the dequeue operation removes an element from the front (head) of the queue.

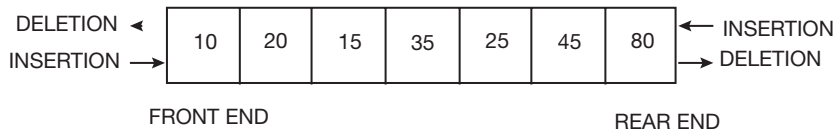
```
def dequeue(self):
    if self.front is None:
        print("Queue is underflow")
        return
    temp = self.front
    if self.front == self.rear:
        self.front = None
        self.rear = None
    else:
        self.front = self.front.next
    print(temp.data)
    del temp
```



Dequeue operation

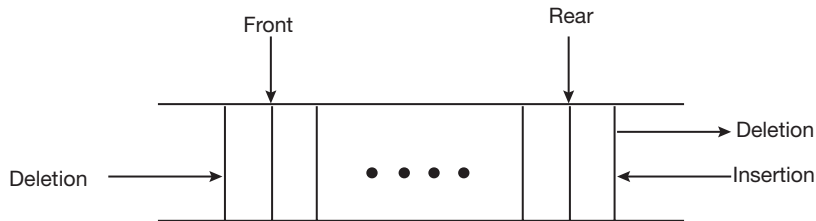
Double Ended Queue (Deque)

A Double-Ended Queue (Deque) is a linear data structure that allows insertion and deletion of elements from both ends (front and rear). It is a generalized form of a queue that provides more flexibility.



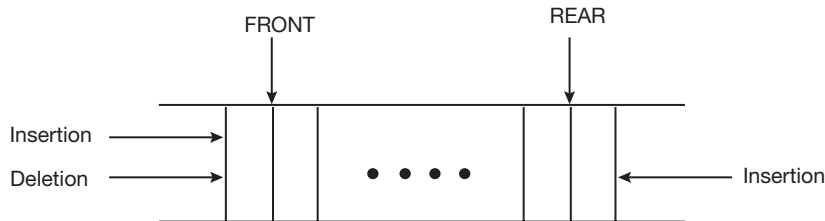
Input Restricted Deque:

Insertion is allowed at only one end, but deletion is possible from both ends.



Output Restricted Deque:

In this type deletions are allowed at one end and insertions are allowed at both ends.



Applications of Queue

- Batch processing in an operating system
- To implement Priority Queues.
- Priority Queues can be used to sort the elements using Heap Sort.
- Simulation.
- Mathematics user Queuing theory.
- Computer networks where the server takes the jobs of the client as per the queue strategy.

Sorting and Searching

Bubble Sort

Bubble Sort is a simple comparison-based sorting algorithm. It works by repeatedly stepping through the list, comparing adjacent elements, and swapping them if they are in the wrong order. The process is repeated for each element of the array until the entire array is sorted. The algorithm gets its name from the way larger elements “bubble” to the end of the array with each iteration. Despite its simplicity, Bubble Sort is not suitable for large datasets as its time complexity is relatively high.

How Bubble Sort Works

- **Comparison and Swap:** Bubble Sort compares each element with its adjacent element in the array. If the element is greater than the adjacent element, the two elements are swapped.
- **Multiple Passes:** The algorithm passes through the array multiple times. After each pass, the largest unsorted element is placed in its correct position at the end of the array.
- **Termination:** The process repeats until no swaps are needed, indicating that the array is fully sorted.

Step-by-Step Explanation

Let's consider an example array:

[5, 2, 9, 1, 5, 6]

Pass 1:

- Compare 5 and 2: Swap them → [2, 5, 9, 1, 5, 6]
- Compare 5 and 9: No swap.
- Compare 9 and 1: Swap them → [2, 5, 1, 9, 5, 6]
- Compare 9 and 5: Swap them → [2, 5, 1, 5, 9, 6]
- Compare 9 and 6: Swap them → [2, 5, 1, 5, 6, 9]
- Now, 9 is in its correct position.

Pass 2:

- Compare 2 and 5: No swap.
- Compare 5 and 1: Swap them $\rightarrow$ [2, 1, 5, 5, 6, 9]
- Compare 5 and 5: No swap.
- Compare 5 and 6: No swap.
- Now, 6 is in its correct position.

Pass 3:

- Compare 2 and 1: Swap them $\rightarrow$ [1, 2, 5, 5, 6, 9]
- Compare 2 and 5: No swap.
- Compare 5 and 5: No swap.
- Now, the array is fully sorted: [1, 2, 5, 5, 6, 9]

Bubble Sort Function in Python

```
def bubble_sort(arr):
    n = len(arr)
    for i in range(n):
        # Flag to check if any swapping happens
        swapped = False
        for j in range(0, n-i-1): # Last i elements are already sorted
            if arr[j] > arr[j+1]:
                # Swap if the element is greater than the next
                arr[j], arr[j+1] = arr[j+1], arr[j]
                swapped = True
        # If no swapping occurs, array is already sorted
        if not swapped:
            break
# Example Usage
arr = [5, 2, 9, 1, 5, 6]
bubble_sort(arr)
print("Sorted array:", arr)
```

Time Complexity of Bubble Sort

- Best Case: $O(n)$
 - This occurs when the array is already sorted. A small optimization can be made by checking if a pass over the array results in no swaps. If no swaps are made, the algorithm terminates early.

- Worst Case: $O(n^2)$
 - This occurs when the array is sorted in reverse order. In this case, the algorithm must compare every element with every other element.
- Average Case: $O(n^2)$
 - On average, the algorithm makes a quadratic number of comparisons and swaps.

Space Complexity

- Space Complexity: $O(1)$
 - Bubble Sort is an in-place sorting algorithm, meaning it does not require extra space beyond the input array.

Advantages of Bubble Sort

1. Simplicity:
 - The Bubble Sort algorithm is very simple to understand and implement, making it a good choice for educational purposes.
2. Stable Sorting:
 - Bubble Sort is a stable sort, meaning that the relative order of equal elements remains unchanged.
3. Adaptive:
 - If the array is already sorted or nearly sorted, Bubble Sort can detect this early and terminate, improving performance in best-case scenarios.

Disadvantages of Bubble Sort

1. Inefficient for Large Datasets:
 - With a time complexity of $O(n^2)$, Bubble Sort becomes very inefficient for large datasets. Other algorithms like Merge Sort, Quick Sort, or Heap Sort are generally much faster for larger arrays.
2. High Number of Comparisons and Swaps:
 - Even though it is simple, Bubble Sort makes a lot of comparisons and swaps, which can make it slower compared to more optimized sorting algorithms.

Selection Sort

Selection Sort is a comparison-based sorting algorithm that works by dividing the array into two parts: the sorted part and the unsorted part. In each iteration, the algorithm selects the smallest (or largest) element from the unsorted part of the array and swaps it with the first element of the unsorted part, thereby extending the sorted part by one element. Despite its simplicity, Selection Sort is not efficient for large datasets due to its $O(n^2)$ time complexity.

How Selection Sort Works

- **Find the Minimum/Maximum:** The algorithm starts by finding the minimum (or maximum) element in the unsorted part of the array.
- **Swap:** After identifying the smallest (or largest) element, it is swapped with the element at the beginning of the unsorted part.
- **Repeat:** The process repeats for the next unsorted portion of the array until the entire array is sorted.

Step-by-Step Explanation

Let's consider an example array:

csharp

Copy

[64, 25, 12, 22, 11]

Pass 1:

1. The algorithm starts by finding the smallest element in the entire array. The smallest element is **11**.
2. Swap **11** with the first element **64**.

After Pass 1, the array looks like this:

[11, 25, 12, 22, 64]

Now, the first element (11) is in its correct position.

Pass 2:

1. Now, find the smallest element in the remaining unsorted part of the array, which is **12**.
2. Swap **12** with **25**.

After Pass 2, the array looks like this:

[11, 12, 25, 22, 64]

Now, the first two elements (11 and 12) are in their correct positions.

Pass 3:

1. Find the smallest element in the remaining unsorted part of the array, which is **22**.
2. Swap **22** with **25**.

After Pass 3, the array looks like this:

[11, 12, 22, 25, 64]

Now, the first three elements (11, 12, and 22) are in their correct positions.

Pass 4:

1. The smallest element in the remaining unsorted part is **25**, which is already in its correct position, so no swap is needed.

After Pass 4, the array looks like this:

[11, 12, 22, 25, 64]

Now, the array is fully sorted

Selection Sort Function in Python

```
def selection_sort(arr):
    n = len(arr)

    # Traverse through all array elements
    for i in range(n):
        # Find the minimum element in the unsorted array
        min_idx = i
        for j in range(i+1, n):
            if arr[j] < arr[min_idx]:
                min_idx = j

        # Swap the found minimum element with the first
        # element
        arr[i], arr[min_idx] = arr[min_idx], arr[i]

# Example Usage
arr = [64, 25, 12, 22, 11]
selection_sort(arr)
print("Sorted array:", arr)
```

Time Complexity of Selection Sort

- **Best Case: $O(n^2)$**
This occurs when the array is already sorted. Even in the best case, the algorithm needs to make the same number of comparisons.
- **Worst Case: $O(n^2)$**
This occurs when the array is sorted in reverse order. The algorithm will still have to make the same number of comparisons and swaps.
- **Average Case: $O(n^2)$**
On average, Selection Sort performs the same number of comparisons regardless of the initial order of the array.

Advantages of Selection Sort

- **Simplicity:**
Selection Sort is simple to understand and implement, making it a good choice for educational purposes.

- **In-Place Sorting:**

Selection Sort is an in-place sorting algorithm, meaning it does not require additional space for sorting, which can be beneficial when memory is limited.

- **Constant Number of Swaps:**

Selection Sort always performs the same number of swaps ($n-1$), making it predictable

Disadvantages of Selection Sort

1. **Inefficient for Large Datasets:**

- With a time complexity of $O(n^2)$, Selection Sort becomes very inefficient for large datasets. Other algorithms like **Merge Sort**, **Quick Sort**, or **Heap Sort** are generally much faster for larger arrays.

2. **High Number of Comparisons:**

- Selection Sort makes a large number of comparisons, even if the array is partially sorted, which can make it slower than other optimized algorithms.

3. **Not Stable:**

- Selection Sort is **not stable**, meaning that it may change the relative order of equal elements in the array.

Insertion Sort

Insertion Sort is a simple comparison-based sorting algorithm that works by building a sorted portion of the array one element at a time. It does this by repeatedly taking the next element from the unsorted portion of the array and inserting it into the correct position within the sorted portion. It's often compared to the way people sort playing cards by picking up one card at a time and inserting it into its correct position in their hand. Although it is simple and easy to implement, Insertion Sort is inefficient for large datasets because of its $O(n^2)$ time complexity. However, for small datasets or nearly sorted arrays, it can be quite efficient.

How Insertion Sort Works

1. **Iterate through the Array:** Insertion Sort starts with the second element and compares it with the first element. If it's smaller, it's inserted into the correct position in the sorted portion.
2. **Shift Elements:** If an element is larger than the current element being considered, it shifts the larger elements one position to the right to make space for the smaller element.
3. **Repeat:** The process continues with the next unsorted element until the entire array is sorted.

Example

Consider an unsorted array as follows,

20 10 60 40 30 15

ORIGINAL	20	10	60	40	30	15	POSITIONS MOVED
After i = 1	10	20	60	40	30	15	1
After i = 2	10	20	60	40	30	15	0
After i = 3	10	20	40	60	30	15	1
After i = 4	10	20	30	40	60	15	2
After i = 5	10	15	20	30	40	60	4
Sorted Array	10	15	20	30	40	60	

Time Complexity of Insertion Sort

- **Best Case: $O(n)$**

This occurs when the array is already sorted. In this case, only one comparison is needed for each element, and no shifts are required.

- **Worst Case: $O(n^2)$**

This occurs when the array is sorted in reverse order. The algorithm needs to compare and shift each element multiple times.

- **Average Case: $O(n^2)$**

On average, the algorithm will make about $n^2/2$ comparisons and shifts, making it inefficient for larger datasets.

Advantages of Insertion Sort

- **Simple to Implement:**

Insertion Sort is easy to understand and implement, making it an ideal choice for educational purposes or small problems.

- **Efficient for Small or Nearly Sorted Arrays:**

When the array is small or nearly sorted, Insertion Sort can perform very efficiently, often faster than more complex algorithms.

- **Stable Sort:**

Insertion Sort is stable, meaning that equal elements maintain their relative order after sorting, which is useful in certain applications.

- **Adaptive:**

If the array is nearly sorted, the algorithm will detect this quickly and make fewer comparisons and shifts.

Disadvantages of Insertion Sort

1. Inefficient for Large Datasets:

- With a time complexity of $O(n^2)$, Insertion Sort becomes very inefficient for large datasets. Other algorithms like **Merge Sort**, **Quick Sort**, or **Heap Sort** are generally faster for larger arrays.

2. High Number of Comparisons and Shifts:

- Even though it performs well for small arrays, Insertion Sort requires a lot of comparisons and shifts for larger datasets, making it less efficient.

Merge Sort

Merge Sort is a divide-and-conquer algorithm that splits the input array into smaller subarrays, recursively sorts them, and then merges them back together. It is a highly efficient and stable sorting algorithm, often used for sorting large datasets. The main idea behind Merge Sort is to break down a problem into smaller parts, solve each part, and then combine the solutions. This method is particularly useful for sorting linked lists or large datasets where other algorithms like Quick Sort might not be optimal. Merge Sort operates with a time complexity of $O(n \log n)$, making it significantly faster than algorithms like Bubble Sort or Selection Sort for large datasets.

How Merge Sort Works

- **Divide:** The array is divided into two halves. This division continues recursively until each subarray has only one element.
- **Conquer:** Each subarray is sorted individually (though a single-element array is trivially sorted).
- **Combine:** The sorted subarrays are merged back together to form a larger sorted array.

Step-by-Step Explanation

Let's consider the array:

[38, 27, 43, 3, 9, 82, 10]

Step 1: Divide

First, we split the array into two halves:

[38, 27, 43, 3] and [9, 82, 10]

Step 2: Recursively Split

- Split the left subarray:
 - [38, 27] and [43, 3]
- Split the right subarray:
 - [9, 82] and [10]

- Continue splitting until each subarray has only one element:
 - [38] [27] [43] [3] [9] [82] [10]

Step 3: Merge and Sort

Now we begin merging and sorting:

1. Merge [38] and [27]:
 - Compare 38 and 27. Since $27 < 38$, we merge them into [27, 38].
2. Merge [43] and [3]:
 - Compare 43 and 3. Since $3 < 43$, we merge them into [3, 43].
3. Merge [9] and [82]:
 - Compare 9 and 82. Since $9 < 82$, we merge them into [9, 82].
4. The array is now:
[27, 38] [3, 43] [9, 82] [10]

Step 4: Final Merge

Now merge all subarrays:

1. Merge [27, 38] and [3, 43]:
 - Compare 27 and 3. Since $3 < 27$, we merge them into [3].
 - Compare 27 and 43. Since $27 < 43$, we merge them into [3, 27].
 - Finally, append 38 and 43: [3, 27, 38, 43].
2. Merge [9, 82] and [10]:
 - Compare 9 and 10. Since $9 < 10$, we merge them into [9, 10].
 - Finally, append 82: [9, 10, 82].
3. Merge the two sorted subarrays [3, 27, 38, 43] and [9, 10, 82]:
 - Compare 3 and 9. Since $3 < 9$, we merge them into [3].
 - Compare 27 and 9. Since $9 < 27$, we merge them into [3, 9].
 - Compare 27 and 10. Since $10 < 27$, we merge them into [3, 9, 10].
 - Compare 27 and 82. Since $27 < 82$, we merge them into [3, 9, 10, 27].
 - Compare 38 and 82. Since $38 < 82$, we merge them into [3, 9, 10, 27, 38].
 - Finally, append 43 and 82: [3, 9, 10, 27, 38, 43, 82].

Now, the array is fully sorted:

[3, 9, 10, 27, 38, 43, 82]

Merge Sort Algorithm

Merge Sort Function in Python

```
def merge_sort(arr):
    if len(arr) > 1:
        mid = len(arr) // 2 # Find the middle of the array
        left_half = arr[:mid] # Divide the array into two
        halves
        right_half = arr[mid:]
        # Recursively sort the two halves
        merge_sort(left_half)
        merge_sort(right_half)
        i = j = k = 0
        # Merge the sorted halves
        while i < len(left_half) and j < len(right_half):
            if left_half[i] < right_half[j]:
                arr[k] = left_half[i]
                i += 1
            else:
                arr[k] = right_half[j]
                j += 1
            k += 1
        # Check if any element was left in the left half
        while i < len(left_half):
            arr[k] = left_half[i]
            i += 1
            k += 1
        # Check if any element was left in the right half
        while j < len(right_half):
            arr[k] = right_half[j]
            j += 1
            k += 1
# Example Usage
arr = [38, 27, 43, 3, 9, 82, 10]
merge_sort(arr)
print("Sorted array:", arr)
```

Time Complexity of Merge Sort

- **Best Case:** $O(n \log n)$
 - Even if the array is already sorted, Merge Sort still performs the same number of operations due to its divide-and-conquer approach.

- **Worst Case:** $O(n \log n)$
 - The worst-case scenario still involves the same number of comparisons and divisions as the best case.
- **Average Case:** $O(n \log n)$
 - On average, Merge Sort performs a logarithmic number of divisions and linear work for each division.

Advantages of Merge Sort

1. **Efficient for Large Datasets:**
 - Merge Sort operates in $O(n \log n)$ time, making it much more efficient for large datasets compared to algorithms like Bubble Sort or Selection Sort.
2. **Stable Sorting Algorithm:**
 - Merge Sort is stable, meaning that it preserves the relative order of equal elements, which can be crucial in some applications.
3. **Works Well with Linked Lists:**
 - Unlike many other sorting algorithms, Merge Sort can efficiently sort linked lists, where random access is not possible.

Disadvantages of Merge Sort

1. **Space Complexity:**
 - Merge Sort requires additional space for the subarrays, which can be a disadvantage in memory-constrained environments.
2. **Slower for Smaller Arrays:**
 - Although Merge Sort is efficient for large datasets, for smaller datasets, algorithms like Insertion Sort or even Bubble Sort can be faster in practice due to lower constant factors.

Quick Sort

Quick Sort is a highly efficient and widely used sorting algorithm that follows the divide-and-conquer approach. It works by selecting a “pivot” element from the array and partitioning the other elements into two subarrays according to whether they are less than or greater than the pivot. The subarrays are then sorted recursively. Quick Sort is known for its $O(n \log n)$ average-case time complexity, making it faster than algorithms like Bubble Sort or Insertion Sort for large datasets. Despite its efficiency, Quick Sort’s worst-case time complexity is $O(n^2)$, which occurs when the pivot elements are chosen poorly. However, with proper pivot selection strategies (e.g., randomization or median-of-three), this can be minimized, and Quick Sort is usually faster than Merge Sort and Heap Sort in practice.

How Quick Sort Works

- **Choose a Pivot:** Select a pivot element from the array. The choice of pivot can impact performance (common strategies include choosing the first element, the last element, or a random element).
- **Partitioning:** Rearrange the array such that all elements less than the pivot come before it, and all elements greater than the pivot come after it. The pivot is now in its correct position.
- **Recursion:** Recursively apply the same process to the subarrays formed by splitting at the pivot. Continue this until the subarrays have only one element.

Step-by-Step Explanation

Let's consider the array:

[10, 80, 30, 90, 40, 50, 70]

Step 1: Choose a Pivot

For this example, let's choose the **last element** as the pivot (in this case, **70**).

Step 2: Partition the Array

We rearrange the array such that elements smaller than **70** come before it, and elements larger than **70** come after it:

- Compare each element with **70**.
- After partitioning:

[10, 30, 40, 50, 70, 90, 80]

The pivot **70** is now at its correct position, index 4.

Step 3: Recursively Apply Quick Sort

Now, we recursively apply the same process to the subarrays:

1. Left subarray: [10, 30, 40, 50]
2. Right subarray: [90, 80]

Left Subarray [10, 30, 40, 50]

1. Choose **50** as the pivot.
2. Partition the array around **50**:
[10, 30, 40, 50]
Now, **50** is at its correct position, index 3.
3. Recursively apply Quick Sort to the left subarray [10, 30, 40].

Left Subarray [10, 30, 40]

1. Choose **40** as the pivot.
2. Partition around **40**:
[10, 30, 40]
Now, **40** is at its correct position, index 2.
3. Recursively apply Quick Sort to the left subarray [10, 30].

Left Subarray [10, 30]

1. Choose **30** as the pivot.
2. Partition around **30**:
[10, 30]
Now, **30** is at its correct position, index 1.
The subarrays [10] and [30] are already sorted.

Right Subarray [90, 80]

1. Choose **80** as the pivot.
2. Partition around **80**:

[80, 90]

Now, **80** is at its correct position, index 5.

The array is now fully sorted:

[10, 30, 40, 50, 70, 80, 90]

Quick Sort Function in Python

```
def quick_sort(arr):  
    if len(arr) <= 1:  
        return arr  
    else:  
        pivot = arr[len(arr) // 2] # Choose the middle  
        element as the pivot  
        left = [x for x in arr if x < pivot] # Elements  
        less than the pivot  
        middle = [x for x in arr if x == pivot] # Elements  
        equal to the pivot  
        right = [x for x in arr if x > pivot] # Elements  
        greater than the pivot  
        return quick_sort(left) + middle + quick_sort(right)  
# Recursively sort and combine  
# Example Usage  
arr = [10, 80, 30, 90, 40, 50, 70]  
sorted_arr = quick_sort(arr)  
print("Sorted array:", sorted_arr)
```

Time Complexity of Quick Sort

Best Case: $O(n \log n)$

- This occurs when the pivot splits the array into two roughly equal subarrays.

Worst Case: $O(n^2)$

- This occurs when the pivot is consistently the smallest or largest element, leading to unbalanced splits. This can happen if the array is already sorted or nearly sorted.

Average Case: $O(n \log n)$

- On average, Quick Sort will split the array into balanced subarrays, resulting in logarithmic recursion depth and linear work for each level.

Advantages of Quick Sort

- **Efficient for Large Datasets:**
Quick Sort performs very well for large datasets with a time complexity of $O(n \log n)$ on average, making it faster than algorithms like Bubble Sort or Insertion Sort.
- **In-place Sorting:**
Quick Sort does not require additional memory beyond the input array, making it memory efficient compared to algorithms like Merge Sort.
- **Cache Efficient:**
The algorithm is cache efficient, making it a good choice for sorting large datasets in practical applications.

Disadvantages of Quick Sort

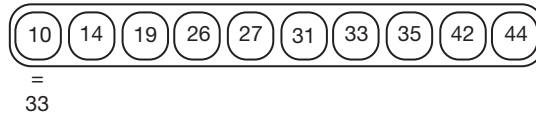
- **Worst-case Time Complexity:**
 - The worst-case time complexity is $O(n^2)$, which can happen when the pivot is poorly chosen (e.g., always the smallest or largest element). This can be mitigated by using more advanced pivot selection methods (e.g., median-of-three or random pivot).
- **Not Stable:**
 - Quick Sort is not a stable sort, meaning it does not preserve the relative order of equal elements. This can be problematic in certain scenarios.
- **Complexity of Implementation:**
 - While Quick Sort is conceptually simple, it can be tricky to implement correctly, especially with edge cases like handling equal elements or choosing an appropriate pivot.

Linearsearch

Linear search is a very basic and simple search algorithm. In Linear search, we search an element or value in a given array by traversing the array from the starting, till the desired Element or value is found.

It compares the element to be searched with all the elements present in the array and when the element is matched successfully, it returns the index of the element in the array, else it return -1.

Linear Search



Linear Search is applied on unsorted or unordered lists, when there are fewer elements in a list. For Example,

Algorithm

Linear Search (Array A, Value x)

Step 1: Set i to 1

Step 2: if $i > n$ then go to step 7

Step 3: if $A[i] = x$ then go to step 6

Step 4: Set i to $i + 1$

Step 5: Go to Step 2

Step 6: Print Element x Found at index i and go to step 8

Step 7: Print element not found

Step 8: Exit Pseudocode

procedure linear_search (list, value)

for each item in the list if match item == value

return the item's location end if

end for

end procedure

Python program for Linear Search

```
# Input the number of elements
n = int(input("How many elements? "))
# Input the elements of the array
a = [ ]
print("Enter array elements:")
for i in range(n):
    a.append(int(input()))
# Input the element to search
x = int(input("\nEnter element to search: "))
# Perform linear search
for i in range(n):
    if a[i] == x:
        print(f"Element found at index {i}")
        break
else:
    print("Element not found")
```

Time Complexity of Linear Search

- **BestCase: $O(1)$**
This occurs when the element to be found is the first element in the array. The algorithm only needs one comparison to find the element.
- **Worst Case: $O(n)$**
This happens when the element is not present in the array, or the element is the last one. In both cases, every element in the array needs to be checked.
- **Average Case: $O(n)$**
On average, the algorithm will check half of the elements in the list. This makes the time complexity $O(n)$ in the average case, where n is the number of elements in the array.

Advantages of Linear Search

- **Simplicity:**
Linear Search is easy to implement and understand. It is a straightforward algorithm that doesn't require any special knowledge or complex logic.
- **No Sorting Required:**
Unlike binary search, which requires the array to be sorted, linear search works on unsorted arrays. This makes it useful for situations where you don't want to or cannot sort the array.
- **Works with Any Data Structure:**
Linear Search works on any data structure that allows sequential access, including arrays, lists, linked lists, and others. It doesn't need random access to the elements.
- **Adaptive:**
If the element you're searching for is near the beginning of the array, linear search can be quite fast, as it may find the element in the first few comparisons.

Disadvantages of Linear Search

- **Inefficient for Large Datasets:**
Linear Search has a time complexity of $O(n)$, meaning it becomes inefficient as the size of the dataset grows. For large arrays or lists, more efficient algorithms like binary search (for sorted data) or hashing can be much faster.
- **High Number of Comparisons:**
In the worst case, linear search may have to compare every element in the list, leading to a significant number of comparisons and making it slower than more advanced algorithms for large datasets.

Binary Search

Binary Search is used with sorted array or list. In binary search, we follow the following steps

1. We start by comparing the element to be searched with the element in the middle of the list/array.
2. If we get a match, we return the index of the middle element.
3. If we do not get a match, we check whether the element to be searched is less or greater than in value than the middle element.
4. If the element/number to be searched is greater in value than the middle number, then we pick the elements on the right side of the middle element (as the list/array is sorted, hence on the right, we will have all the numbers greater than the middle number), and start again from the step 1.
5. If the element/number to be searched is lesser in value than the middle number, then we pick the elements on the left side of the middle element, and start again from the step 1.

Binary Search is useful when there are large number of elements in an array and they are sorted. So a necessary condition for Binary search to work is that the list/array should be sorted.

Features of Binary Search

1. It is great to search through large sorted arrays.
2. It has a time complexity of $O(\log n)$ which is a very good time complexity. It has a simple implementation.

Binary search is a fast search algorithm with run-time complexity of $O(\log n)$. This search algorithm works on the principle of divide and conquers. For this algorithm to work properly, the data collection should be in the sorted form.

Binary search looks for a particular item by comparing the middle most item of the collection. If a match occurs, then the index of item is returned. If the middle item is greater than the item, then the item is searched in the sub-array to the left of the middle item. Otherwise, the item is searched for in the sub-array to the right of the middle item. This process continues on the sub-array as well until the size of the sub array reduces to zero.

How Binary Search Works

Binary Search is an efficient algorithm for finding an element in a sorted array or list. The primary idea is to repeatedly divide the search interval in half. If the value of the search key is less than the item in the middle of the interval, the algorithm narrows the interval to the lower half. If the value is greater, it narrows the interval to the upper half. This continues until the target element is found or the interval is empty. Binary Search works only on sorted arrays or lists, and it has a time complexity of $O(\log n)$, making it much faster than linear search for large datasets.

Step-by-Step process

Step 1: Set Initial Values

Begin by setting the low and high pointers:

- low = 0 (the first index of the array)
- high = 9 (the last index of the array)

Step 2: Calculate the Mid Index

Now, we calculate the **mid** index using the formula:

$$\text{mid} = \text{low} + \left(\frac{\text{high} - \text{low}}{2} \right)$$

For the first iteration:

$$\text{mid} = 0 + \left(\frac{9 - 0}{2} \right) = 4$$

So, mid = 4. The element at index 4 is 27.

Step 3: Compare with Target

We compare the element at index 4 (27) with the target value 31:

- Since 27 is less than 31, we know that 31 must be in the upper half of the array.
Thus, we update low to be mid + 1, which is:
Low = 4 + 1 = 5
Now, we will search in the subarray starting from index 5 to index 9.

Step 4: Recalculate Mid and Compare Again

We recalculate the **mid** index for the new subarray:

$$\text{mid} = 5 + \left(\frac{9 - 5}{2} \right) = 7$$

Now, **mid** = 7. The element at index 7 is 35.

We compare 35 with 31:

- Since 35 is greater than 31, we know that 31 must be in the **lower half** of the subarray.
Thus, we update **high** to be **mid - 1**, which is:
High = 7 - 1 = 6
Now, we will search in the subarray starting from index 5 to index 6.

Step 5: Recalculate Mid and Compare Again

We recalculate the **mid** index for the new subarray:

$$\text{mid} = 5 + \left(\frac{6 - 5}{2} \right) = 5$$

Now, **mid** = 5. The element at index 5 is 31.

We compare 31 with 31:

- This time, we find a match!
We conclude that the element **31** is found at **index 5**.

Time Complexity of Binary Search

- **Best Case:** $O(1)$
This occurs if the middle element is the target in the first comparison.
- **Worst Case:** $O(\log n)$
In the worst case, the algorithm will halve the search space each time, leading to a logarithmic time complexity.
- **Average Case:** $O(\log n)$
On average, binary search will take logarithmic time due to halving the array each time.

Advantages of Binary Search

- **Efficient for Large Datasets:**
Binary Search is much faster than linear search for large datasets because it reduces the search space by half with each comparison. It has a time complexity of $O(\log n)$ compared to $O(n)$ for linear search.
- **Logarithmic Time Complexity:**
The $O(\log n)$ time complexity makes Binary Search highly efficient, especially for large arrays. This makes it ideal for searching in sorted data.
- **In-place Search:**
Binary Search requires only a constant amount of extra space, making it a space-efficient algorithm.
- **Fast for Sorted Data:**
If the data is already sorted, Binary Search can quickly find the desired element.

Disadvantages of Binary Search

- **Requires Sorted Data:**
Binary Search only works on sorted data. If the data is unsorted, it cannot be applied directly unless the array is first sorted.
- **Not Suitable for Linked Lists:**
Binary Search is more suited to arrays or data structures that allow random access. It is inefficient for linked lists as they do not support direct access to the middle element.
- **Complexity in Implementation:**
Compared to linear search, Binary Search can be more difficult to implement and understand, especially when dealing with edge cases like empty arrays or arrays with duplicate elements.

Hashing

HashTable

The hashtable data structure is an array of some fixed size, containing the keys. A key is a value associated with each record.

Location	Slot 1
1	
2	92
3	43
4	
5	85
6	
7	
8	
9	
10	

Hashing Function

A hashing function is a key - to - address transformation, which acts upon a given key to compute the relative position of the key in an array.

A simple Hash function

$$\text{HASH (KEYVALUE)} = \text{KEYVALUE MOD TABLESIZE}$$

Example : - Hash (92)

$$\text{Hash (92)} = 92 \bmod 10 = 2$$

The keyvalue '92' is placed in the relative location '2'.

Routine For Simple Hash Function

```
Hash (Char *key, int Table Size)
{
    int Hashvalue = 0;
    while (* key != '\0')
        Hashval + = * key ++;
    return Hashval % Tablesizes;
}
```

Some of the Methods of Hashing Function

1. ModuleDivision
2. Mid - SquareMethod
3. FoldingMethod
4. PSEUDO RandomMethod
5. Digit or Character ExtractionMethod
6. RadixTransformation.

Collisions

A Collision occurs when two or more elements are hashed (mapped) to same value (i.e) When two key values hash to the same position.

Collision Resolution

When two items hash to the same slot, there is a systematic method for placing the second item in the hash table. This process is called collision resolution.

Some of the Collision Resolution Techniques

1. SeperateChaining
2. 2.OpenAddressing
3. 3. MultipleHashing

Collision Handling

Collision in hashing occurs when two different keys hash to the same index in a hash table. Since each index of a hash table can only store one value, a collision results in the need to handle it so that all data can be stored and retrieved correctly. Effective collision handling is critical for maintaining the efficiency of a hash table, especially as the number of stored elements increases.

Types of Collisions

- **Primary Collision:** This occurs when two distinct keys produce the same hash value and are mapped to the same index in the hash table.
- **Secondary Collision:** After a primary collision is resolved, a secondary collision may occur when new keys also hash to the same index or nearby index.

Collision Handling Techniques

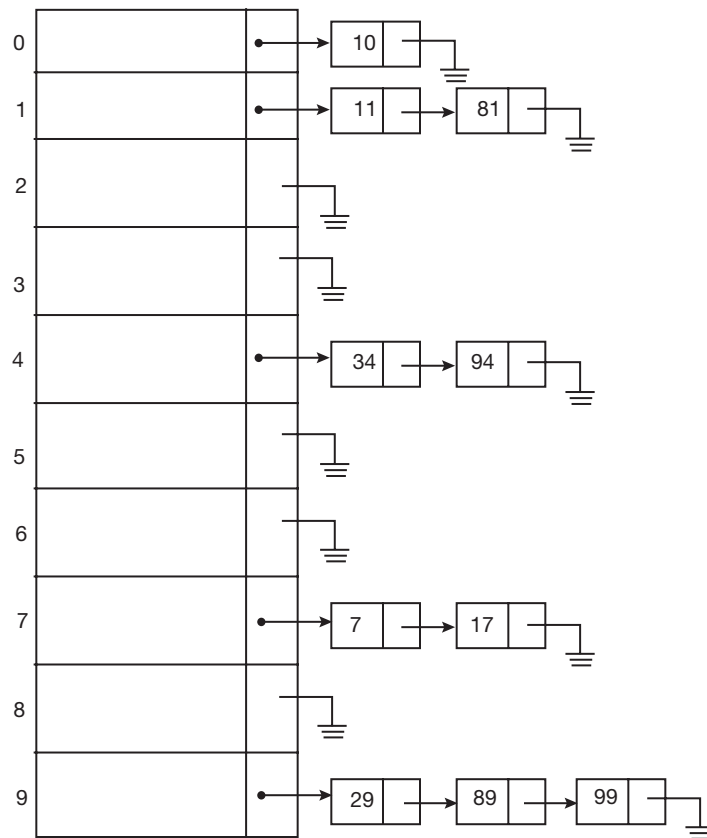
There are primarily two main methods for handling collisions in a hash table: **Open Addressing** and **Chaining**.

Separate Chaining

Separate chaining is an open hashing technique. A pointer field is added to each record location. When an overflow occurs this pointer is set to point to overflow blocks making a linked list.

In this method, the table can never overflow, since the linked list are only extended upon the arrival of new keys.

Insert: 10, 11, 81, 10, 7, 34, 94, 17



Insertion

- To perform the insertion of an element, traverse down the appropriate list to check whether the element is already in place.
- If the element is new one, the inserted it is either at the front of the list or at the end of the list.
- If it is a duplicate element, an extra field is kept and placed.

INSERT 10 :

Hash (k) = k % Tablesize Hash (10) = 10 % 10

Hash (10) = 0

INSERT 11 :

Hash (11) = 11 % 10

Hash (11) = 1

INSERT 81 :

Hash (81) = 81 % 10

Hash (81) = 1

- The element 81 collides to the same hash value 1. To place the value 81 at this position perform the following.
- Traverse the list to check whether it is already present.
- Since it is not already present, insert at end of the list. Similarly the rest of the elements are inserted.

Routine To Perform Insertion

```
void Insert (int key, Hashtable H)
```

```
{
    Position Pos, Newcell; List L;

    /* Traverse the list to check whether the key is
    already present */ Pos = FIND (Key, H);

    If (Pos == NULL) /* Key is not found */
    {
        Newcell = malloc (size of (struct ListNode));
        If (Newcell != NULL)
        {
            L = H TheLists [Hash (key, H Tablesize)];
            Newcell Next = L Next;
            Newcell Element = key;
            /* Insert the key at the front of the list
            L Next = Newcell;
            */ L Next = Newcell;
        }
    }
}
```

AQ: Please check this program and provide missing characters

Find Routine

AQ: Please check this program and provide missing characters

```

Position Find (int key, Hashtable H)
{
    Position P; List L;
    L = H TheLists [Hash (key, H Tablesize)]; P = L Next;
    while (P! = NULL && P Element != key) P = p Next;
    return p;
}

```

Advantage

More number of elements can be inserted as it uses array of linked lists.

Disadvantage of Seperate Chaining

- It requires pointers, which occupies more memoryspace.
- It takes more effort to perform a search,since it takes time to evaluate the hash function and also to traverse the list.

Open Addressing

Open addressing is also called **closed Hashing**, which is an alternative to resolve the collisions with linked lists.

In this hashing system,if a collision occurs,alternative cells are tried until an empty cell is found. (ie) cells $h_0(x)$, $h_1(x)$, $h_2(x)$. are tried insuccession.

INSERT : 42, 39, 69, 21, 71, 55, 33

EMPTY TABLE	After 42	After 39	After 69	After 21	After 71	After 55	After 33
0			69	69	69	69	69
1				21	21	21	21
2	42	42	42	42	42	42	42
3					71	71	71
4							33
5						55	55
6							
7							
8							
9		39	39	39	39	39	39

AQ: Please check if all content are placed properly from this section to end.

There are three common collision resolution strategies. They are

- (i) Linear Probing
- (ii) Quadratic probing
- (iii) Double Hashing.

Linear Probing

In linear probing, for the i th probe the position to be tried is $(h(k) + i) \bmod \text{table size}$, where $F(i) = i$, is the linear function.

In linear probing, the position in which a key can best red is found by sequentially searching all position starting from the position calculated by the hash function until an empty cell is found.

If the end of the table is reached and no empty cells has been found, then the search is continued from the beginning of the table. It has a tendency to create clusters in the table.

In fig 4.6.3 first collision occurs when 69 is inserted, which is placed in the next available spot namely spot 0, which is open.

The next collision occurs when 71 is inserted, which is placed in the next available spot namely spot 3. The collision for 33 is handled in a similar manner.

Advantage :

- It doesn't requires pointers

Disadvantage

- It forms clusters, which degrades the performance of the hash table for storing and retrieving data.

Quadratic probing

Quadratic Probing is similar to Linear probing. The difference is that if the element is inserted into a space that is filled, then $1^2 = 1$ element away then $2^2 = 4$ elements away, then $3^2 = 9$ elements away then $4^2 = 16$ elements away and so on is checked.

With linear probing, there is always an open spot found, if one. However, this is not the case with quadratic probing unless choosing of the table size is considered.

For example

Table size is 16. First 5 pieces of data that all hash to index 2

- First piece goes to index 2.
- Second piece goes to 3 $((2 + 1) \% 16)$
- Third piece goes to 6 $((2 + 4) \% 16)$
- Fourth piece goes to 11 $((2 + 9) \% 16)$
- Fifth piece doesn't get inserted because $(2 + 16) \% 16 == 2$ which is full so we end up back where we started and we haven't searched all empty spot

In order to guarantee that quadratic probes will hit every single available spots eventually, the table size must meet these requirements:

- Be a prime number
- never be more than half full (even by one element)

Double Hashing

Double Hashing works on a similar idea to linear and quadratic probing. A big table is used and hashed into it. Whenever a collision occurs, another spot is chosen in table to put the value. The difference here is that instead of choosing next opening, a second hash function is used to determine the location of the next spot. For example, given hash function H1 and H2 and key do the following:

- Check location $\text{hash1}(\text{key})$. If it is empty, put record in it.
- If it is not empty calculate $\text{hash2}(\text{key})$.
- check if $\text{hash1}(\text{key}) + \text{hash2}(\text{key})$ is open, if it is, put it in
- repeat with $\text{hash1}(\text{key}) + 2\text{hash2}(\text{key})$, $\text{hash1}(\text{key}) + 3\text{hash2}(\text{key})$ and so on, until an opening is found.

Like quadratic probing, choosing hash2 is considered . hash2 CANNOT ever return 0. hash2 must be done so that all cells will be probed eventually.

Load Factor and Efficiency in Hashing

in hashing, the load factor plays a critical role in determining the efficiency of operations such as insertion, search, and deletion. It is defined as the ratio of the number of elements stored in the hash table to the total available slots. A low load factor indicates that there is plenty of free space in the table, leading to fewer collisions and faster lookups. However, this can result in wasted memory. Conversely, a high load factor means the table is densely packed, increasing the chances of collisions, which slow down operations.

1. Load Factor (α) in Hashing

The **load factor** (α) is a crucial parameter that determines the efficiency of a **hash table**. It is defined as:

$$\alpha = \frac{N}{M}$$

where:

- **N** = Number of elements stored in the hash table
- **M** = Total size (capacity) of the hash table

2. Effects of Load Factor:

- **Low Load Factor ($\alpha < 0.5$):**
 - Less collisions
 - Wastes memory
 - Faster insertions and lookups
- **High Load Factor ($\alpha > 0.7$ or **1.0 in some cases**):**
 - More collisions
 - Increased probing time (for open addressing)
- **Triggers rehashing** (in dynamic hash tables)

Typical Threshold for Rehashing: When $\alpha > 0.7$, rehashing is triggered to resize the table and reduce collisions.

As the load factor approaches a certain threshold, typically around 0.7, hash tables that use dynamic resizing will trigger a rehashing process. This involves creating a new table with a larger size (often a prime number that is at least twice as large as the previous one), followed by reinserting all existing elements according to the new hash function. This helps maintain efficiency, ensuring that lookup and insertion times remain close to constant in the average case. The efficiency of a hash table depends on both the load factor and the method used to handle collisions. In separate chaining, where each slot contains a linked list of elements, an increase in load factor does not immediately degrade performance unless chains become excessively long. In contrast, open addressing techniques like linear probing and quadratic probing experience a decline in efficiency as the table fills up, since finding an empty slot requires additional comparisons. If the load factor becomes too high in these methods, primary and secondary clustering can severely affect performance, making insertions and searches significantly slower.

To maintain optimal efficiency, most implementations ensure that the load factor remains within a manageable range. Keeping it too low wastes space, while letting it grow too high increases collisions and degrades performance. The balance between memory usage and computational speed is key to ensuring a well-functioning hash table.

2. Efficiency in Hashing

The efficiency of a hash table depends on **collision resolution strategies** and the **load factor**.

(a) Time Complexity Analysis

Operation	Best Case (O)	Average Case (O)	Worst Case (O)
Search	$O(1)$	$O(1)$	$O(n)$ (if collisions are high)
Insert	$O(1)$	$O(1)$	$O(n)$ (if many collisions)
Delete	$O(1)$	$O(1)$	$O(n)$ (if many collisions)

Efficiency improves when the load factor is maintained within an optimal range (e.g., $\alpha \leq 0.7$).

(b) Impact of Collision Resolution Methods

1. **Separate Chaining:**

- Even with a high load factor, performance remains **$O(1)$ on average** if the chain lengths are small.
- Too many elements in a chain **degrade performance to $O(n)$** .

2. **Open Addressing (Linear Probing, Quadratic Probing, Double Hashing):**

- **Performance degrades as α increases** due to clustering.
- **Ideal efficiency is maintained when $\alpha \leq 0.5$.**

Rehashing

If the table becomes too full, the rehashing method creates a new table approximately twice the original size. It then scans the entire original hash table, recalculates the hash value for each element, and inserts it into the new table. Rehashing is a very expensive operation with a running time of $O(N)$, as there are N elements to rehash and the table size is roughly $2N$. Rehashing can be implemented in several ways with quadratic probing such as:

- Rehash, as soon as the table is half full.
- Rehash only when an insertion fails
- Rehash when the table reaches a certain load factor.

Routine for Rehashing for open addressing hash tables

```
class HashTable:
    def __init__(self, size):
        """Initialize the hash table with given size."""
        self.size = size
        self.table = [None] * size
        self.count = 0 # Tracks the number of elements

    def hash_function(self, key):
        """Simple hash function using modulo operation."""
        return key % self.size

    def insert(self, key):
        """Insert an element into the hash table."""
        index = self.hash_function(key)
```

```
# Linear probing for collision resolution
while self.table[index] is not None:
    index = (index + 1) % self.size # Move to the
next index

self.table[index] = key
self.count += 1
# Rehash if load factor exceeds 0.7
if self.count / self.size > 0.7:
    self.rehash()

def rehash(self):
    """Rehashes the hash table to a larger size (twice
the original)."""
    old_table = self.table
    old_size = self.size
    # Double the size and reinitialize table
    self.size *= 2
    self.table = [None] * self.size
    self.count = 0 # Reset count

    # Re-insert old elements
    for key in old_table:
        if key is not None:
            self.insert(key)

def display(self):
    """Display the hash table."""
    print("HashTable:", self.table)

# Example Usage
H = HashTable(5)
H.insert(10)
H.insert(20)
H.insert(30)
H.insert(40) # This should trigger rehashing
H.display()
```

Suppose the elements 13, 15, 24 and 6 are inserted into an open addressing hash table of size 7 with hash function $h(X) = X \bmod 7$ and if linear probing is used to resolve collisions, then the resulting hash table appears as follows:

0	6
1	15
2	
3	24
4	
5	
6	13

Open addressing hash table with linear probing with input 13, 15, 6, 24

If 23 is inserted into the table, the resulting table will be over 70 percent full.

0	6
1	15
2	23
3	24
4	
5	
6	13

A new table is created as the previous table is full. The size of this table is **17**, as this is the first prime number that is at least twice as large as the old table size. The new hash function is then defined as:

$$h(X) = X \bmod 17$$

The old table is scanned, and the elements 6, 15, 23, 24, and 13 are inserted into the new table based on this hash function.

0	
1	
2	
3	
4	
5	
6	6
7	23
8	24
9	
10	
11	
12	
13	13
14	
15	15
16	

Advantages

- Programmer doesn't worry about the table size.
- Simple to implement.
- Can be used in other data structures as well.

AQ: Please check
and provide
Chapte Title

4.1 Tree

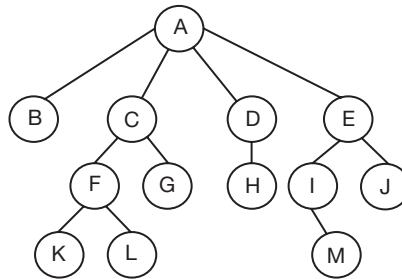
A **tree** is a hierarchical data structure composed of nodes, with one special node designated as the **Root**. This structure consists of a finite set of nodes where each node may have zero or more child nodes. The subtrees of the root are connected by directed edges.

A tree is a non-linear data structure with a root node and multiple levels of hierarchical nodes. It can be empty or consist of a root node with one or more subtrees.

4.1.1 Key Properties of Trees in Data Structures

- **Hierarchical Structure** – Trees organize data in a parent-child relationship.
- **Root Node** – The topmost node from which all other nodes originate.
- **Edges** – Connections between nodes that define relationships.
- **Nodes** – Individual elements containing data.
- **Parent and Child Nodes** – Nodes connected directly, where the parent has one or more children.
- **Subtree** – A branch of the tree that forms its own smaller tree.
- **Depth** – The number of edges from the root to a specific node.
- **Height** – The longest path from the root to a leaf node.
- **Leaf Nodes** – Nodes without children.
- **Degree** – The number of children a node has.
- **Balanced Tree** – A tree where the height difference between subtrees is minimal.
- **Binary Tree** – A tree where each node has at most two children.

Example : Let us consider



- **Root:**
 - In the given tree, **A** is the root.
- **Node:**
 - A basic unit of information in the tree.
- **Leaf Node (Terminal Node):**
 - In the given tree, **B, K, L, G, H, M, and J** are leaf nodes.
- **Siblings:**
 - **B, C, D, and E** (children of A)
 - **F and G** (children of C)
 - **I and J** (children of E)
 - **K and L** (children of F)
- **Path:**
 - The path from **A to L** is **A → C → F → L**.
- **Path Length:**
 - The path from **A to L** has a length of **3** (edges: A → C, C → F, F → L).
- **Degree of a Node:**
 - **Degree of A** = 4 (B, C, D, E)
 - **Degree of C** = 2 (F, G)
 - **Degree of D** = 1 (H)
 - **Degree of H** = 0 (leaf node)
- **Level of a Node:**
 - **A** is at level **1**
 - **B, C, D, and E** are at level **2**
 - **F, G, H, I, and J** are at level **3**
 - **K, L, and M** are at level **4**
- **Depth of a Node:**
 - **Depth of F** = 2 (A → C → F)
 - **Depth of L** = 3 (A → C → F → L)

- **Height of a Node:**
 - **Height of F = 1** ($F \rightarrow L$ or $F \rightarrow K$)
 - **Height of L = 0** (since L is a leaf node)

4.1.2 Common Types of Trees in Data Structures

- **General Tree**
A tree where each node can have any number of children.
- **Binary Tree**
Each node has at most two children (left and right).
- **Full Binary Tree**
Every node has either 0 or 2 children (no node has only one child).
- **Complete Binary Tree**
All levels are completely filled except possibly the last, which is filled from left to right.
- **Perfect Binary Tree**
A binary tree where all leaf nodes are at the same level, and every parent has exactly two children.
- **Balanced Binary Tree**
A binary tree where the height difference between left and right subtrees of any node is at most one.
- **Binary Search Tree (BST)**
A binary tree where the left subtree has smaller values and the right subtree has larger values than the root.
- **AVL Tree**
A self-balancing BST where the height difference between left and right subtrees is at most one.
- **Red-Black Tree**
A self-balancing BST where each node follows color-based rules to maintain balance.
- **N-ary Tree**
A tree where each node can have at most N children.

4.2 Tree Abstract Data Type (ADT)

A **Tree ADT (Abstract Data Type)** is a hierarchical data structure that consists of nodes connected by edges. It follows a parent-child relationship, where each node may have multiple child nodes but only one parent (except the root node).

4.2.1 Basic Operations in Tree ADT

- **Create** – Initializes an empty tree.
- **Insert** – Adds a new node to the tree.
- **Delete** – Removes a node from the tree.
- **Search** – Finds a node with a specific value.
- **Traverse** – Visits all nodes (Preorder, Inorder, Postorder, Level Order).
- **Find Parent/Child** – Retrieves parent or child nodes of a given node.
- **Find Height/Depth** – Determines the height of the tree or depth of a node.

4.2.2 Tree ADT Implementation in Python Using a Node Class and Basic Tree Operations

- **Create** a tree node with a value.
- **Add a child** to a node.
- **Remove a child** from a node.
- **Preorder Traversal** to visit nodes in Root → Children order.

Python Implementation of Tree ADT :

```
classTreeNode:
def __init__(self, value):
    """Initialize a tree node with a value and an empty
list of children."""
    self.value = value
    self.children = []
defadd_child(self, child):
    """Adds a child node to the current node."""
    self.children.append(child)
defremove_child(self, child):
    """Removes a child node if it exists."""
    self.children = [c for c in self.children if c != child]
deftraverse_preorder(self):
    """Preorder traversal: Root → Children"""
    print(self.value, end=" ")
    for child in self.children:
        child.traverse_preorder()
# Example usage:
root = TreeNode("A")
node_b = TreeNode("B")
```

```
node_c = TreeNode("C")
node_d = TreeNode("D")
root.add_child(node_b)
root.add_child(node_c)
node_b.add_child(node_d)
print("Preorder Traversal:")
root.traverse_preorder() # Output: A B D C
```

4.2.3 Advantages of Trees

- **Efficient Operations** – Balanced trees (like BSTs) allow fast searching, insertion, and deletion in $O(\log n)$ time.
- **Hierarchical Structure** – Ideal for organizing data like file systems, databases, and XML/HTML documents.
- **Faster Data Access** – Provides quicker access than linked lists, especially in balanced trees.
- **Memory Efficient** – Uses dynamic memory allocation, making it space-efficient for large datasets.
- **Widely Used in Algorithms** – Essential for networking (routing), AI (decision trees), and data compression (Huffman coding).
- **Versatile Data Organization** – Different tree types (BST, Trie, AVL, Red-Black) cater to various applications like text searching and structured storage.

4.2.4 Disadvantages of Trees

- **Complex to Implement** – More challenging to build than arrays or linked lists.
- **Extra Memory Overhead** – Requires additional space for pointers and balancing operations.
- **Performance Issues in Unbalanced Trees** – If not balanced, operations can slow down to $O(n)$ time.
- **Difficult Maintenance** – Balancing trees (like AVL and Red-Black) requires extra computations and rotations.
- **Slower Traversal** – Accessing elements is slower compared to arrays due to non-contiguous memory storage.

4.2.5 Space and Time Complexity of Tree ADT

1. Space Complexity

- **Best Case:** $O(n)$
- **Worst Case:** $O(n)$

2. Time Complexity for Common Operations

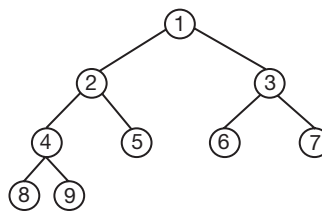
Operation	Best Case	Average Case	Worst Case
Insertion	$O(1)$	$O(\log n)$	$O(n)$
Deletion	$O(1)$	$O(\log n)$	$O(n)$
Searching	$O(1)$	$O(\log n)$	$O(n)$
Traversal	$O(n)$	$O(n)$	$O(n)$
Height Calculation	$O(1)$	$O(\log n)$	$O(n)$

- **Balanced trees (AVL, Red-Black, B-Trees)** maintain $O(\log n)$ time for search, insert, and delete.
- **Unbalanced trees (skewed BSTs)** can degrade to $O(n)$ time.
- **Traversal (Preorder, Inorder, Postorder, Level-order)** always takes $O(n)$ time as each node is visited once.

4.3 Binary Tree ADT (Abstract Data Type)

A **Binary Tree ADT** is a hierarchical data structure where each node has at most **two children**: a **left child** and a **right child**. It follows a parent-child relationship and is widely used in searching, sorting, and hierarchical data representation.

Example:



4.3.1 Key Properties of Binary Tree ADT

- **Each Node has at Most Two Children** – A node can have 0, 1, or 2 children.
- **Recursive Structure** – A binary tree is either empty or consists of a root node and two sub-trees (left and right).
- **Root Node** – The topmost node in the tree.
- **Height** – The longest path from the root to a leaf node.
- **Leaf Nodes** – Nodes without children.
- **Depth** – The number of edges from the root to a node.

4.3.2 Operations in Binary Tree ADT

- **Insertion** – Adds a new node at the appropriate position.
- **Deletion** – Removes a node while maintaining structure.
- **Search** – Finds a node based on its value.
- **Traversal** – Visits all nodes using:
 - Preorder (Root → Left → Right)
 - Inorder (Left → Root → Right)
 - Postorder (Left → Right → Root)
 - Level Order (BFS traversal)

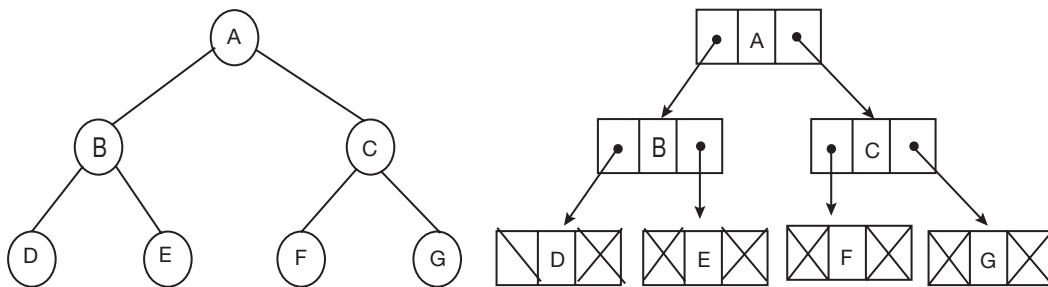
4.3.3 Types of Binary Trees

1. **Full Binary Tree** – Every node has 0 or 2 children.
2. **Complete Binary Tree** – All levels are full except the last, filled left to right.
3. **Perfect Binary Tree** – All leaf nodes are at the same level, and all internal nodes have two children.
4. **Balanced Binary Tree** – The height difference between left and right subtrees is minimal ($O(\log n)$ height).
5. **Binary Search Tree (BST)** – A sorted binary tree where the left subtree contains smaller values, and the right subtree contains larger values.

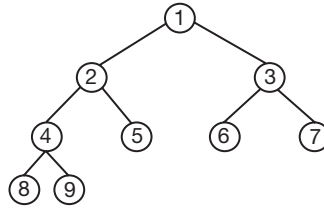
4.3.4 Representation of a Binary Tree

A binary tree can be represented in multiple ways:

1. Linked Representation :- Nodes and Pointers
2. Sequential / Linear Representation :- Array



For example : Complete Binary Tree



i. Linked Representation (Node-Based)

Each node contains:

- Data (value)
- Pointer to the left child
- Pointer to the right child

Example : consider above tree :

```

root = Node(1)
root.left = Node(2)
root.right = Node(3)
root.left.left = Node(4)
root.left.right = Node(5)
root.right.left = Node(6)
root.right.right = Node(7)
root.left.left.left = Node(8)
root.left.left.right = Node(9)
  
```

ii. Array Representation

- If a node is at index i ,
 - Left child is at $2*i + 1$
 - Right child is at $2*i + 2$
- Commonly used for **complete binary trees**.

Example : Consider above tree :

Array [1,2,3,4,5,6,7,8,9]

4.3.5 Python Implementation of a Binary Tree ADT with Basic Operations Like Insertion, Search, and Traversal

Consider node from 1 to 5

- **Insertion** : (Level-order for a full tree)
- **Search** : (Breadth-first search)
- **Inorder Traversal**: (Left $\rightarrow$ Root $\rightarrow$ Right)

Example : 1

```

class Node:
def __init__(self, value):
  
```

```

        """Initialize a binary tree node with a value and left/
right children as None."""
self.value = value
self.left = None
self.right = None
class BinaryTree:
def __init__(self, root_value):
    """Initialize the binary tree with a root node."""
self.root = Node(root_value)
def insert(self, value):
    """Insert a node into the binary tree (Level Order
Insertion)."""
queue = [self.root]
while queue:
current = queue.pop(0)
if not current.left:
current.left = Node(value)
return
else:
queue.append(current.left)
if not current.right:
current.right = Node(value)
return
else:
queue.append(current.right)
def search(self, value):
    """Search for a node with the given value (Level
Order)."""
queue = [self.root]
while queue:
current = queue.pop(0)
if current.value == value:
return True
if current.left:
queue.append(current.left)
if current.right:
queue.append(current.right)
return False
def inorder_traversal(self, node):
    """Inorder traversal (Left → Root → Right)."""
if node:
self.inorder_traversal(node.left)

```

```

print(node.value, end=" ")
self.inorder_traversal(node.right)
# Example Usage
bt = BinaryTree(1) # Root node with value 1
bt.insert(2)
bt.insert(3)
bt.insert(4)
bt.insert(5)
print("Inorder Traversal:")
bt.inorder_traversal(bt.root) # Output: 4 2 5 1 3
print("\nSearch 3:", bt.search(3)) # Output: True
print("Search 6:", bt.search(6)) # Output: False

```

4.4 Tree Traversals

Tree traversal means visiting each node in a tree in a specific order.

There are two main types:

1. **Depth-First Traversal (DFS)** – Goes deep into the tree before backtracking.
 - **Preorder (Root → Left → Right)**
 - **Inorder (Left → Root → Right)** → Used for sorting in BST.
 - **Postorder (Left → Right → Root)** → Used in deleting trees.
2. **Breadth-First Traversal (BFS)** – Visits level by level.
 - **Level Order (Top to Bottom, Left to Right)**
 - Preorder (Root → Left → Right) → Used in expression trees and tree cloning.
 - Inorder (Left → Root → Right) → Retrieves elements in sorted order (for BST).
 - Postorder (Left → Right → Root) → Useful for deleting nodes.
 - Level Order (BFS Traversal) → Used in shortest path algorithms.

4.4.1 Python Implementation of Tree Traversal

```

class Node:
def __init__(self, value):
    """Initialize a tree node with a value and left/right
    children as None."""
    self.value = value
    self.left = None
    self.right = None
class BinaryTree:
def __init__(self, root_value):
    """Initialize the binary tree with a root node."""

```

```

self.root = Node(root_value)
def preorder(self, node):
    """Preorder Traversal (Root → Left → Right)"""
    if node:
        print(node.value, end=" ")
        self.preorder(node.left)
        self.preorder(node.right)
def inorder(self, node):
    """Inorder Traversal (Left → Root → Right)"""
    if node:
        self.inorder(node.left)
        print(node.value, end=" ")
        self.inorder(node.right)
def postorder(self, node):
    """Postorder Traversal (Left → Right → Root)"""
    if node:
        self.postorder(node.left)
        self.postorder(node.right)
        print(node.value, end=" ")
def level_order(self):
    """Level Order Traversal (BFS - Left to Right, Level
    by Level)"""
    queue = [self.root]
    while queue:
        current = queue.pop(0)
        print(current.value, end=" ")
        if current.left:
            queue.append(current.left)
        if current.right:
            queue.append(current.right)
# Example
bt = BinaryTree(1)
bt.root.left = Node(2)
bt.root.right = Node(3)
bt.root.left.left = Node(4)
bt.root.left.right = Node(5)
print("Preorder Traversal:")
bt.preorder(bt.root)  # Output: 1 2 4 5 3
print("\nInorder Traversal:")
bt.inorder(bt.root)  # Output: 4 2 5 1 3
print("\nPostorder Traversal:")
bt.postorder(bt.root)  # Output: 4 5 2 3 1

```

```
print("\nLevel Order Traversal:")
bt.level_order() # Output: 1 2 3 4 5
```

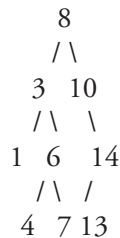
4.5 Binary Search Tree (BST)

A **Binary Search Tree (BST)** is a special type of binary tree where each node follows these rules:

1. **Left Subtree Rule** – The left child contains values smaller than the parent node.
2. **Right Subtree Rule** – The right child contains values greater than the parent node.
3. **No Duplicates** – A BST does not allow duplicate values.

4.5.1 Key Properties of BST

- The inorder traversal of a BST gives elements in sorted order.
- Efficient search, insertion, and deletion operations with $O(\log n)$ time in a balanced BST.
- If the tree becomes skewed (like a linked list), operations degrade to $O(n)$ time.



- Left subtree has smaller values (1, 3, 4, 6, 7).
- Right subtree has larger values (10, 13, 14).

4.5.2 Python Implementation of BST with Insert, Search, and Delete

```
class Node:
    def __init__(self, value):
        """Initialize a node with a value and left/right children as None."""
        self.value = value
        self.left = None
        self.right = None
    class BST:
        def __init__(self):
            """Initialize an empty Binary Search Tree."""
            self.root = None
        def insert(self, value):
```

```

        """Insert a new value into the BST."""
self.root = self._insert_recursive(self.root, value)
def _insert_recursive(self, node, value):
    """Helper function to insert recursively."""
    if node is None:
        return Node(value)
    if value < node.value:
        node.left = self._insert_recursive(node.left, value)
    else:
        node.right = self._insert_recursive(node.right, value)
    return node
def search(self, value):
    """Search for a value in the BST."""
    return self._search_recursive(self.root, value)
def _search_recursive(self, node, value):
    """Helper function to search recursively."""
    if node is None or node.value == value:
        return node is not None
    if value < node.value:
        return self._search_recursive(node.left, value)
    return self._search_recursive(node.right, value)
def delete(self, value):
    """Delete a node from the BST."""
    self.root = self._delete_recursive(self.root, value)
def _delete_recursive(self, node, value):
    """Helper function to delete a node."""
    if node is None:
        return node
    if value < node.value:
        node.left = self._delete_recursive(node.left, value)
    elif value > node.value:
        node.right = self._delete_recursive(node.right, value)
    else:
        # Case 1: Node with no children (leaf node)
        if node.left is None and node.right is None:
            return None
        # Case 2: Node with only one child
        if node.left is None:
            return node.right
        if node.right is None:
            return node.left

```

```

        # Case 3: Node with two children (Find the inorder
        successor)
        min_larger_node = self._find_min(node.right)
        node.value = min_larger_node.value
        node.right = self._delete_recursive(node.right, min_larger_
        node.value)
        return node
    def _find_min(self, node):
        """Find the minimum value node in a subtree (leftmost
        node)."""
        while node.left:
            node = node.left
        return node
    def inorder_traversal(self, node):
        """Inorder traversal (Left → Root → Right) to get
        sorted order."""
        if node:
            self.inorder_traversal(node.left)
            print(node.value, end=" ")
            self.inorder_traversal(node.right)
# Example:
bst = BST()
values = [8, 3, 10, 1, 6, 14, 4, 7, 13]
for v in values:
    bst.insert(v)
print("Inorder Traversal (Sorted Order):")
bst.inorder_traversal(bst.root) # Output: 1 3 4 6 7 8 10 13 14
print("\nDeleting 6...")
bst.delete(6)
print("Inorder Traversal after deletion:")
bst.inorder_traversal(bst.root) # Output: 1 3 4 7 8 10 13 14
print("\nSearch 6:", bst.search(6)) # Output: False
print("Search 10:", bst.search(10)) # Output: True

```

4.5.3 Advantages and Disadvantages of Binary Search Tree (BST)

Advantages of BST

- **Quick Search** – Searching takes $O(\log n)$ time in a well-balanced BST.
- **Efficient Insertions & Deletions** – These operations also run in $O(\log n)$ in balanced trees.
- **Sorted Data** – In-order traversal naturally arranges elements in ascending order.

- **Scalable & Flexible** – BSTs dynamically expand without predefined size limits, unlike arrays.
- **Optimized for Range Queries** – Searching within a range runs in $O(\log n + k)$ time, where k is the number of results.
- **Versatile Applications** – Used in databases, priority queues, and predictive search features.

Disadvantages of BST

- **Performance Drops in Unbalanced Trees** – A skewed BST degrades to $O(n)$ time complexity.
- **Extra Space Requirement** – Each node needs additional memory for left and right pointers.
- **Complex to Maintain** – Keeping a BST balanced (e.g., AVL or Red-Black trees) requires extra logic.
- **Not Ideal for Small Data Sets** – Simpler structures like arrays or hash tables may be faster.
- **Frequent Modifications Can Cause Imbalance** – Constant insertions and deletions can require rebalancing, adding overhead.

4.6 AVL Tree

An **AVL Tree** (Adelson-Velsky and Landis Tree) is a self-balancing Binary Search Tree (BST) where the height difference between the left and right subtrees of any node is at most **1**. Such as Data structure AVL tree guarantees that the heights of the subtrees differ by not more than one.

In an AVL data structure, every node has a balance factor. The balance factor is the height difference between the left subtree and the right subtree. The balance factor is -1, 0, or 1. When the balance factor becomes out of this range, the tree performs a rotation to self-correct. This keeps the tree short and broad, which makes it quicker to search, insert, and delete items.

4.6.1 Properties of AVL Tree

The main properties of AVL tree in data structure, defined as:

Balanced Tree

An AVL tree remains balanced under all circumstances. This implies that the left and right subtrees of any node have a difference in heights of maximum one.

Height-Balanced Property

The height difference (or the balance factor) of the left and right subtrees of any node is -1, 0, or 1.

Self-Balancing

An AVL tree internally balances itself by performing rotations automatically after every deletion and insertion.

Binary Search Tree Properties

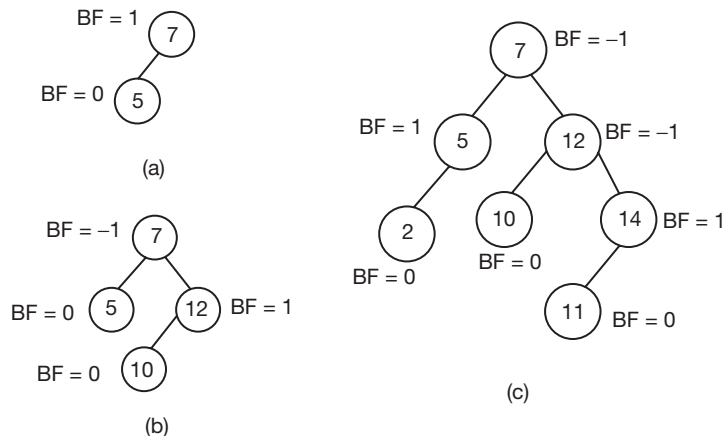
An AVL tree is a binary search tree. This implies that for any node in an AVL tree, the nodes to the left of the node contain values smaller than the node's value, and the nodes to the right of the node contain values larger than the node's value.

Rotations

In order to preserve balance, an AVL tree employs four kinds of rotations: left rotation, right rotation, left-right rotation, and right-left rotation.

4.6.2 Key Properties

- The height of an empty tree is -1.
- The balance factor of a node is calculated as:
Balance Factor = Height of Left Subtree – Height of Right Subtree
- A valid AVL tree has balance factors of -1, 0, or +1 for all nodes.
- If the balance factor exceeds this range (less than -1 or greater than 1), the tree is rebalanced using single or double rotations.



4.6.3 Types of Rotations

When an AVL tree becomes unbalanced, it must be rebalanced using rotations.

There are four types of rotations used to restore balance:

1. Right Rotation (LL Rotation)
2. Left Rotation (RR Rotation)
3. Left-Right Rotation (LR Rotation)
4. Right-Left Rotation (RL Rotation)

1. Right Rotation (LL Rotation)

Occurs when a node is inserted into the **left subtree of the left child**, causing the tree to become unbalanced.

Example Before Rotation

```

  30
 /
20
/
10

```

Balance Factors:

- 30 → +2 (Unbalanced)
- 20 → +1
- 10 → 0

Right Rotation

```

  20
 / \
10 30

```

Example : Python Implementation

```

def rightRotate(z):
    y = z.left
    T3 = y.right
    # Perform rotation
    y.right = z
    z.left = T3
    return y # New root after rotation

```

2. Left Rotation (RR Rotation)

Occurs when a node is inserted into the **right subtree of the right child**, causing the tree to become unbalanced.

```

  10
   \
   20
    \
    30

```

Balance Factors:

- 10 → -2 (Unbalanced)
- 20 → -1
- 30 → 0

Left Rotation

```

  20
 / \
10 30

```

3. Left-Right Rotation (LR Rotation)

Occurs when a node is inserted into the **right subtree of the left child**, causing the tree to become unbalanced.

Steps for AVL Tree Insertion

1. **Insert the node** following BST rules.
2. **Calculate the balance factor** for each node.
3. **If the balance factor is out of range** (-1, 0, 1), perform the required **rotation**:
 - **LL Rotation** (Right Rotation) → Left-heavy (Left-Left case)
 - **RR Rotation** (Left Rotation) → Right-heavy (Right-Right case)
 - **LR Rotation** (Left-Right case) → Left Rotation followed by Right Rotation
 - **RL Rotation** (Right-Left case) → Right Rotation followed by Left Rotation

Insertion Process

1. Insert **10** → Balanced
2. Insert **20** → Balanced
3. Insert **30** → **Unbalanced (RR Case)** → Perform **Left Rotation**
4. Insert **40** → Balanced
5. Insert **50** → **Unbalanced (RR Case)** → Perform **Left Rotation**
6. Insert **25** → **Unbalanced (RL Case)** → Perform **Right Rotation**, then **Left Rotation**

```

      30
     / \
    20  40
   / \  \
  10 25 50

```

```

# AVL Tree Node Class
class Node:
    def __init__(self, value):
        self.value = value
        self.left = None
        self.right = None
        self.height = 1 # Height of a new node is 1
# AVL Tree Class with Insertion
class AVLTree:
    def getHeight(self, node):
        return node.height if node else 0
    def getBalanceFactor(self, node):
        if not node:

```

```

return 0
return self.getHeight(node.left) - self.getHeight(node.right)
def rightRotate(self, z):
    y = z.left
    T3 = y.right
    # Perform rotation
    y.right = z
    z.left = T3
    # Update heights
    z.height = max(self.getHeight(z.left), self.getHeight(z.
right)) + 1
    y.height = max(self.getHeight(y.left), self.getHeight(y.
right)) + 1
    return y # New root after rotation
def leftRotate(self, z):
    y = z.right
    T2 = y.left
    # Perform rotation
    y.left = z
    z.right = T2
    # Update heights
    z.height = max(self.getHeight(z.left), self.getHeight(z.
right)) + 1
    y.height = max(self.getHeight(y.left), self.getHeight(y.
right)) + 1
    return y # New root after rotation
def insert(self, root, key):
    # Step 1: Perform BST Insertion
    if not root:
        return Node(key)
    elif key < root.value:
        root.left = self.insert(root.left, key)
    else:
        root.right = self.insert(root.right, key)
    # Step 2: Update height
    root.height = max(self.getHeight(root.left), self.
getHeight(root.right)) + 1
    # Step 3: Get Balance Factor
    balance = self.getBalanceFactor(root)
    # Step 4: Perform Rotations if necessary
    # Case 1: Left-Left (LL Rotation)
    if balance > 1 and key < root.left.value:

```

```

returnself.rightRotate(root)
    # Case 2: Right-Right (RR Rotation)
if balance < -1 and key >root.right.value:
returnself.leftRotate(root)
    # Case 3: Left-Right (LR Rotation)
if balance > 1 and key >root.left.value:
root.left = self.leftRotate(root.left)
returnself.rightRotate(root)
    # Case 4: Right-Left (RL Rotation)
if balance < -1 and key <root.right.value:
root.right = self.rightRotate(root.right)
returnself.leftRotate(root)
return root # Return unchanged root if no rotation is needed
defpreOrder(self, root):
if root:
print(root.value, end=" ")
self.preOrder(root.left)
self.preOrder(root.right)
# Driver Code
avl = AVLTree()
root = None
# Inserting nodes
nodes = [10, 20, 30, 40, 50, 25]
for node in nodes:
root = avl.insert(root, node)
# Printing Preorder Traversal (should be balanced)
print("Preorder Traversal of AVL Tree:")
avl.preOrder(root)

```

Output :30 20 10 25 40 50

Example 1: Before Rotation

```

    30
   /
  10
   \
   20

```

Left Rotation on 10

```

    30
   /
  20
 /
10

```

Right Rotation on 30

```

  20
 / \
10  30

```

4. Right-Left Rotation (RL Rotation)

Occurs when a node is inserted into the **left subtree of the right child**, causing imbalance.

Example Before Rotation

```

  10
   \
   30
  /
 20

```

Right Rotation on 30

```

  10
   \
   20
    \
    30

```

Left Rotation on 10

```

  20
 / \
10  30

```

Time Complexity:

- **Search, Insert, Delete** → $O(\log n)$ (Better than unbalanced BST)

4.6.4 Applications

- Database Indexing (MySQL, Oracle)
- File Systems (Linux Ext3/Ext4)
- Operating Systems (Memory Management)
- Network Routing Algorithms (OSPF)
- Compiler Design & Expression Trees
- Geospatial & GIS Systems (Google Maps)
- Autocomplete, AI, NLP Applications

4.7 Heap

A **Heap** is a complete binary tree that follows the heap property, meaning that the value of each node is always greater than or equal to (Max-Heap) or less than or equal to (Min-Heap) the values of its children.

4.7.1 Types of Heaps

1. Max-Heap:

- The root node has the largest value in the tree.
- Every parent node is greater than or equal to its children.
- This property applies recursively to all subtrees.

2. Min-Heap:

- The root node contains the smallest value in the tree.
- Every parent node is less than or equal to its children.
- This rule holds for all subtrees.

4.7.2 Operations of Heap

1. Insertion ($O(\log n)$)

Adding a new element while maintaining the heap property.

- Insert the element at the **leftmost available position**.
- Compare it with its **parent**.
- **Swap if necessary** to restore heap property (**Heapify-Up**).

2. Deletion ($O(\log n)$)

Removing the root element (highest in Max-Heap, lowest in Min-Heap).

- Replace the root with the **last element**.
- **Heapify-Down** (swap with the larger/smaller child) to maintain heap property.

3. Heapify ($O(\log n)$)

Rearranges the heap **to maintain the heap property**.

Two types:

- **Heapify-Up (Percolate-Up)** → Used in **Insertion**.
- **Heapify-Down (Percolate-Down)** → Used in **Deletion**.

4. Peek ($O(1)$)

Returns the **top element** (root) without removing it.

- **Max-Heap** → Returns the largest element.
- **Min-Heap** → Returns the smallest element.

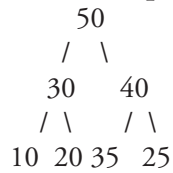
5. Heap Sort ($O(n \log n)$)

Heap Sort uses a **Heap (Min or Max)** to sort an array:

1. **Build a Max-Heap** from an array.
2. Swap **root with the last element** and reduce heap size.
3. **Heapify** to restore heap property.
4. Repeat until all elements are sorted.

Example of Max-Heap

A **Max-Heap** ensures that the highest value is always at the root.

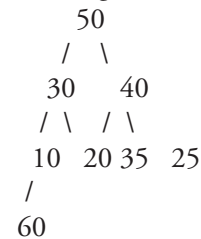


Heap Properties:

- 50 (root) is the largest.
- Each parent node is **greater than or equal to** its children.

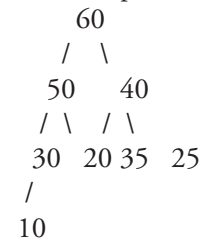
Max-Heap Insert Example

Inserting 60:



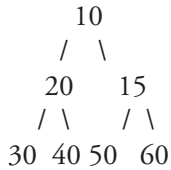
- 60 is inserted at the leftmost empty position.
- Swap 60 with 10 (parent) → **Heap Property Violated**
- Swap 60 with 50 → **Heap Property Restored**

Final Heap:



Example of Min-Heap

A **Min-Heap** ensures that the **smallest** value is always at the root.

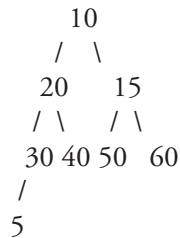


Heap Properties:

- 10 (root) is the smallest.
- Each parent node is **less than or equal to** its children.

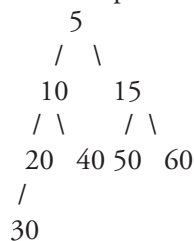
Min-Heap Insert Example

Inserting 5:



- 5 is inserted at the leftmost empty position.
- Swap 5 with 30 → **Heap Property Violated**
- Swap 5 with 10 → **Heap Property Restored**

Final Heap:



4.7.3 Advantages and Disadvantages of Heap Data Structure

Advantages:

1. **Fast Operations:** Insertion and deletion take **$O(\log n)$** time, making heaps efficient for handling large datasets. Finding the minimum or maximum takes only $O(1)$ time.
2. **Space Efficient:** Since a heap is a complete binary tree, it can be stored in an array without wasting space.

3. **Dynamic:** Heaps can grow and shrink dynamically, making them ideal for applications that require frequent updates, such as real-time scheduling.
4. **Priority-Based Processing:** Heaps allow efficient priority management, making them useful for applications like task scheduling, stock market analysis, and load balancing.
5. **In-Place Sorting:** Heaps are used in Heap Sort, which sorts elements without needing extra memory.

Disadvantages:

- **Limited Flexibility:** Heaps enforce a strict order, which may not be suitable for applications requiring flexible structures.
- **Inefficient Searching:** Searching for a specific element requires $O(n)$ time, as heaps are not optimized for searching.
- **Not Stable:** Equal elements may not retain their original order after heap operations.
- **Higher Complexity:** Although insertion and deletion are efficient, Heap Sort has a worst-case complexity of $O(n \log n)$, which might not be optimal for some tasks.

4.7.4 Applications

- **Priority Queues** (Dijkstra's algorithm, job scheduling)
- **Heap Sort** (Efficient $O(n \log n)$ sorting)
- **Graph Algorithms** (Prim's, Dijkstra's shortest path)
- **Memory Management** (Dynamic memory allocation)

4.8 Multi-Way Search Trees

Introduction

An **M-Way Search Tree** is a tree data structure where each node can **have up to M children and M-1 keys**. It is an extension of the **Binary Search Tree (BST)** but allows for **multiple keys** in a single node, making searches and insertions more efficient for large datasets.

Multi-way search trees are an extension of binary search trees that allow each node to have more than two children. Unlike binary search trees, which are limited to two children per node, multi-way search trees can have multiple children, making them efficient for searching and sorting large datasets.

4.8.1 Properties of an M-Way Search Tree

1. Each node has at most M children and M-1 keys.
2. Keys within each node are stored in sorted order.
3. For any key at position i in a node:
 - All keys in the i-th child (left subtree) are smaller than the key at position i.

- All keys in the $(i+1)$ -th child (right subtree) are greater than the key at position i .
4. The tree is not necessarily balanced, but a well-constructed M-way tree performs efficient searches.

4.8.2 3-Way Search Tree (Ternary Search Tree)

A **3-Way Search Tree** (order $M=3$) can have up to **3 children** and **2 keys** per node.

Example: Inserting Elements

Let's insert the numbers: **20, 40, 10, 30, 50, 60** into a **3-way search tree**.

Step 1: Insert 20 and 40

[20 | 40]

Step 2: Insert 10

Since $10 < 20$, it goes into the left child.

Step 3: Insert 30

Since **30 is between 20 and 40**, it goes in the middle child:

Step 4: Insert 50

Since $50 > 40$, it goes to the right child:

Step 5: Insert 60

Since $60 > 50$, it goes into the right child:

[20 | 40]

/ | \

[10] [30] [50 | 60]

Now, the tree remains balanced while allowing **faster searching** than a standard BST.

4.8.3 Operations in an M-Way Search Tree

1. Insertion ($O(\log M N)$)

- Insert in the appropriate node while keeping keys sorted.
- If the node overflows (more than $M-1$ keys), split and push a key up.

2. Deletion ($O(\log M N)$)

- Remove the key and adjust the structure.
- If a node becomes underfilled, borrow from siblings or merge nodes.

3. Searching ($O(\log M N)$)

- Start at the root and compare with keys in the node.
- Move to the correct child until the key is found or a leaf is reached.

4.8.4 Types of Multi-Way Search Trees

Several types of multi-way search trees exist, but two of the most common are:

- i. **2-3 Tree**
- ii. **B-Tree**

i. 2-3 Tree

A 2-3 tree is a multi-way search tree where each node can have two or three children.

- The smallest key is always placed in the leftmost child, while the largest key is in the rightmost child.
- 2-3 trees are commonly used in compilers and interpreters for efficient data management.

Example:

A node in a **2-3 tree** contains one or two keys, organizing data in a way that maintains balance and ensures efficient searches.

ii. Structure of a B-Tree

A **B-tree** is a self-balancing search tree used to manage large amounts of data efficiently. Unlike a binary search tree (BST), where each node has at most two children, a B-tree node can have multiple keys and multiple children.

4.8.5 Key Properties of a B-Tree

1. **Order (M):** A B-tree of order M can have at most M children and M-1 keys in each node.
2. **Sorted Keys:** The keys in each node are arranged in ascending order.
3. **Balanced Structure:** The tree remains balanced, meaning all leaf nodes are at the same depth.
4. **Efficient Search:** The search process is similar to a binary search, reducing time complexity to $O(\log N)$.

i. Inserting a Key into a B-Tree

When adding a key to a B-tree, the primary objective is to keep the tree balanced while preserving its structural properties.

Steps for Insertion:

1. **Locate the correct position** for the new key by traversing the tree.
2. **Insert the key** into the appropriate node while maintaining sorted order.
3. **Handle node overflow** - if the node exceeds the allowed number of keys, split it into two and move the middle key up to the parent.
4. **Continue splitting if needed** - if the parent also exceeds its key limit, propagate the split upwards until the tree remains balanced.

ii. Deletion in a B-Tree

Removing a key from a B-tree requires restructuring to maintain balance.

Steps for Deletion:

1. **Find the key** in the tree.
2. If the key is in a **leaf node**, remove it directly.

3. If the key is in an **internal node**, replace it with its **predecessor** or **successor** key from a child node.
4. If a node has too few keys after deletion, **borrow from a sibling** or **merge nodes** to maintain balance.

4.8.6 Applications of B-Trees

B-Trees are widely used in scenarios where efficient searching, insertion, and deletion of large amounts of data are required. Their balanced structure ensures optimal performance even with high-volume data.

Some common applications include:

- B-Trees are used in database indexing (e.g., MySQL, PostgreSQL) to store and retrieve data efficiently.
- B-Trees help in storing and retrieving indexed web pages quickly.
- Used in virtual memory management to maintain balanced page tables.
- B-Trees are used in network databases for routing table management and data retrieval.
- Distributed databases and NoSQL databases (e.g., MongoDB, Cassandra) use B-Trees for efficient key-value storage.
- Helps in maintaining structured symbol tables for quick access to variables, functions, and objects.
- Used in cloud storage and enterprise systems for handling vast amounts of hierarchical data efficiently.

Questions: Part-A

1. Define a tree in data structures.
2. What does the degree of a node in a tree represent?
3. Explain the concept of a root node in a tree.
4. How do you determine the height of a tree?
5. What is a binary tree?
6. What defines a full binary tree?
7. What conditions make a binary tree complete?
8. How many nodes are present in a full binary tree of height h ?
9. What are the different types of binary tree traversals?
10. Describe the traversal sequence in a level-order traversal.
11. Explain the inorder traversal of a binary tree.
12. Which tree traversal method is used in Depth First Search (DFS)?
13. How does a Binary Search Tree (BST) organize its elements?
14. What is the best and worst-case time complexity for searching in a BST?
15. What is the key structural property of a BST?

16. Why does a skewed BST have a higher search time complexity?
17. What is an AVL tree, and why is it used?
18. What types of rotations are used to balance an AVL tree?
19. What is the role of the balance factor in an AVL tree?
20. How does a multi-way search tree work?
21. What is a B-tree, and where is it commonly used?
22. How does a B-tree differ from a Binary Search Tree (BST), and why is it preferred in databases?

Questions: Part-B

1. What is a tree data structure? Define its key terminologies such as root, parent, child, sibling, leaf, degree, depth, and height with examples.
2. What are the key differences between general trees and binary trees? List the advantages of using trees in data structures.
3. Define a binary tree and explain its different types with examples.
4. Describe different ways of storing binary trees in memory. How do array and linked list representations compare?
5. Write an algorithm to insert and delete a node in a binary tree. Discuss its time complexity.
6. What are the different types of tree traversal methods? Explain Inorder, Preorder, Postorder, and Level Order traversals with examples.
7. How can a binary tree be traversed using both recursive and iterative approaches? Provide algorithms for Inorder, Preorder, and Postorder traversals.
8. Given a binary tree, how can its preorder and inorder traversals be derived? How can the original tree be reconstructed from these traversals?
9. Compare Depth-First Search (DFS) and Breadth-First Search (BFS) in terms of tree traversal. What are their real-world applications?
10. What is a Binary Search Tree (BST)? Describe its properties with an example.
11. How can nodes be inserted, deleted, and searched in a BST? Analyze the time complexity for best, average, and worst cases.
12. Given the sequence {50, 30, 70, 20, 40, 60, 80}, construct a Binary Search Tree and illustrate the step-by-step process.
13. What are the disadvantages of an unbalanced BST? How does an AVL tree address these issues?
14. Define an AVL tree and explain why it is called a self-balancing tree.
15. What are the different types of rotations used in AVL trees (LL, RR, LR, RL)? Explain with examples.
16. Write an algorithm for inserting and deleting nodes in an AVL tree. How do rotations help in maintaining balance?
17. What is a heap? How do Min Heap and Max Heap differ? Provide examples.

18. Describe the operations performed on a heap, including insertion, deletion, and heapify, with algorithms.
19. What is a Multi-Way Search Tree? How does it differ from a Binary Search Tree? Provide examples.
20. Explain the structure of a B-tree. How are insertion and deletion performed in a B-tree? Provide examples.
21. Compare B-trees and AVL trees in terms of height balancing, searching efficiency, and insertion performance.

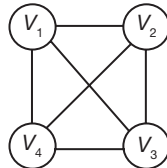
Graph Structures

Graph ADT – Representations of Graph – Graph Traversals – DAG – Topological ordering
– Shortest Paths – Minimum Spanning Tree – Disjoint Sets

Graph ADT

A **graph** in mathematics and computer science is a collection of **nodes (or vertices)** connected by **edges**. It is used to represent relationships between objects.

Vertices are referred to as nodes and the arc between the nodes are referred to as Edges.



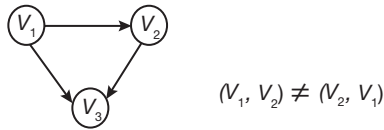
Graph Representation

Here V_1, V_2, V_3, V_4 are the vertices and $(V_1, V_2), (V_2, V_3), (V_3, V_4), (V_4, V_1), (V_2, V_4), (V_1, V_3)$ are edges.

Types of Graphs:

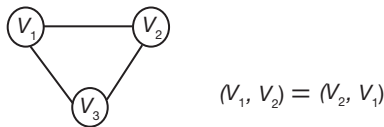
i) Directed Graph (Digraph)

A **Directed Graph (Digraph)** is a type of graph where edges have a direction. Each edge is represented as an ordered pair (u, v) , indicating a connection from **node u to node v**.



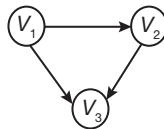
ii) Undirected Graph:

An undirected graph is a graph, which consists of undirected edges. If (v, w) is an undirected edge then $(v, w) = (w, v)$

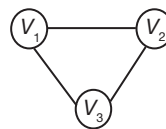


iii) Weighted Graph:

- A **Weighted Graph** is a type of graph where each edge has an associated numerical value called a **weight or cost**.
- These weights often represent distances, costs, or capacities in real-world applications.
- It can be directed or undirected graph.



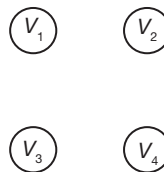
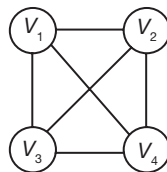
Directed Weighted Graph



Undirected Weighted Graph

iv) Complete Graph:

- A **Complete Graph** is a type of graph where every pair of distinct vertices is connected by a unique edge.
- Every node is connected to every other node (fully connected).
- Number of edges in an undirected complete graph with n vertices: $E = n(n-1)/2$
- In a directed complete graph, the number of edges is: $E = n(n-1)$

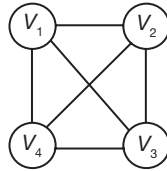


Complete Graph

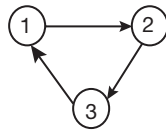
v) Path:

A **path** in a graph is a sequence of vertices connected by edges, where each edge connects two consecutive vertices in the sequence.

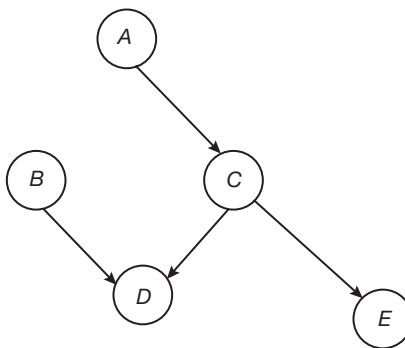
Path from V_1 to V_3 is i) $v_1 \rightarrow v_3$ or $v_1 \rightarrow v_2 \rightarrow v_3$

**Path****vi) Cycle:**

A **cycle** in a graph is a path that starts and ends at the same vertex **without repeating any other vertices**.

**Cycle Graph****vii) Acyclic Graph:**

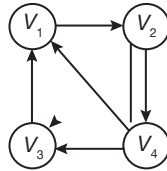
A directed graph which has no cycles is referred to as acyclic graph. It is abbreviated as DAG.

**DAG****viii) Degree:**

- The **degree** of a vertex in a graph is the number of edges connected to it.

- The indegree of the vertex V , is the number of edges entering in to the vertex V .
- The outdegree of the vertex V is the number of edges exiting from that vertex V .

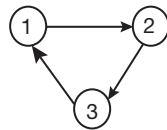
$$\text{Indegree}(V_1) = 2 \quad \text{outdegree}(V_1) = 1$$



Degree

ix) Loop:

A **loop** (or **self-loop**) in a graph is an edge that connects a vertex to itself.



Loop

Representation of Graph

Graph can be represented by Adjacency Matrix and Adjacency list. One simple way to represent a graph is Adjacency Matrix.

The adjacency Matrix A for a graph $G = (V, E)$ with n vertices is an $n \times n$ matrix, such that

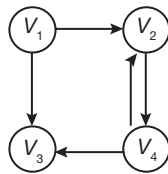
$A_{ij} = 1$, If there is an edge V_i to V_j

$A_{ij} = 0$, if there is no edge.

i) Adjacency Matrix for Directed Graph:

An adjacency matrix is a 2D array representation of a directed graph where:

- Rows represent starting nodes.
- Columns represent ending nodes.
- A value of 1 (or weight) at $\text{matrix}[i][j]$ means there is an edge from node i to node j .
- A value of 0 means no edge exists.

**Directed Graph**

	V_1	V_2	V_3	V_4
V_1	0	1	1	0
V_2	0	0	0	1
V_3	0	1	0	0
V_4	0	0	1	0

Matrix Representation

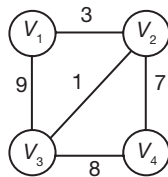
$V_{1,2}=1$ Since there is an edge V_1 to V_2 Similarly $V_{1,1}$ & $V_{1,4}=0$, there is no edge.

$V_{1,3}=1$, there is an edge V_1 to V_3

ii) Adjacency Matrix For Undirected Graph:

An adjacency matrix for an undirected graph is a 2D array representation, where:

- Rows and columns represent nodes.
- A value of 1 (or weight) at matrix[i][j] means there is an edge between node i and node j

**Undirected Graph**

	V_1	V_2	V_3	V_4
V_1	0	3	9	
V_2		0		7
V_3		1	0	
V_4		1	8	0

Matrix Representation

Advantage

- Simple to implement.

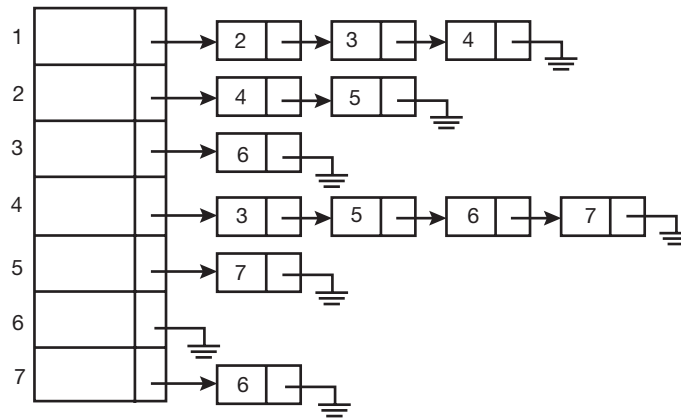
Disadvantage

- Takes $O(n^2)$ space to represent the graph
- It takes $O(n^2)$ time to solve the most of the problems

iii) Adjacency List Representation:

An adjacency list is a more space-efficient way to represent a graph compared to an adjacency matrix. It uses a dictionary (or list of lists) where:

- Each node is a key.
- The value is a list of adjacent nodes (or tuples for weighted graphs).



Adjacency List Representation

Graph Traversal

Graph traversal is the process of **visiting all nodes (vertices) in a graph** systematically. There are two types of graph traversal namely,

- Depth first traversal
- Breadth first traversal

i) Depth first traversal:

Depth-First Search (DFS) is a **graph traversal algorithm** that explores nodes **deeply** before backtracking. It starts at a source node, **visits an adjacent unvisited node**, and continues recursively (or using a stack) until all nodes are explored.

It uses a **stack (LIFO)** to ensure that nodes are visited in layers.

To implement the Depth first Search performs the following Steps:

Step 1: Start at a source node.

Step 2: Using the adjacency matrix of the graph, Visit the first unvisited neighbor.

Step 3: Keep exploring deeply until no unvisited neighbors remain.

Step 4: Backtrack and repeat for the next unvisited neighbor.

Step 5: Continue until all nodes are visited.

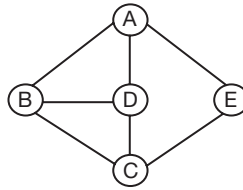
Routine for Depth First Search:

```
def bfs(graph, start):
    visited = set()
```

```

queue = deque([start])
while queue:
    node = queue.popleft()
    if node not in visited:
        print(node, end=" ")
        visited.add(node)
        queue.extend(graph.get(node, []))

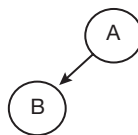
```

Example:**Undirected Graph****Adjacency Matrix:**

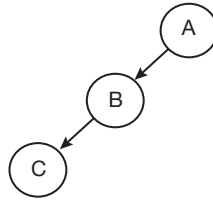
	A	B	C	D	E
A	0	1	0	1	1
B	1	0	1	1	0
C	0	1	0	1	1
D	1	1	1	0	0
E	1	0	1	0	0

Implementation

We start at vertex "A". Then Mark A as visited and call DFS(B) recursively, Dfs(B) Marks B as visited and calls Dfs(c) recursively.

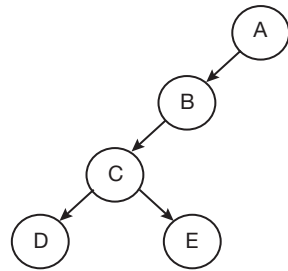
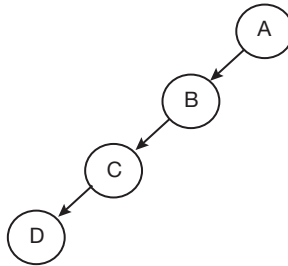


Dfs (c) marks C as visited and calls Dfs (D) recursively. No recursive calls are made to Dfs (B) since B is already visited.



Dfs(D) marks D as visited. Dfs(D) sees A, B, C as marked. So no recursive call is made there, and Dfs(D) returns back to Dfs(C).

Dfs(C) calls Dfs(E), where E is unseen adjacent vertex to C.



Depth First spanning Tree

Since all the vertices starting from "A" are visited, the above graph is said to be connected. If the graph is not connected, then processing all nodes requires reversal calls to Dfs, and each generates a tree. This entire collection is a depth first spanning forest.

Applications of Depth First Search

1. To check whether the undirected graph is connected or not.
2. To check whether the connected undirected graph is Biconnected or not.
3. To check the Acyclicity of the directed graph.

ii) Breadth first traversal:

Breadth-First Search (BFS) is a **graph traversal algorithm** that explores all **neighbouring nodes first** before moving to the next level of neighbours.

It uses a **queue (FIFO)** to ensure that nodes are visited in layers.

Steps to implement breadth first search

Step 1: Choose any node in the graph, designate it as the search node and mark it as visited.

Step 2: Using the adjacency matrix of the graph, find all the unvisited adjacent nodes to the search node and enqueue them into the queue Q.

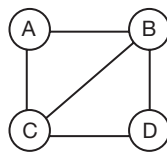
Step 3: Then the node is dequeued from the queue. Mark that node as visited and designate it as the new search node.

Step 4: Repeat step 2 and 3 using the new search node.

Step 5: This process continues until the queue Q which keeps track of the adjacent nodes is empty.

Routine for breadth first search:

```
def bfs(graph, start):
    visited = set()
    queue = deque([start])  # FIFO queue
    while queue:
        node = queue.popleft()  # Dequeue the first element
        if node not in visited:
            print(node, end=" ")  # Process node
            visited.add(node)
            for neighbor in graph.get (node, []):
                if neighbor not in visited:
                    queue.append (neighbor)
```

Example:

The image shows an **adjacency matrix** representation of an **undirected graph** with four vertices: **A, B, C, and D**.

Adjacency Matrix for the given graph:

	A	B	C	D
A	0	1	1	0
B	1	0	1	1
C	1	1	0	1
D	0	1	1	0

Implementation

1. Let "A" be the source vertex. Mark it to as visited.
2. Find the adjacent unvisited vertices of "A" and enqueue then into the queue. Here B and C are adjacent nodes of A and B and C are enqueued.

B	C	
---	---	-------

3. Then vertex "B" is dequeued and its adjacent vertices C and D are taken from the adjacency matrix for enqueueing. Since vertex C is already in the queue, vertex D alone is enqueued.

C	D	
---	---	-------

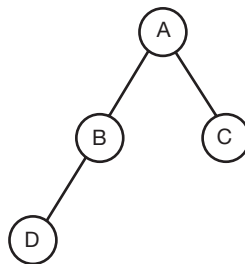
Here B is dequeued, D is enqueued.

4. Then vertex "C" is dequeued and its adjacent vertices A, B and D are found out. Since vertices A and B are already visited and vertex D is also in the queue, no enqueue operation takes place.

D	
---	-------

Here C is dequeued

5. Then vertex "D" is dequeued. This process terminates as all the vertices are visited and the queue is also empty.

**Breadth first spanning tree**

Applications of breadth first search:

- Explores all **neighboring nodes first**, before moving deeper.
- BFS guarantees the **shortest path (minimum edges)** in an **unweighted graph**.
- Works on Both Directed and Undirected Graphs

DAG – Direct Acyclic Graph

A **Directed Acyclic Graph (DAG)** is a directed graph that has **no cycles**. This means that there is no path that starts from a node and eventually returns to the same node.

Key Properties of a DAG:

1. **Directed** – All edges have a direction.
2. **Acyclic** – No cycles exist (i.e., no path leads back to the same node).
3. **Topological Order** – A DAG allows for a **topological sorting**, meaning nodes can be ordered such that for every directed edge ($u \rightarrow v$), u appears before v .

Topological Sort

Topological Sorting is a linear ordering of vertices in a Directed Acyclic Graph (DAG) such that:

- For every directed edge ($u \rightarrow v$), vertex “ u ” appears before vertex “ v ” in the ordering.
- Topological ordering is not possible. If the graph has a cycle, since for two vertices v and w on the cycle, v precedes w and w precedes v .

To implement the topological sort, perform the following steps.

Step 1 : Find the indegree for every vertex.

Step 2: Place the vertices whose indegree is "0" on the empty queue.

Step 3: Dequeue the vertex V and decrement the indegree's of all its adjacent vertices.

Step 4: Enqueue the vertex on the queue, if its indegree falls to zero.

Step 5: Repeat from step3 until the queue becomes empty.

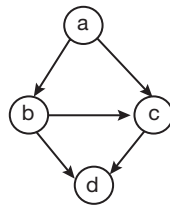
Step 6: The topological ordering is the order in which the vertices dequeued.

Routine to perform Topological Sort:

```

from collections import deque
def topological_sort(graph):
    in_degree = {node: 0 for node in graph}    # Step 1:
Initialize in-degree
    for node in graph:
        for neighbor in graph[node]:
            in_degree[neighbor] += 1    # Compute in-degree
    queue = deque([node for node in graph if in_degree[node]
== 0])
    topo_order = []
    while queue:
        node = queue.popleft()    # Process node
        topo_order.append(node)
        for neighbor in graph[node]:    # Reduce in-degree for
neighbors
            in_degree[neighbor] -= 1
            if in_degree[neighbor] == 0:
                queue.append(neighbor)

```

Example 1:

	a	b	c	d
a	0	1	1	0
b	0	0	1	1
c	0	0	0	1
d	0	0	0	0

Step 1:

Number of 1's present in each column of adjacency matrix represents the Indegree of the corresponding vertex.

Indegree [a]= 0
 Indegree[b]= 1
 Indegree [c]= 2
 Indegree [d] =2

Step 2:

- Enqueue the vertex, whose indegree is "0"
- Vertex "a" is 0, so place it on the queue

Step 3:

Dequeue the vertex "a" from the queue and decrement the indegree's of its adjacent vertex "b" & "c"

Hence, Indegree [b]=0

Indegree [c] =1

Now, Enqueue the vertex "b" as its indegree becomes zero.

Step 4:

Dequeue the vertex "b" from Q and decrement the indegree's of its adjacent vertex "c" and "d".

Hence,

Indegree [c] =0

Indegree [d] =1

Now, Enqueue the vertex "c" as its indegree falls to zero.

Step 5:

Dequeue the vertex "c" from Q and decrement the indegree's of its adjacent vertex "d".

Hence, Indegree [d] =0

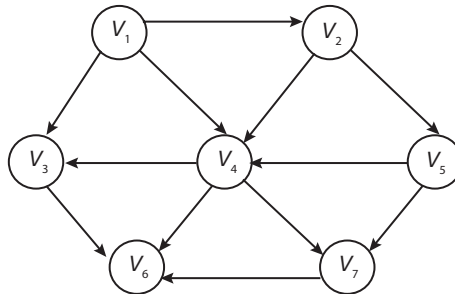
Now, Enqueue the vertex "d" as its indegree falls to zero

Step 6:

Dequeue the vertex "d".

VERTEX	1	2	3	4
a	0	0	0	0
b	1	0	0	0
c	2	1	0	0
d	2	2	1	0
ENQUEUE	a	b	c	d
DEQUEUE	a	b	c	d

The topological ordering is a,b,c,d

Example 2 :**Adjacency Matrix for the given graph:**

	V ₁	V ₂	V ₃	V ₄	V ₅	V ₆	V ₇
V ₁	0	1	1	1	0	0	0
V ₂	0	0	0	1	1	0	0
V ₃	0	0	0	0	0	1	0
V ₄	0	0	1	0	0	1	1
V ₅	0	0	0	1	0	0	1
V ₆	0	0	0	0	0	0	0
V ₇	0	0	0	0	0	1	0
INDEGREE	0	1	2	3	1	3	2

Find indegree of given graph:Indegree[V₁]=0Indegree[V₂]=1Indegree[V₃]=2Indegree[V₄]=3Indegree[V₅]=1Indegree[V₆]=3Indegree[V₇]=2The topological order is V₁, V₂, V₅, V₄, V₃, V₇, V₆

VERTEX	1	2	3	4	5	6	7
V_1	0	0	0	0	0	0	0
V_2	1	0	0	0	0	0	0
V_3	2	1	1	1	0	0	0
V_4	3	2	1	0	0	0	0
V_5	1	1	0	0	0	0	0
V_6	3	3	3	3	2	1	0
V_7	2	2	2	1	0	0	0
INQUEUE	V_1	V_2	V_5	V_4	V_3	V_7	V_6
DEQUEUE	V_1	V_2	V_5	V_4	V_3	V_7	V_6

Minimum Spanning Tree (MST)

A Minimum Spanning Tree (MST) is a subset of edges from a connected, weighted, and un-directed graph that connects all the vertices without forming any cycles and has the minimum possible total edge weight. MSTs are widely used in network design, clustering, and optimizing routes.

Key Properties of MST:

- **Connected Graph:** The graph must be connected to find an MST.
- **Acyclic:** MST cannot contain cycles.
- **n-1 Edges:** For a graph with n vertices, the MST has n-1 edges.
- **Minimum Weight:** The sum of the edge weights in the MST is minimal among all spanning trees.

Algorithms for Finding MST

1. Kruskal's Algorithm

Kruskal's algorithm builds the MST by selecting the smallest edge and adding it to the growing forest while ensuring no cycles are formed.

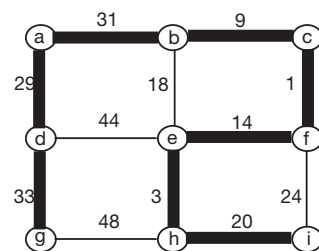
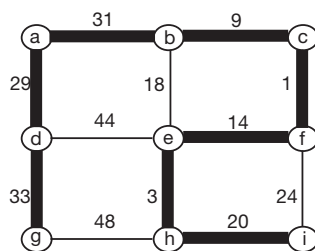
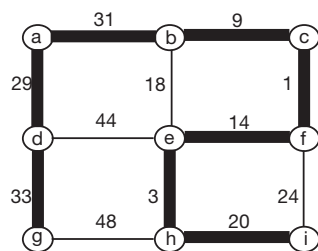
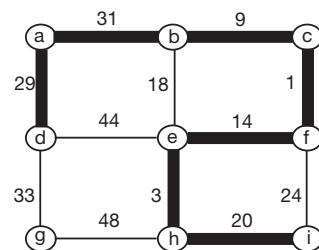
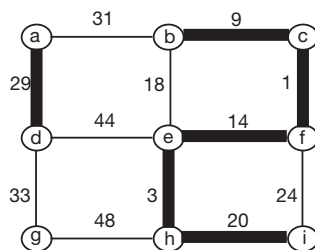
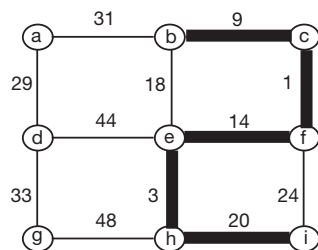
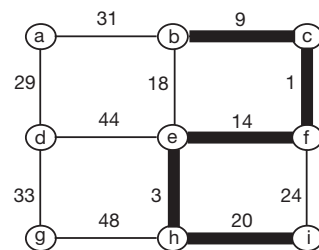
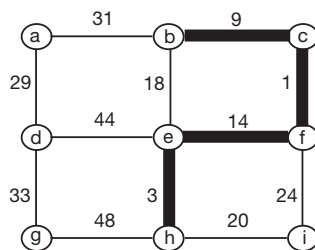
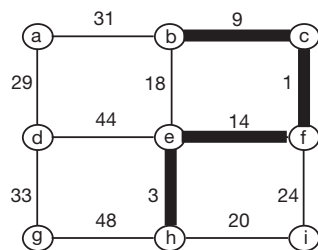
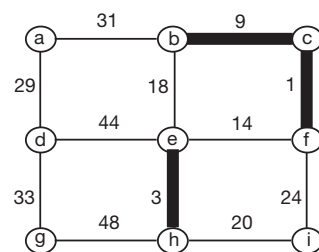
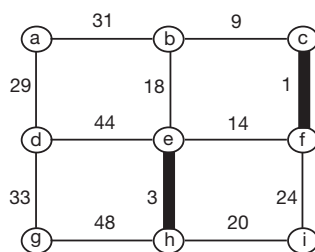
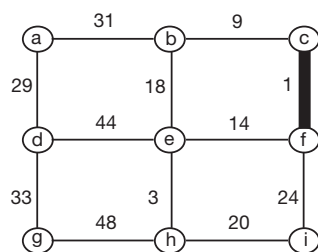
Steps of Kruskal's Algorithm:

1. Sort all edges in non-decreasing order of their weights.
2. Pick the smallest edge. Check if adding it forms a cycle using a Disjoint Set Union (DSU) structure.

3. If no cycle is formed, include the edge in the MST.
4. Repeat until there are $n-1$ edges in the MST.

Python Implementation of Kruskal's Algorithm

```
class DisjointSet:
    def __init__(self, n):
        self.parent = list(range(n))
        self.rank = [0] * n
    def find(self, u):
        if u != self.parent[u]:
            self.parent[u] = self.find(self.parent[u])
        return self.parent[u]
    def union(self, u, v):
        root_u = self.find(u)
        root_v = self.find(v)
        if root_u != root_v:
            if self.rank[root_u] > self.rank[root_v]:
                self.parent[root_v] = root_u
            elif self.rank[root_u] < self.rank[root_v]:
                self.parent[root_u] = root_v
            else:
                self.parent[root_v] = root_u
                self.rank[root_u] += 1
def kruskal(n, edges):
    mst = []
    dsu = DisjointSet(n)
    edges.sort(key=lambda x: x[2])
    for u, v, weight in edges:
        if dsu.find(u) != dsu.find(v):
            dsu.union(u, v)
            mst.append((u, v, weight))
    return mst
# Example Usage
vertices = 4
edges = [
    (0, 1, 10), (0, 2, 6), (0, 3, 5),
    (1, 3, 15), (2, 3, 4)
]
print("MST:", kruskal(vertices, edges))
```



Example for Kruskal's Algorithm

2. Prim's Algorithm

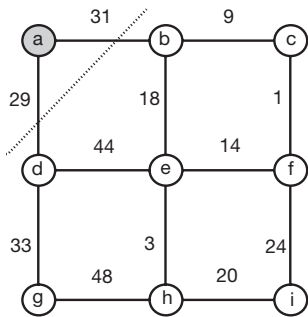
Prim's algorithm grows the MST by starting from an arbitrary vertex and repeatedly adding the smallest edge connecting a new vertex to the existing MST.

Steps of Prim's Algorithm:

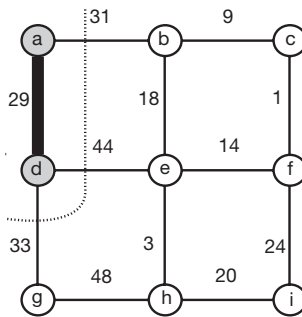
1. Initialize an empty MST.
2. Select an arbitrary starting vertex.
3. Repeatedly add the smallest edge that connects a new vertex.
4. Continue until all vertices are included.

Python Implementation of Prim's Algorithm

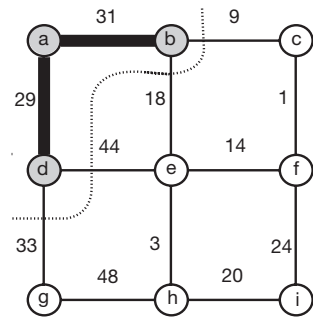
```
import heapq
def prim(n, graph):
    mst = []
    visited = [False] * n
    min_heap = [(0, 0)] # (weight, vertex)
    while len(mst) < n - 1:
        weight, u = heapq.heappop(min_heap)
        if visited[u]:
            continue
        visited[u] = True
        for v, w in graph[u]:
            if not visited[v]:
                heapq.heappush(min_heap, (w, v))
        if u != 0:
            mst.append((u, weight))
    return mst
# Example Usage
graph = {
    0: [(1, 10), (2, 6), (3, 5)],
    1: [(0, 10), (3, 15)],
    2: [(0, 6), (3, 4)],
    3: [(0, 5), (1, 15), (2, 4)]
}
print("MST:", prim(4, graph))
```



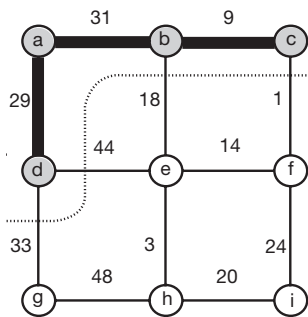
(a) Add a



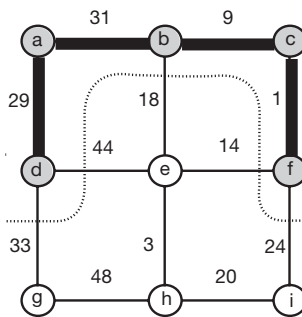
(b) Link to d via (a,d)



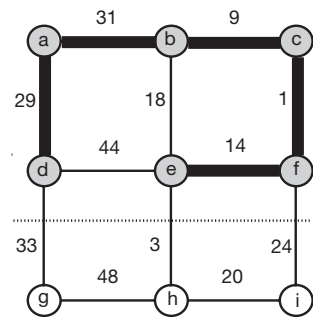
(c) Link to b via (a,b)



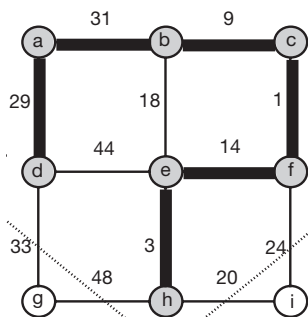
(d) Link to c via (b,c)



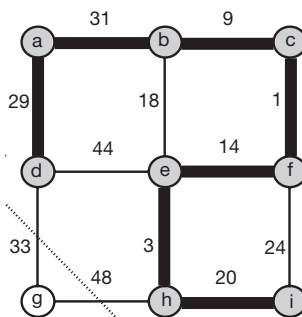
(e) Link to f via (c,f)



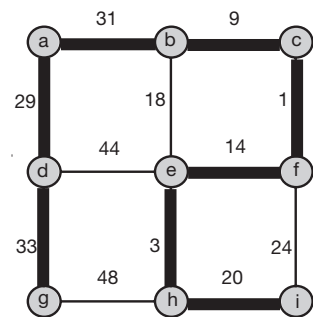
(f) Link to e via (f,e)



(g) Link to h via (e,h)



(h) Link to i via (h,i)



(i) Link to g via (d,g)

Example for Prim's Algorithm

Applications of MST

- Network Design: Designing cost-efficient communication, transportation, and electrical networks.
- Clustering: Identifying natural groupings in data.
- Image Segmentation: Grouping pixels based on similarity.

- Circuit Design: Minimizing the length of interconnecting wires.

Minimum Spanning Tree algorithms like Kruskal's and Prim's provide efficient ways to optimize graph-based problems. Kruskal's algorithm is ideal for sparse graphs due to edge sorting, while Prim's is faster for dense graphs due to its priority queue approach. Both algorithms have broad applications in real-world optimization problems.

Shortest Path

Dijkstra's Algorithm: Shortest Path Calculation

Dijkstra's Algorithm is a classic algorithm used to find the shortest path from a source node to all other nodes in a weighted graph. It operates by progressively finding the shortest known path to each node, making it useful for various applications like navigation systems, network routing, and robotics.

Key Characteristics:

- Works only on graphs with **non-negative weights**.
- Solves the **single-source shortest path** problem.
- Utilizes a **greedy** approach by expanding the nearest unvisited node.

Explanation of Dijkstra's Algorithm

The algorithm maintains a set of unvisited nodes and calculates the tentative distances to each node. It iterates by selecting the node with the smallest tentative distance, updating its neighboring nodes, and marking it as visited.

Steps of Dijkstra's Algorithm

1. Initialization:

- Set the distance to the source node as **0** and all other nodes as **infinity**.
- Create a priority queue to store nodes based on their distances.

2. Exploration:

- Extract the node with the **smallest** distance from the priority queue.
- For each neighboring node, calculate the new distance through the current node.
- If the new distance is **smaller**, update it and add the neighbor to the queue.

3. Completion:

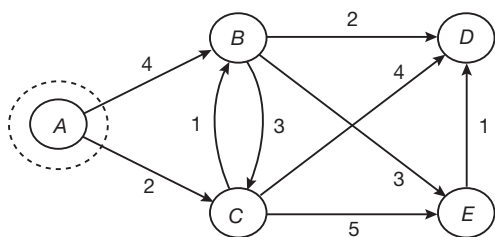
- Repeat until all nodes are visited or the queue is empty.
- The result is the shortest path from the source node to all other nodes.

Python Code for Dijkstra's Algorithm

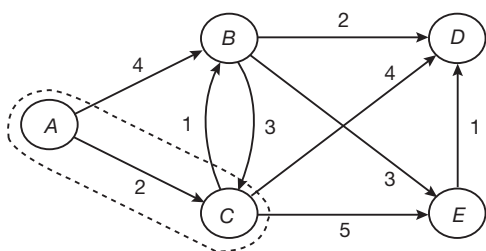
```
import heapq
def dijkstra(graph, start):
    # Initialize distances with infinity
    distances = {vertex: float('infinity') for vertex in graph}
    distances[start] = 0
    # Priority queue to select the node with the smallest
    distance
    priority_queue = [(0, start)]
    while priority_queue:
        current_distance, current_vertex = heapq.
heappop(priority_queue)
        # Skip if the current distance is not optimal
        if current_distance > distances[current_vertex]:
            continue
        # Explore neighbors
        for neighbor, weight in graph[current_vertex].items():
            distance = current_distance + weight
            # Update the shortest path if a better path is
            found
            if distance < distances[neighbor]:
                distances[neighbor] = distance
                heapq.heappush(priority_queue, (distance,
neighbor))
    return distances
# Example graph
graph = {
    'A': {'B': 4, 'C': 5, 'D': 2},
    'B': {'A': 4, 'C': 1, 'D': 7},
    'C': {'A': 5, 'B': 1},
    'D': {'A': 2, 'B': 7, 'E': 3},
    'E': {'D': 3}
}
# Find shortest paths from vertex 'A'
print(dijkstra(graph, 'A'))
Output:
{'A': 0, 'B': 4, 'C': 5, 'D': 2, 'E': 5}
```

Complexity Analysis

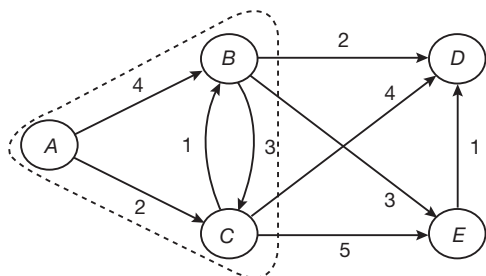
- **Time Complexity:** due to the use of a priority queue (where n is the number of vertices and m is the number of edges).
- **Space Complexity:** for storing graph data and distances.



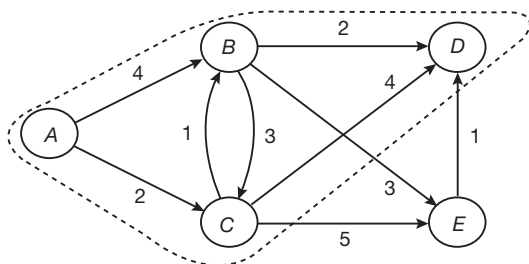
A:0	D:∞
B:4	E:∞
C:2	



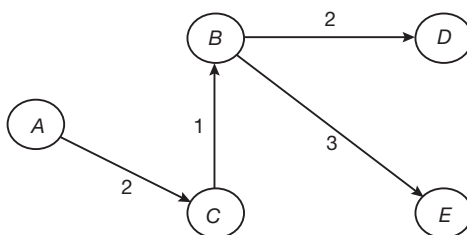
A:0	D:6
B:3	E:7
C:2	



A:0	D:5
B:3	E:6
C:2	



A:0	D:5
B:3	E:6
C:2	



Example of Dijkstra's Algorithm

Applications of Dijkstra's Algorithm

1. **Network Routing:** Optimizing packet delivery paths in computer networks.
2. **Navigation Systems:** Calculating the shortest driving routes for GPS.
3. **Robotics:** Path planning for autonomous vehicles and drones.
4. **Telecommunications:** Minimizing the cost of data transmission.
5. **Scheduling:** Optimizing resource allocation in large systems.

Dijkstra's Algorithm is a foundational method for solving the shortest path problem in weighted graphs. Its efficiency and reliability make it applicable across a broad range of real-world problems in technology, transportation, and communication systems.

Disjoint Sets

A disjoint set is a collection of sets where no two sets share any common elements. Formally, two sets A and B are disjoint if their intersection is empty:

$$A \cap B = \emptyset$$

A Disjoint Set Data Structure, also known as Union-Find, is a data structure that keeps track of a partition of a set into disjoint (non-overlapping) subsets. It supports two key operations efficiently:

1. **Find(x)** – Determines which set an element belongs to (finds the representative of the set).
2. **Union(x, y)** – Merges the sets containing xxx and yyy.

Routine for implementation:

```
class DisjointSet:
    def __init__(self, n):
        self.parent = [i for i in range(n)] # Each element is
        # its own parent initially
        self.rank = [1] * n # Rank used for union by rank
    def find(self, x):
        if self.parent[x] != x: # Path compression
            self.parent[x] = self.find(self.parent[x])
        return self.parent[x]
    def union(self, x, y):
        rootX = self.find(x)
        rootY = self.find(y)
        if rootX != rootY: # Only merge if they are in dif-
            # ferent sets
            if self.rank[rootX] > self.rank[rootY]: # Attach
                # smaller tree to larger
                self.parent[rootY] = rootX
```

```

        elif self.rank[rootX] < self.rank[rootY]:
            self.parent[rootX] = rootY
        else:
            self.parent[rootY] = rootX
            self.rank[rootX] += 1 # Increase rank if same
size

def connected(self, x, y):
    return self.find(x) == self.find(y)

```

Key Concepts:

- **Representative (Leader):** Each set has a leader (or representative) element.
- **Path Compression:** A technique to flatten the structure of the tree whenever Find is called, improving efficiency.
- **Union by Rank/Size:** A technique to always attach the smaller tree under the larger one to keep the tree shallow.

Examples:

1. $A=\{1,2,3\}$ and $B=\{4,5,6\}$ are disjoint because they have no common elements.
2. $A=\{a,b,c\}$ and $B=\{b,c,d\}$ are not disjoint because they share elements bbb and ccc.

Applications of Disjoint Sets:

- Set Theory: Used in mathematical proofs and logic.
- Graph Theory: In connected components of graphs.
- Disjoint Set Data Structure (Union-Find): Helps in network connectivity problems, Kruskal's algorithm for Minimum Spanning Trees, and dynamic connectivity queries.